通用图形处理器

体系结构

General-Purpose Graphics Processor Architectures

Tor M. Aamodt

Wilson Wai Lun Fung

Timothy G. Rogers

中文修订学习版

根据 Morgan & Claypool Publishers 于 2018 年出版的英文原著整理；非官方授权译本。

译稿说明

本书的技术表述以 2018 年英文原著为准。保留原作者、原始引用及指令级技术示例。本次修订重点处理章节编号、重复页眉页码、图表结构、代码误识别及中文排版，不意味着已经完成出版级逐句人工审校。

摘要

图形处理器（GPU）最初为电子游戏的图形渲染而开发，如今越来越多地用于机器学习、加密货币挖矿等通用（非图形）应用。相较于中央处理器（CPU），GPU 将更大比例的硬件资源用于计算，因此能够获得更高的性能与能效。此外，与特定领域加速器相比，现代 GPU 的通用可编程性也对软件开发者具有吸引力。本书面向希望研究支持通用计算的 GPU 体系结构的读者，汇集了此前散见于多种不同来源的相关资料。本书作者主持开发了广泛用于 GPU 体系结构学术研究的 GPGPU-Sim 模拟器。

第一章介绍 GPU 的基本硬件结构并简述其发展历程。第二章概述理解后续内容所需的 GPU 编程模型。第三章讨论 GPU 计算核心的体系结构；第四章讨论 GPU 内存系统的体系结构。这两章在介绍现有系统后，还综述相关研究。第五章总结同时影响计算核心和内存系统的跨领域研究。

本书旨在为希望理解用于加速通用应用的 GPU 体系结构的读者提供参考，也为希望了解如何改进这些 GPU 的快速增长的研究文献的读者提供入门指导。

关键词： GPGPU；计算机体系结构

目录

译稿说明 ii

摘要 iii

前言 vi

致谢 vii

第一章 引言 1
- 1.1 计算加速器的格局 . . . 1
- 1.2 GPU 硬件基础 . . . 2
- 1.3 GPU 简史 . . . 5
- 1.4 本书大纲 . . . 6

第二章 编程模型 7
- 2.1 执行模型 . . . 7
- 2.2 GPU 指令集架构 . . . 11
  - 2.2.1 NVIDIA GPU 指令集架构 . . . 11
  - 2.2.2 AMD 图形核心下一代指令集架构 . . . 13

第三章 SIMT 核心：指令与寄存器数据流 17
- 3.1 单循环近似 . . . 18
  - 3.1.1 SIMT 执行掩码 . . . 18
  - 3.1.2 SIMT 死锁与无栈 SIMT 架构 . . . 21
  - 3.1.3 WARP 调度 . . . 26
- 3.2 双循环近似 . . . 28
- 3.3 三循环近似 . . . 29
  - 3.3.1 操作数收集器 . . . 32
  - 3.3.2 指令重放：处理结构冒险 . . . 34
- 3.4 分支发散的研究方向 . . . 35
  - 3.4.1 WARP 压缩 . . . 36
  - 3.4.2 Warp 内分歧路径管理 . . . 39
  - 3.4.3 添加 MIMD 能力 . . . 44
  - 3.4.4 低复杂度的高效分支发散管理 . . . 45
- 3.5 标量化与仿射执行的研究方向 . . . 47
  - 3.5.1 统一或仿射变量的检测 . . . 48
  - 3.5.2 利用 GPU 中的均匀或仿射变量 . . . 50
- 3.6 寄存器文件架构的研究方向 . . . 52
  - 3.6.1 分层寄存器文件 . . . 52
  - 3.6.2 休眠状态寄存器文件 . . . 53
  - 3.6.3 寄存器文件虚拟化 . . . 53
  - 3.6.4 分区寄存器文件 . . . 54
  - 3.6.5 RegLess . . . 54

第四章 内存系统 56

4.1 第一级内存结构 . . . 56
4.1.1 暂存器内存和 L1 数据缓存 . . . 57
4.1.2 L1 纹理缓存 . . . 60
4.1.3 统一纹理与数据缓存 . . . 61
4.2 片上互连网络 . . . 61
4.3 内存分区单元 . . . 63
4.3.1 L2 缓存 . . . 63
4.3.2 原子操作 . . . 63
4.3.3 内存访问调度器 . . . 63
4.4 GPU 内存系统的研究方向 . . . 65
4.4.1 内存访问调度与互连网络设计 . . . 65
4.4.2 缓存有效性 . . . 65
4.4.3 内存请求优先级和缓存绕过 . . . 66
4.4.4 利用线程束间异构性 . . . 67
4.4.5 协同缓存旁路 . . . 68
4.4.6 自适应缓存管理 . . . 68
4.4.7 缓存优先级 . . . 69
4.4.8 虚拟内存页面放置 . . . 69
4.4.9 数据放置 . . . 70
4.4.10 多芯片模块 GPU . . . 70

第五章 GPU 计算体系结构的跨领域研究 **72**
5.1 线程调度 . . . 72
5.1.1 线程块到核心分配的研究 . . . 72
5.1.2 逐周期调度决策的研究 . . . 74
5.1.3 多内核调度研究 . . . 77
5.1.4 细粒度同步感知调度 . . . 78
5.2 表达并行性的其他方式 . . . 79
5.3 对事务内存的支持 . . . 80
5.3.1 KILO TM . . . 81
5.3.2 线程束 TM 与时间冲突检测 . . . 82
5.4 异构系统 . . . 83

参考文献 **85**

作者传记 **98**

前言

本书面向希望理解图形处理器（GPU）体系结构，并入门了解探索如何改进其设计的日益增多的研究文献的读者。假设读者熟悉流水线、缓存等计算机体系结构概念，并有兴趣开展与 GPU 体系结构相关的研究和/或开发工作。此类工作往往侧重于不同设计之间的权衡，因此本书旨在提供对这些权衡的洞见，使读者无需通过试错来学习经验丰富的设计者已知的内容。

为实现这一目标，本书将目前散见于专利、产品文档和研究论文等广泛不同来源的许多相关信息汇集于一处。我们希望这有助于减少刚开始从事研究的学生或从业者变得高效所需的时间。

虽然本书涵盖当前 GPU 设计的各个方面，但也试图“综合”已发表的研究。部分原因是不得已而为之，因为供应商很少谈及特定 GPU 产品的微体系结构。在描述“基线”GPGPU 体系结构时，本书既依赖已发布的产品描述（期刊论文、白皮书、手册），在某些情况下也依赖专利中的描述。专利中的细节可能与实际产品的微体系结构存在重大差异。在某些情况下，微基准研究为研究人员澄清了一些细节，但在其他情况下，我们的基线代表了基于公开信息的“最佳猜测”。尽管如此，我们相信这将有所帮助，因为我们的重点是理解已被研究过或未来研究可能感兴趣的体系结构权衡。

本书的若干部分着重总结近年来关于改进 GPU 体系结构的众多研究论文。由于该主题近年来热度激增，本书无法尽数涵盖。因此，我们不得不做出艰难取舍，决定哪些内容纳入、哪些内容略去。

Tor M. Aamodt、Wilson Wai Lun Fung、Timothy G. Rogers　2018 年 4 月

致谢

我们要感谢家人在本书写作期间给予的支持。此外，我们感谢出版商 Michael Morgan 和编辑 Margaret Martonosi，在本书成稿过程中展现了极大的耐心。我们还感谢 Carole-Jean Wu、Andreas Moshovos、Yash Ukidave、Aamir Raihan 和 Amruth Sandhupatla 对本书早期草稿提供了详细反馈。最后，我们感谢 Mark Hill 分享了他关于撰写 Synthesis Lectures 系列著作策略的思考以及对本书的具体建议。

Tor M. Aamodt、Wilson Wai Lun Fung 和 Timothy G. Rogers 2018 年 4 月

第一章　引言

本书探讨图形处理器单元（GPU）的硬件设计。GPU 最初是为了实现实时渲染而引入的，主要面向视频游戏。如今，从智能手机、笔记本电脑、数据中心一直到超级计算机，GPU 无处不在。事实上，对 Apple A8 应用处理器的分析表明，其集成 GPU 所占的芯片面积超过了中央处理器单元（CPU）核心所占的面积 [$A8H$]。对更逼真图形渲染的需求是 GPU 创新的最初驱动力 [$Montrym\ and\ Moreton$, 2005]。尽管图形加速仍然是 GPU 的主要用途，但 GPU 越来越多地支持非图形计算。当前备受关注的一个突出例子是 GPU 在开发和部署机器学习系统中的日益广泛使用 [$NVIDIA\ Corp.$, 2017]。因此，本书的重点在于与提升非图形应用性能和能效相关的特性。

本导论章对 GPU 进行简要概述。我们从第 1.1 节开始，考察更广泛的计算加速器类别的动机，以理解 GPU 与其他选项的比较。然后，在第 1.2 节中，我们快速概述当代 GPU 硬件。最后，第 1.4 节提供本书其余部分的路线图。

1.1　计算加速器的格局

几十年来, 一代又一代计算系统的每美元性能呈指数级增长。其根本原因在于晶体管尺寸缩小、硬件体系结构改进、编译器技术进步以及算法改进的综合作用。据某些估计, 这些性能提升中有一半归因于晶体管尺寸缩小带来的器件运行速度提升 [Hennessy and Patterson，2011]。然而，大约自 2005 年以来，晶体管缩放未能遵循如今被称为 Dennard Scaling 的经典规律 [Dennard et al., 1974]。一个关键后果是，随着器件变小，时钟频率的提升速度大大放缓。要提升性能，就需要找到更高效的硬件体系结构。

通过利用硬件专用化,可以将能效提升高达 $500\times$[Hameed et al., 2010]。正如 Hameed 等人所示，实现如此效率提升有几个关键方面。转向向量硬件（如 GPU 中的硬件）通过消除指令处理开销，可获得约 $10\times$ 的效率提升。硬件专用化剩余收益的很大一部分来自最小化数据移动，这

可以通过引入复杂操作来实现，这些操作执行多个算术运算，同时避免访问大型存储阵列（如寄存器堆）。

当今计算机体系结构设计师面临的一个关键挑战是，找到更好的方法来平衡使用专用硬件可获得的效率提升与支持广泛程序所需的灵活性需求。在缺乏体系结构的情况下，只有可用于大量应用的算法才能高效运行。一个新兴的例子是专用于支持深度神经网络的硬件，如谷歌的张量处理单元 [Jouppi et al., 2017]。虽然机器学习似乎可能占据计算硬件资源的很大一部分，并且这些资源可能会迁移到专用硬件上，但我们认

为，仍然需要高效支持以传统编程语言编写的软件所表达的计算。

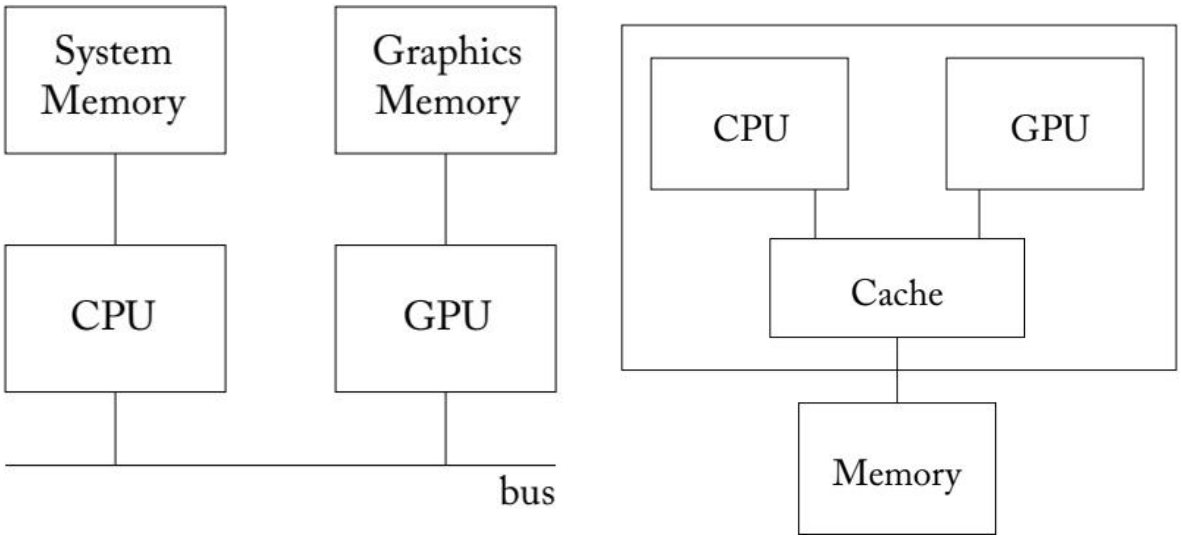
在机器学习之外，人们对 GPU 计算产生浓厚兴趣的一个原因是，现代 GPU 支持图灵完备的编程模型。所谓图灵完备，是指只要有足够的时间和内存，任何计算都可以运行。相对于专用加速器，现代 GPU 具有灵活性。对于能够充分利用 GPU 硬件的软件，GPU 的效率可比 CPU 高一个数量级 [Lee et al., 2010]。这种灵活性与效率的结合非常理想。因此，许多顶级超级计算机（无论是在峰值性能还是能效方面）现在都采用 GPU[top]。在后续几代产品中，GPU 制造商不断完善 GPU 体系结构和编程模型，以在提高能效的同时增加灵活性。

1.2　GPU 硬件基础

初次接触 GPU 的人常常会问，GPU 最终是否会完全取代 CPU。这似乎不太可能。在当前的系统中，GPU 并非独立的计算设备。相反，它们与 CPU 结合，要么集成在单芯片上，要么通过将仅含 GPU 的附加卡插入包含 CPU 的系统。CPU 负责在 GPU 上启动计算，并在 GPU 与系统之间传输数据。CPU 与 GPU 之间这种分工的一个原因是，计算的开始和结束通常需要访问输入/输出（I/O）设备。尽管目前正在努力开发直接在 GPU 上提供 I/O 服务的应用编程接口（API），但迄今为止这些接口都假设存在邻近的 CPU[Kim et al., 2014，Silberstein et al., 2013]。这些 API 通过提供便捷的接口来隐藏 CPU 与 GPU 之间通信管理的复杂性，而非完全消除对 CPU 的需求。为什么不消除 CPU？用于访问 I/O 设备以及提供操作系统服务的软件似乎缺乏诸如大规模并行性之类的特性，而这些特性才能使它们适合在 GPU 上运行。因此，我们首先考虑 CPU 与 GPU 的交互。

图 1.1 展示了一个包含 CPU 和 GPU 的典型系统的抽象示意图。左侧是典型的独立 GPU 配置，包括连接 CPU 和 GPU 的总线（例如 PCIe），适用于 NVIDIA 的 Volta GPU 等架构；右侧是典型的集成 CPU 和 GPU 的逻辑图，例如 AMD 的 Bristol Ridge APU 或移动 GPU。注意，包含独立 GPU 的系统为 CPU（通常称为系统内存）和 GPU（通常称为设备内存）设有独立的 DRAM 内存空间。用于这些内存的 DRAM 技术通常不同（CPU 使用 DDR，GPU 使用 GDDR）。CPU DRAM 通常针对低延迟访问进行优化，而 GPU DRAM 则针对高吞吐量进行优化。相比之下，集成 GPU 的系统具有单一的 DRAM 内存空间，因此必然使用相同的内存技术。由于集成 CPU 和 GPU 常见于低功耗移动设备，共享的 DRAM 内存通常针对低功耗进行优化（例如 LPDDR）。

GPU 计算应用程序从 CPU 上开始运行。通常，应用程序的 CPU 部分会分配并初始化一些数据结构。在 NVIDIA 和 AMD 的较旧独立 GPU 上，GPU 计算应用程序的 CPU 部分通常会在 CPU 和 GPU 内存中为数据结构分配空间。对于这些 GPU，应用程序的 CPU 部分必须协调数据从 CPU 内存到 GPU 内存的移动。较新的独立 GPU（例如 NVIDIA 的 Pascal 架构）具有软件和硬件支持，可自动将数据从 CPU 内存传输到 GPU 内存。这可以通过利用 CPU 和 GPU 上的虚拟内存支持来实现 [Gelado et al., 2010]。



（a）独立 GPU 系统　　（b）集成 CPU-GPU 系统

图 1.1　包含 CPU 的 GPU 计算系统。

NVIDIA 称之为“统一内存”。在 CPU 和 GPU 集成在同一芯片上并共享同一内存的系统中，无需程序员控制从 CPU 内存到 GPU 内存的复制。然而，由于 CPU 和 GPU 使用缓存，并且

其中一些缓存可能是私有的，因此可能存在缓存一致性问题，硬件开发者需要解决这个问题 [Power et al., 2013b]。

在某个时刻，CPU 必须启动 GPU 上的计算。在当前系统中，这是通过运行在 CPU 上的驱动程序来完成的。在启动 GPU 上的计算之前，GPU 计算应用程序指定哪些代码应在 GPU 上运行。这段代码通常称为内核（更多细节见第 2 章）。同时，GPU 计算应用程序的 CPU 部分还指定应运行多少线程以及这些线程应在何处查找输入数据。要运行的内核、线程数量和数据位置通过运行在 CPU 上的驱动程序传送到 GPU 硬件。驱动程序将转换这些信息，并将其放置在 GPU 可访问的内存中，即 GPU 配置为查找的位置。然后，驱动程序向 GPU 发出信号，表示有新的计算需要运行。

现代 GPU 由许多核心组成，如图 1.2 所示。NVIDIA 称这些核心为流式多处理器，AMD 称它们为计算单元。每个 GPU 核心执行与已启动在 GPU 上运行的内核相对应的单指令多线程（SIMT）程序。GPU 上的每个核心通常可以运行约一千个线程。在单个核心上执行的线程可以通过暂存器内存进行通信，并使用快速屏障操作进行同步。每个核心通常还包含一级指令和数据缓存。这些缓存充当带宽过滤器，以减少发送到内存系统较低层次的通信量。核心上运行的大量线程用于在数据未在一级缓存中找到时隐藏访问内存的延迟。

为了维持高计算吞吐量，有必要在高计算吞吐量和高内存带宽之间取得平衡。这反过来又需要内存系统中的并行性。

在 GPU 中，这种并行性通过包含多个内存通道来提供。通常，每个内存通道在内存分区中关联一部分末级缓存。GPU 核心和内存分区通过片上互连网络（如交叉开关）

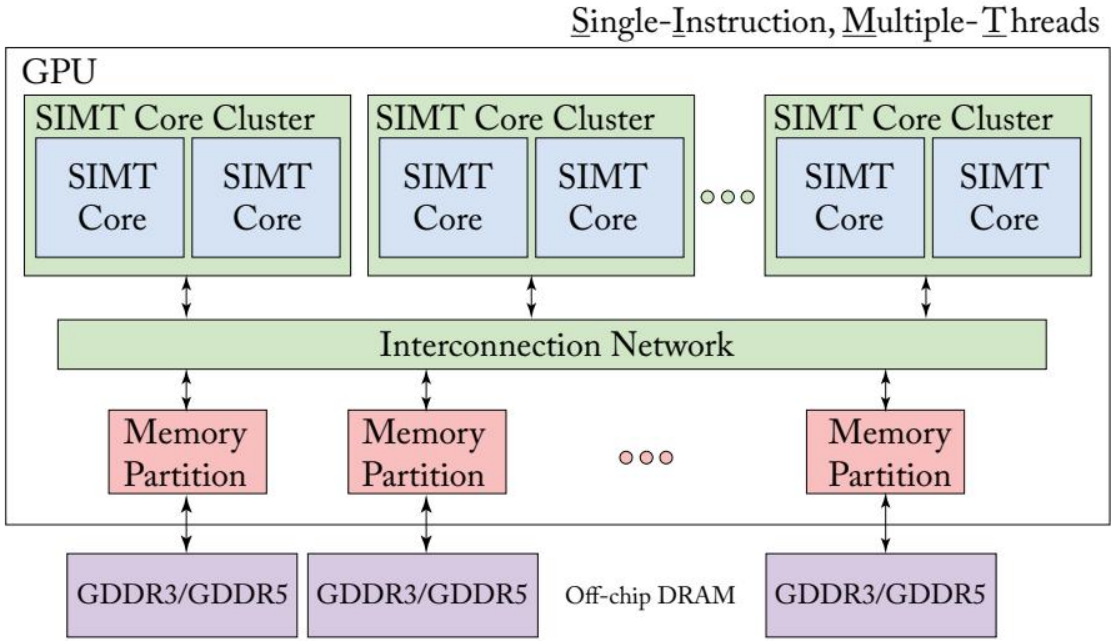


图 1.2　通用的现代 GPU 架构。

连接。其他组织方式也是可能的。例如，在超级计算市场上与 GPU 直接竞争的 Intel Xeon Phi 将末级缓存与核心分布在一起。

在高并行工作负载上，与超标量乱序 CPU 相比，GPU 通过将更大比例的芯片面积专用于算术逻辑单元并相应减少控制逻辑面积，可以获得更高的单位面积性能。为了培养对 CPU 和 GPU 架构之间权衡的直觉，Guz 等人 [2009] 开发了一个富有洞察力的分析模型，展示了性能如何随线程数量变化。为了保持模型简单，他们假设了一个简单的缓存模型，其中线程不共享数据且片外内存带宽无限。图 1.3 再现了他们论文中的一张图，说明了他们通过模型发现的一个有趣的权衡。当大型缓存由少量线程共享时（如多核 CPU 的情况），性能随线程数量增加而提高。然而，如果线程数量增加到缓存无法容纳整个工作集的程度，性能就会下降。随着线程数量进一步增加，性能因多线程隐藏长片外延迟的能力而提高。GPU 架构由图右侧表示。GPU 设计为通过采用多线程来容忍频繁的缓存未命中。

随着登纳德缩放定律的终结 [Horowitz et al., 2005]，提高能效已成为计算机体系结构研究创新的主要驱动力。一个关键观察是，访问大型内存结构所消耗的能量可能与计算相当，甚至更多。

例如, 表 1.1 提供了 45 纳米工艺技术下各种操作的能耗数据 [Han et al., 2016]。在提出新颖的 GPU 架构设计时，考虑能耗非常重要。为了帮助实现这一点，近期的 GPGPU 架构模拟器（如 GPGPU-Sim [Bakhoda et al., 2009]）集成了能耗模型 [Leng et al., 2013]。

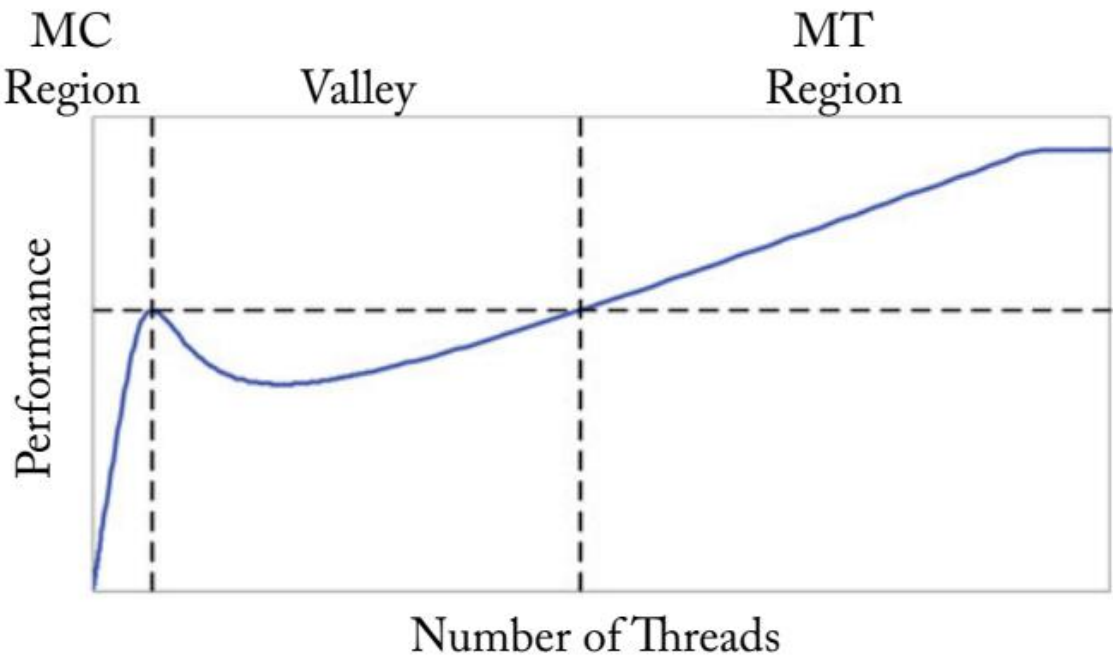


图 1.3 基于分析模型的多核（MC）CPU 架构与多线程（MT）架构（如 GPU）之间的性能权衡分析显示，如果线程数量不足以覆盖片外内存访问延迟，可能会出现“性能低谷”（基于 Guz 等人 [2009] 的图 1）。

表 1.1 45 nm 工艺下各类操作的能耗及相对成本（据 Han 等，2016）。

操作	能耗（pJ）	相对成本
32 位整数加法	0.1	1
32 位浮点加法	0.9	9
32 位整数乘法	3.1	31
32 位浮点乘法	3.7	37
32 位 SRAM（32 KB）	5	50
32 位 DRAM	640	6400

1.3 GPU 简史

本节简要介绍图形处理单元的历史。计算机图形学兴起于 20 世纪 60 年代，代表性项目包括 Ivan Sutherland 的 Sketchpad [Sutherland，1963]。从早期开始，计算机图形学就一直是电影动画离线渲染不可或缺的一部分，同时并行地推动了视频游戏中实时渲染的发展。早期的视频卡始于 1981 年的 IBM 单色显示适配器（MDA），它仅支持文本。后来，视频卡引入了 2D 加速，随后是 3D 加速。除了视频游戏，3D 加速器还面向计算机辅助设计。早期的 3D 图形处理器（如 NVIDIA GeForce 256）相对来说是固定功能的。NVIDIA 在 GeForce 3 中通过顶点着色器 [Lindholm et al., 2001] 和像素着色器为 GPU 引入了可编程性，该产品于 2001 年推出。研究人员很快学会了如何利用这

些早期 GPU 实现线性代数，方法是将矩阵数据映射到纹理并应用着色器 [Krüger and Westermann，2003]，随后学术界致力于将通用计算映射到 GPU 上，使程序员无需了解图形学 [Buck et al., 2004]。这些努力启发了 GPU 制造商在图形处理之外直接支持通用计算。第一个这样做的商业产品是 NVIDIA GeForce 8 系列。GeForce 8 系列引入了多项创新，包括从着色器写入任意内存地址的能力，以及限制片外带宽的暂存器内存，这些在早期 GPU 中一直缺失。下一个创新是通过 NVIDIA 的 Fermi 架构实现读写数据的缓存。随后的改进包括 AMD 的 Fusion 架构，它将 CPU 和 GPU 集成在同一芯片上，以及动态并行性，允许从 GPU 本身启动线程。最近，NVIDIA 的 Volta 引入了专门针对机器学习加速的 Tensor Cores 等功能。

1.4　本书大纲

在设计硬件时，考虑其将支持的软件非常重要。因此，在第 2 章中，我们简要总结编程模型、代码开发过程和编译流程。

在第 3 章中，我们探讨支持数千线程执行的单个 GPU 核心的架构。我们逐步深入理解支持高吞吐量和灵活编程模型所涉及的权衡。本章最后总结了与 GPU 核心架构相关的近期研究，以帮助该领域的新人快速跟上进度。

在第 4 章中，我们探讨内存系统，包括 GPU 核心中的一级缓存以及内存分区的内部组织。理解 GPU 的内存系统非常重要，因为在 GPU 上运行的计算通常受限于片外内存带宽。本章最后总结了与 GPU 内存系统架构相关的近期研究。

最后，第 5 章概述了 GPU 计算架构的其他研究，这些研究不能完全归入第 3 章或第 4 章。

第二章　编程模型

本章的目标是提供足够的背景知识，说明 GPU 如何针对非图形计算进行编程，以便没有 GPU 使用经验的读者能够理解后续章节的讨论。我们在此聚焦于基本材料，更深入的介绍留给其他参考文献（例如 [Kirk and Wen-Mei，2016]）。存在许多可用于架构研究的 GPU 计算基准测试套件。学习 GPU 如何编程对于对 GPU 计算感兴趣的计算机架构师来说是有益的，可以更好地理解硬件/软件接口，但如果您想探索作为研究的一部分对硬件/软件接口进行更改，那么这变得至关重要。在后一种情况下，现有的基准测试可能不存在，因此可能需要创建，或许通过修改现有 GPU 计算应用程序的源代码来实现。例如，探索在 GPU 上引入事务内存（TM）的研究就需要这样做，因为当前的 GPU 不支持 TM（见第 5.3 节）。

现代 GPU 采用宽 SIMD 硬件来利用 GPU 应用中的数据级并行性。GPU 计算 API（如 CUDA 和 OpenCL）没有直接将这种 SIMD 硬件暴露给程序员，而是具有类似 MIMD 的编程模型，允许程序员将大量标量线程启动到 GPU 上。这些标量线程中的每一个都可以遵循其独特的执行路径，并可以访问任意内存位置。在运行时，GPU 硬件在 SIMD 硬件上以锁步方式执行称为 warp（在 AMD 术语中称为 wavefront）的标量线程组，以利用它们的规律性和空间局部性。这种执行模型称为单指令多线程（SIMT）[Lindholm et al., 2008a，Nickolls and Reusch，1993]。

本章其余部分将扩展这一讨论，组织如下。在第 2.1 节中，我们探讨近期 GPU 编程模型使用的概念执行模型，并简要总结过去十年发布的典型 GPU 的执行模型。在第 2.2 节中，我们探讨 GPU 计算应用程序的编译过程，并简要介绍 GPU 指令集架构。

2.1　执行模型

GPU 计算应用程序从 CPU 上开始执行。对于独立 GPU，应用程序的 CPU 部分通常会为 GPU 上的计算分配内存，然后启动将输入数据传输到 GPU 内存，最后在 GPU 上启动计算内核。对于集成 GPU，仅最后一步是必需的。一个计算内核

由（通常）数千个线程组成。每个线程执行相同的程序，但根据计算结果可能遵循不同的控制流。下面我们使用一个用 CUDA 编写的具体代码示例来详细考虑这一流程。在下一节中，我们将从汇编层面审视执行模型。我们的讨论不会过多关注 GPU 编程模型的性能方面。然而，Seo 等人 [2011] 在 OpenCL（一种类似于 CUDA 的编程模型，可编译到多种架构）的背景下提出了一个有趣的观察：针对一种架构（例如 GPU）精心优化的代码在另一种架构（例如 CPU）上可能表现不佳。

图 2.1 给出了一个 CPU 实现的 C 代码，用于执行众所周知的单精度标量值 A 乘以向量值 X 加上向量值 Y 的操作，称为 SAXPY。SAXPY 是著名的基础线性代数软件（BLAS）库 [Lawson et al., 1979] 的一部分，并且对于实现更高级的矩阵运算（如高斯消元法 [McCool et al., 2012]）很有用。鉴于其简单性和实用性，在计算机体系结构教学中经常被用作示例 [Hennessy and Patterson，2011]。图 2.2 提供了相应的 CUDA 版本的 SAXPY，将执行分布在 CPU 和 GPU 上。

图 2.2 中的示例展示了 CUDA 及相关编程模型（例如 OpenCL [Kaeli et al., 2015]）提供的抽象。代码从函数 main（）开始执行。为了使示例聚焦于 GPU 计算的特定细节，我们省略了分配和初始化数组 x 和 y 的细节。接下来，调用函数 saxpy_serial。该函数将向量 x 和 y 中的元素数量作为参数 n，标量值作为参数 a，以及指向用于表示向量 x 和 y 的数组的指针作为输入参数。该函数遍历数组 x 和 y 的每个元素。在每次迭代中，第 4 行的代码使用循环变量 i 读取值 x[i] 和 y[i]，将 x[i] 乘以 a 然后加上 y[i]，然后用结果更新 x[i]。为简单起见，我们省略了 CPU 如何使用函数调用结果的细节。

接下来，我们考虑 SAXPY 的 CUDA 版本。与传统的 C 或 C++ 程序类似，图 2.2 中的代码通过在 CPU 上运行函数 main（）开始执行。我们不会逐行讲解这段代码，而是首先强调 GPU 执行特有的方面。

在 GPU 上执行的线程是由函数指定的计算内核的一部分。在图 2.2 所示的 SAXPY 的 CUDA 版本中，第 1 行的 CUDA 关键字 __global__ 表示内核函数 saxpy 将在 GPU 上运行。在图 2.2 的示例中，我们并行化了图 2.1 中的“for”循环。具体来说，图 2.1 中原始仅 CPU 的 C 代码中第 4 行“for”循环的每次迭代被转换为一个单独的线程，运行图 2.2 中第 3-5 行的代码。一个计算内核通常由数千个线程组成，每个线程都从运行相同的函数开始。在我们的示例中，CPU 在第 17 行使用 CUDA 的内核配置语法在 GPU 上启动计算。内核配置语法看起来很像 C 中的函数调用，但附加了一些信息，指定了包含的线程数量

```
void saxpy_serial(int n, float a, float *x, float *y)
{
  for (int i = 0; i < n; ++i)
    y[i] = a*x[i] + y[i];
}
main() {
  float *x, *y;
  int n;
  // omitted: allocate CPU memory for x and y and initialize contents
  saxpy_serial(n, 2.0, x, y); // Invoke serial SAXPY kernel
  // omitted: use y on CPU, free memory pointed to by x and y
}
```

图 2.1　传统 CPU 版本的 SAXPY 代码（据 Harris，2012）。

在三重尖括号（«»）之间。组成计算内核的线程组织成一个层次结构，由线程块组成的网格构成，线程块又由线程束组成。在 CUDA 编程模型中，单个线程执行操作数为

```
__global__ void saxpy(int n, float a, float *x, float *y)
{
   int i = blockIdx.x*blockDim.x + threadIdx.x;
   if(i<n)
     y[i] = a*x[i] + y[i];
}
int main() {
  float *h_x, *h_y;
  int n;
  // omitted: allocate CPU memory for h_x and h_y and initialize contents
  float *d_x, *d_y;
  int nblocks = (n + 255) / 256;
  cudaMalloc( &d_x, n * sizeof(float) );
  cudaMalloc( &d_y, n * sizeof(float) );
  cudaMemcpy( d_x, h_x, n * sizeof(float), cudaMemcpyHostToDevice );
  cudaMemcpy( d_y, h_y, n * sizeof(float), cudaMemcpyHostToDevice );
  saxpy<<<nblocks, 256>>>(n, 2.0, d_x, d_y);
  cudaMemcpy( h_x, d_x, n * sizeof(float), cudaMemcpyDeviceToHost );
  // omitted: use h_y on CPU, free memory pointed to by h_x, h_y, d_x, and d_y
}
```

图 2.2 CUDA 版本的 SAXPY 代码（据 Harris，2012）。

标量值（例如 32 位浮点数）的指令。为了提高效率，典型的 GPU 硬件以锁步方式一起执行线程组。这些组被 NVIDIA 称为线程束（warps），被 AMD 称为波前（wavefronts）。NVIDIA 的线程束由 32 个线程组成，而 AMD 的波前由 64 个线程组成。线程束被分组为更大的单元，NVIDIA 称之为协作线程数组（CTA）或线程块。第 17 行指示计算内核应启动一个由 nblocks 个线程块组成的单一网格，其中每个线程块包含 256 个线程。CPU 代码传递给内核配置语句的参数被分发到 GPU 上每个运行线程的实例。

当今许多移动设备片上系统将 CPU 和 GPU 集成到单个芯片中，当今笔记本电脑和台式计算机中的处理器也是如此。然而，传统上，GPU 拥有自己的 DRAM 内存，并且今天用于机器学习的数据中心内的 GPU 仍然如此。我们注意到 NVIDIA 推出了统一内存，它可以透明地从 CPU 内存更新 GPU 内存，并从 GPU 内存更新 CPU 内存。在支持统一内存的系统中，运行时和硬件负责代表程序员执行复制。鉴于对机器学习的兴趣日益增长，并且本书的目标是理解硬件，在我们的示例中，我们考虑由程序员管理的独立 GPU 和 CPU 内存的一般情况。

遵循许多 NVIDIA CUDA 示例的风格，我们使用前缀 h_ 命名在 CPU 内存中分配的指针变量，使用 d_ 命名在 GPU 内存中分配的指针。在第 13 行，CPU 调用 CUDA 库函数 cudaMalloc。该函数调用 GPU 驱动程序，并要求它在 GPU 上分配内存供程序使用。对 cudaMemcpy 的调用将 d_x 设置为指向 GPU 内存中的一个区域，该区域包含足够的空间来容纳 n 个 32 位浮点值。在第 15 行，CPU 调用 CUDA 库函数 cudaMemcpy。该函数调用 GPU 驱动程序，并要求它将 h_x 指向的 CPU 内存中的数组内容复制到 d_x 指向的 GPU 内存中的数组。

最后，让我们聚焦于 GPU 上线程的执行。并行编程中常用的一种策略是为每个

线程分配一部分数据。为了方便这种策略，GPU 上的每个线程可以在线程块网格中查找自己的身份。在 CUDA 中实现这一点的机制使用了网格、线程块和线程标识符。在 CUDA 中，网格和线程块具有 x、y 和 z 维度。在执行时，每个线程在网格和线程块内都有一个固定的、唯一的非负整数 x、y 和 z 坐标组合。每个线程块在网格内具有 x、y 和 z 坐标。类似地，每个线程在线程块内具有 x、y 和 z 坐标。这些坐标的范围由内核配置语法设置（第 17 行）。在我们的示例中，未指定 y 和 z 维度，因此所有线程的 y 和 z 线程块坐标和线程坐标均为零。在第 3 行，threadIdx.x 的值标识线程在其线程块内的 x 坐标，blockIdx.x 指示线程块在其网格内的 x 坐标。值 $blockDim.x$ 表示 x 维度上的最大线程数。在我们的示例中，$blockDim.x$ 将计算为 256，因为这是第 17 行指定的值。表达式 $blockIdx.x * blockDim.x + threadIdx.x$ 用于计算访问数组 x 和 y 时使用的偏移量 i。正如我们将看到的，使用索引 i，我们为每个线程分配了 x 和 y 的唯一元素。

在很大程度上，编译器和硬件的结合使程序员能够对 warp 中线程执行的锁步特性保持无感。编译器和硬件使得 warp 中的每个线程看起来是独立执行的。在图 2.2 的第 4 行，我们将索引 i 的值与 n（数组 x 和 y 的大小）进行比较。对于 i 小于 n 的线程，执行第 5 行。图 2.2 中的第 5 行执行了图 2.1 中原始循环的一次迭代。网格中的所有线程完成后，计算内核在第 17 行之后将控制权返回给 CPU。在第 18 行，CPU 调用 GPU 驱动程序，将 d_y 指向的数组从 GPU 内存复制回 CPU 内存。

CUDA 编程模型的一些额外细节未在 SAXPY 示例中说明，但我们将在后面讨论，如下所示。

CTA 内的线程可以通过每个计算核心的暂存器内存高效地相互通信。NVIDIA 将这种暂存器称为共享内存。每个流式多处理器（SM）包含一个共享内存。共享内存的空间在该 SM 上运行的所有 CTA 之间划分。AMD 的 Graphics Core Next（GCN）架构 [AMD，2012] 包含类似的暂存器内存，AMD 称之为本地数据存储（LDS）。这些暂存器内存很小，每个 SM 从 16–64 KB 不等，并作为不同的内存空间暴露给程序员。程序员使用源代码中的特殊关键字（例如 CUDA 中的“_shared_”）将内存分配到暂存器内存中。暂存器内存充当软件控制的缓存。虽然 GPU 也包含硬件管理的缓存，但通过此类缓存访问数据可能导致频繁的缓存未命中。当程序员能够识别频繁且可预测地重用的数据时，应用程序会受益于使用暂存器内存。与 NVIDIA 的 GPU 不同，AMD 的 GCN GPU 还包含一个由 GPU 上所有核心共享的全局数据存储（GDS）暂存器内存。暂存器内存在图形应用程序中用于在不同图形着色器之间传递结果。例如，在 GCN 中，LDS 用于在顶点着色器和像素着色器之间传递参数值 [AMD，2012]。

CTA 内的线程可以使用硬件支持的屏障指令高效地同步。不同 CTA 中的线程可以通信，但通过所有线程均可访问的全局地址空间进行。访问此全局地址空间在时间和能耗上通常比访问共享内存更昂贵。

NVIDIA 在 Kepler 代 GPU 中引入了 CUDA 动态并行（CDP）[NVIDIA Corporation, a]。CDP 的动机是观察到数据密集型不规则应用程序可能导致 GPU 上运行的线程之间

负载不均衡，从而导致

GPU 硬件利用率不足。在许多方面，其动机与动态 Warp 形成（DWF）[Fung et al., 2007] 以及第 3.4 节中讨论的相关方法类似。

2.2 GPU 指令集架构

在本节中，我们简要讨论计算内核从 CUDA 和 OpenCL 等高级语言到 GPU 硬件执行的汇编级别的转换，以及当前 GPU 指令集的形式。GPU 架构的一个有趣方面与 CPU 架构有些不同，即 GPU 生态系统支持指令集演进的方式。例如，x86 微处理器向后兼容 1976 年发布的 Intel 8086。向后兼容意味着为上一代架构编译的程序无需任何更改即可在下一代架构上运行。因此，40 年前为 Intel 8086 编译的软件理论上可以在当今任何 x86 处理器上运行。

2.2.1 NVIDIA GPU 指令集架构

鉴于有时有大量供应商提供 GPU 硬件（每个都有自己的硬件设计），随着早期 GPU 变得可编程，通过 OpenGL 着色语言（OGSL）和微软的高级着色语言（HLSL）进行一定程度的指令集虚拟化变得普遍。当 NVIDIA 在 2007 年初引入 CUDA 时，他们决定遵循类似的道路，并引入了自己的用于 GPU 计算的高级虚拟指令集架构，称为并行线程执行 ISA，或 PTX [NVI，2017]。NVIDIA 在每次发布 CUDA 时都完整记录了此虚拟指令集架构，以至于本书作者很容易开发出支持 PTX 的 GPGPU-Sim 模拟器 [Bakhoda et al., 2009]。PTX 在许多方面类似于标准的精简指令集计算机（RISC）指令集架构，如 ARM、MIPS、SPARC 或 ALPHA。它也与优化编译器中使用的中间表示有相似之处。一个这样的例子是使用无限量的虚拟寄存器。图 2.3 展示了图 2.2 中 SAXPY 程序的 PTX 版本。

在 GPU 上运行 PTX 代码之前，必须将 PTX 编译成硬件支持的实际指令集架构。NVIDIA 将这一层称为 SASS，是“Streaming ASSembler”的缩写 [Cabral，2016]。从 PTX 转换为 SASS 的过程可以通过 GPU 驱动程序或 NVIDIA CUDA 工具包中提供的名为 ptxas 的独立程序来完成。NVIDIA 并未完全公开 SASS 的文档。这使得学术研究人员难以开发能够捕捉所有编译器优化效果的架构模拟器，但也使 NVIDIA 免受客户对硬件级别向后兼容性的需求，从而能够在一代又一代之间完全重新设计指令集架构。不可避免地，希望从底层理解性能的开发者开始创建自己的工具来反汇编 SASS。第一个这样的努力由 Wladimir Jasper van der Laan 完成，名为“decuda”[van der Lann]，于 2007 年底针对 NVIDIA 的 GeForce 8 系列（G80）发布，距离首款支持 CUDA 的硬件发布大约一年。

decuda 项目对 SASS 指令集进行了足够详细的了解，以至于可以开发汇编器。这有助于在 GPGPU-Sim 3.2.2 中开发对 SASS 的支持，直至 NVIDIA 的 GT200 架构 [Tor

```
.visible .entry _Z5saxpyifPfS_(
.param .u32 _Z5saxpyifPfS__param_0,
.param .f32 _Z5saxpyifPfS__param_1,
.param .u64 _Z5saxpyifPfS__param_2,
.param .u64 _Z5saxpyifPfS__param_3
)
{
.reg .pred %p<2>;
.reg .f32 %f<5>;
.reg .b32 %r<6>;
.reg .b64 %rd<8>;


ld.param.u32 %r2, [_Z5saxpyifPfS__param_0];
ld.param.f32 %f1, [_Z5saxpyifPfS__param_1];
ld.param.u64 %rd1, [_Z5saxpyifPfS__param_2];
ld.param.u64 %rd2, [_Z5saxpyifPfS__param_3];
mov.u32 %r3, %ctaid.x;
mov.u32 %r4, %ntid.x;
mov.u32 %r5, %tid.x;
mad.lo.s32 %r1, %r4, %r3, %r5;
setp.ge.s32 %p1, %r1, %r2;
@%p1 bra BB0_2;

cvta.to.global.u64 %rd3, %rd2;
cvta.to.global.u64 %rd4, %rd1;
mul.wide.s32 %rd5, %r1, 4;
add.s64 %rd6, %rd4, %rd5;
ld.global.f32 %f2, [%rd6];
add.s64 %rd7, %rd3, %rd5;
ld.global.f32 %f3, [%rd7];
fma.rn.f32 %f4, %f2, %f1, %f3;
st.global.f32 [%rd7], %f4;

BB0_2:
ret;
}
```

图 2.3　与图 2.2 的 SAXPY 内核对应的 PTX 代码（CUDA 8.0）。

M. Aamodt et al。]。NVIDIA 最终引入了一个名为 cuobjdump 的工具，并开始部分公开 SASS 文档。NVIDIA 的 SASS 文档 [NVIDIA Corporation，c] 目前（2018 年 4 月）仅提供了汇编操作码名称列表，但没有关于操作数格式或 SASS 指令语义的详细信息。最近，随着 GPU 在机器学习中的使用激增以及对性能优化代码的需求，其他人针对后续架构（如 NVIDIA 的 Fermi [Yunqing] 和 NVIDIA 的 Maxwell 架构 [Gray]）开发了类似于 decuda 的工具。

图 2.4 展示了我们的 SAXPY 内核为 NVIDIA 的 Fermi 架构 [NVI，2009] 编译的 SASS 代码，并使用 NVIDIA 的 cuobjdump（CUDA 工具包的一部分）提取。图 2.4 中的第一列是指令的地址。第二列是汇编，第三列是编码指令。如上所述，NVIDIA 仅部分公开了其硬件汇编文档。比较图 2.3 和图 2.4，可以注意到虚拟 ISA 层和硬件 ISA 层之间的相似之处和差异。在高层次上，有一些重要的相似之处，例如两者都是 RISC（都使用加载和存储来访问内存），并且都使用谓词执行 [Allen et al., 1983]。更微妙的差异包括：（1）PTX 版本具有基本上无限的寄存器集，因此每个定义通常使用一个新寄存器，很像静态单赋值 [Cytron et al., 1991]，而 SASS 使用有限的寄存器集；（2）内核参数通过分体常量内存传递，在 SASS 中可以通过非加载/存储指令访问，而在 PTX 中参数被分配到它们自己单独的“参数”地址空间。

图 2.5 展示了由相同版本的 CUDA 为 NVIDIA 的 Pascal 架构生成的 SAXPY 的 SASS 代码，并使用 NVIDIA 的 cuobjdump 提取。将图 2.5 与图 2.4 进行比较，很明显 NVIDIA 的 ISA 已经发生了显著变化，包括指令编码方面。图 2.5 包含一些没有反汇编指令的行（例如，第 3 行地址 0x0000 处）。这些是 NVIDIA Kepler 架构中引入的特殊“控制指令”，以消除使用记分板进行显式依赖性检查的需要 [NVIDIA Corporation，b]。Lai 和 Seznec [2013] 探索了 Kepler 架构控制指令的编码。正如 Lai 和 Seznec [2013] 所指出的，这些控制指令似乎类似于 Tera 计算机系统上的显式依赖前瞻 [Alverson et al., 1990]。Gray 描述了他们在 NVIDIA 的 Maxwell 架构中能够推断出的控制指令编码的广泛细节。根据 Gray 的说法，在 Maxwell 中每三条常规指令就有一条控制指令。如图 2.5 所示，NVIDIA 的 Pascal 架构似乎也是如此。根据 Gray 的说法，Maxwell 上的 64 位控制指令包含三组 21 位，为以下三条指令中的每一条编码以下信息：停顿计数；让步提示标志；以及写、读和等待依赖屏障。Gray 还描述了常规指令上寄存器重用标志的使用，这也可以在图 2.5 中看到（例如，第 7 行整数短乘加指令 XMAD 的第一个源操作数使用了 R0.reuse）。这似乎表明从 Maxwell 开始，NVIDIA GPU 中增加了“操作数重用缓存”（参见第 3.6.1 节的相关研究）。这个操作数重用缓存似乎使得每次主寄存器文件访问可以多次读取寄存器值，从而降低能耗和/或提高性能。

2.2.2 AMD 图形核心下一代指令集架构

与 NVIDIA 不同，AMD 在推出 Southern Islands 架构时，发布了完整的硬件级 ISA 规范 [AMD，2012]。Southern Islands 是 AMD 的图形核心下一代（GCN）架构的第一代。AMD 硬件 ISA 文档的可用性帮助学术研究人员开发了在较低级别工作的模拟器

```
Address     Dissassembly                                        Encoded Instruction
========    ================================================    ======================
/*0000*/         MOV R1, c[0x1][0x100];                        /* 0x2800440400005de4 */
/*0008*/         S2R R0, SR_CTAID.X;                           /* 0x2c00000094001c04 */
/*0010*/         S2R R2, SR_TID.X;                             /* 0x2c00000084009c04 */
/*0018*/         IMAD R0, R0, c[0x0][0x8], R2;                 /* 0x2004400020001ca3 */
/*0020*/         ISETP.GE.AND P0, PT, R0, c[0x0][0x20], PT;    /* 0x1b0e40008001dc23 */
/*0028*/     @P0 BRA.U 0x78;                                   /* 0x40000001200081e7 */
/*0030*/    @!P0 MOV32I R5, 0x4;                               /* 0x18000000100161e2 */
/*0038*/    @!P0 IMAD R2.CC, R0, R5, c[0x0][0x28];             /* 0x200b8000a000a0a3 */
/*0040*/    @!P0 IMAD.HI.X R3, R0, R5, c[0x0][0x2c];           /* 0x208a8000b000e0e3 */
/*0048*/    @!P0 IMAD R4.CC, R0, R5, c[0x0][0x30];             /* 0x200b8000c00120a3 */
/*0050*/    @!P0 LD.E R2, [R2];                                /* 0x840000000020a085 */
/*0058*/    @!P0 IMAD.HI.X R5, R0, R5, c[0x0][0x34];           /* 0x208a8000d00160e3 */
/*0060*/    @!P0 LD.E R0, [R4];                                /* 0x8400000000402085 */
/*0068*/    @!P0 FFMA R0, R2, c[0x0][0x24], R0;                /* 0x3000400090202000 */
/*0070*/    @!P0 ST.E [R4], R0;                                /* 0x9400000000402085 */
/*0078*/         EXIT;                                         /* 0x8000000000001de7 */
```

图 2.4　NVIDIA Fermi 架构的 SASS 代码（据原书图 2.4）。

[Ubal et al., 2012]。AMD 的编译流程还包括一个虚拟指令集架构，称为 HSAIL，作为异构系统架构（HSA）的一部分。

AMD 的 GCN 架构与 NVIDIA GPU（包括 NVIDIA 最新的 Volta 架构 [NVIDIA Corp., 2017]）之间的一个关键区别是标量指令和向量指令的分离。图 2.6 和 2.7 复制了 AMD [2012] 的一个示例，展示了高级 OpenCL（类似于 CUDA）代码和 AMD South-的等效机器指令。

ern Islands 架构。在图 2.7 中，标量指令以 s_ 为前缀，向量指令以 v_ 为前缀。在 AMD GCN 架构中，每个计算单元（例如 SIMT 核心）包含一个标量单元和四个向量单元。向量指令在向量单元上执行，并为波前中的每个线程计算不同的 32 位值。相比之下，标量指令在标量单元上执行，计算一个由波前中所有线程共享的 32 位值。在图 2.7 所示的示例中，标量指令与控制流处理相关。特别地，exec 是一个特殊寄存器，用于为 SIMT 执行谓词化各个向量通道的执行。图形处理器单元上使用掩码进行控制流处理的更多细节将在第 3.1.1 节中描述。GCN 架构中标量单元的另一个潜在优势是，SIMT 程序中计算的某些部分常常会独立于线程 ID 计算出相同的结果（见第 3.5 节）。

AMD 的 GCN 硬件指令集手册 [AMD，2012] 提供了许多关于 AMD 图形处理器单元硬件的有趣见解。例如，为了启用长延迟操作的数据依赖解析，AMD 的 GCN 架构包含了 s_WAITCNT 指令。对于每个波前，有三个计数器：向量内存计数、本地/全局数据存储计数和寄存器导出计数。每个计数器指示给定类型的未完成操作的数量。编译器或程序员插入 s_WAITCNT 指令，使波前等待，直到未完成操作的数量降至指定阈值以下。

```
Address     Dissassembly                                        Encoded Instruction
========    ============================================        ======================
                                                                /* 0x001c7c00e22007f6 */
/*0008*/       MOV R1, c[0x0][0x20];                            /* 0x4c98078000870001 */
/*0010*/       S2R R0, SR_CTAID.X;                              /* 0xf0c8000002570000 */
/*0018*/       S2R R2, SR_TID.X;                                /* 0xf0c8000002170002 */
                                                                /* 0x001fd840fec20ff1 */
/*0028*/       XMAD.MRG R3, R0.reuse, c[0x0] [0x8].H1, RZ;      /* 0x4f107f8000270003 */
/*0030*/       XMAD R2, R0.reuse, c[0x0] [0x8], R2;             /* 0x4e00010000270002 */
/*0038*/       XMAD.PSL.CBCC R0, R0.H1, R3.H1, R2;              /* 0x5b30011800370000 */
                                                                /* 0x081fc400ffa007ed */
/*0048*/       ISETP.GE.AND P0, PT, R0, c[0x0][0x140], PT;      /* 0x4b6d038005070007 */
/*0050*/    @P0 EXIT;                                           /* 0xe30000000000000f */
/*0058*/       SHL R2, R0.reuse, 0x2;                           /* 0x3848000000270002 */
                                                                /* 0x081fc440fec007f5 */
/*0068*/       SHR R0, R0, 0x1e;                                /* 0x3829000001e70000 */
/*0070*/       IADD R4.CC, R2.reuse, c[0x0][0x148];             /* 0x4c10800005270204 */
/*0078*/       IADD.X R5, R0.reuse, c[0x0][0x14c];              /* 0x4c10080005370005 */
                                                                /* 0x0001c800fe0007f6 */
/*0088*/       IADD R2.CC, R2, c[0x0][0x150];                   /* 0x4c10800005470202 */
/*0090*/       IADD.X R3, R0, c[0x0][0x154];                    /* 0x4c10080005570003 */
/*0098*/       LDG.E R0, [R4];        }                         /* 0xeed4200000070400 */
                                                                /* 0x0007c408fc400172 */
/*00a8*/       LDG.E R6, [R2];                                  /* 0xeed4200000070206 */
/*00b0*/       FFMA R0, R0, c[0x0][0x144], R6;                  /* 0x4980030005170000 */
/*00b8*/       STG.E [R2], R0;                                  /* 0xeedc200000070200 */
                                                                /* 0x001f8000ffe007ff */
/*00c8*/       EXIT;                                            /* 0xe30000000007000f */
/*00d0*/       BRA 0xd0;                                        /* 0xe2400fffff87000f */
/*00d8*/       NOP;                                             /* 0x50b0000000070f00 */
                                                                /* 0x001f8000fc0007e0 */
/*00e8*/       NOP;                                             /* 0x50b0000000070f00 */
/*00f0*/       NOP;                                             /* 0x50b0000000070f00 */
/*00f8*/       NOP;                                             /* 0x50b0000000070f00 */
```

图 2.5 与图 2.2 内核对应的 NVIDIA Pascal 架构 SASS 代码（CUDA 8.0）。

```
float fn0(float a,float b)
{
    if(a>b)
        return(a * a - b);
    else
        return(b * b - a);
}
```

图 2.6 OpenCL 代码示例（据 AMD，2012）。

```
// Registers r0 contains "a", r1 contains "b"
// Value is returned in r2
    v_cmp_gt_f32 r0, r1 // a>b
    s_mov_b64 s0, exec  // Save current exec mask
    s_and_b64 exec, vcc, exec // Do "if"
    s_cbranch_vccz label0 // Branch if all lanes fail
    v_mul_f32 r2, r0, r0 // result = a * a
    v_sub_f32 r2, r2, r1 // result = result - b
label0:
    s_not_b64 exec, exec // Do "else"
    s_and_b64 exec, s0, exec // Do "else"
    s_cbranch_execz label1 // Branch if all lanes fail
    v_mul_f32 r2, r1, r1 // result = b * b
    v_sub_f32 r2, r2, r0 // result = result - a
label1:
    s_mov_b64 exec, s0   // Restore exec mask
```

图 2.7　AMD Southern Islands（GCN）汇编代码示例（据 AMD，2012）。

第三章　SIMT 核心：指令与寄存器数据流

在本章及下一章中，我们将考察现代图形处理器单元的体系结构与微架构。我们将图形处理器单元架构的讨论分为两部分：（1）在本章考察执行计算的 SIMT 核心，然后（2）在下一章研究内存系统。

在传统的图形渲染角色中，图形处理器单元访问的数据集（如精细纹理映射）太大，无法完全在片上缓存。为了实现高性能可编程性（这在图形学中是可取的，既能随着图形模式数量的增加而降低验证成本，又能使游戏开发者更容易地差异化其产品[Lindholm et al., 2001]），有必要采用能够维持大芯片外带宽的架构。因此，当今的图形处理器单元并发执行数万个线程。虽然每个线程的片上内存存储量很小，但缓存仍能有效减少大量芯片外内存访问。例如，在图形工作负载中，相邻像素操作之间存在显著的空间局部性，这可以被片上缓存捕获。

图 3.1 展示了本章讨论的图形处理器单元流水线的微架构。该图展示了图 1.2 中单个 SIMT 核心的内部组织。流水线可分为 SIMT 前端和 SIMD 后端。流水线由三个在单个流水线中协同工作的调度“循环”组成：指令取指循环、指令发射循环和寄存器访问调度循环。指令取指循环包括标记为取指、指令缓存、解码和指令缓冲区的模块。指令发射循环包括标记为指令缓冲区、记分板、发射和 SIMT 栈的模块。寄存器访问调度循环包括标记为操作数收集器、算术逻辑单元和内存的模块。在本章的剩余部分，我们将通过考虑依赖于这些循环的架构关键方面，帮助您全面理解图中的各个模块。

由于完全理解这一组织涉及许多细节，我们将讨论分成若干部分。我们安排这些部分的目的是逐步形成对核心微架构越来越详细的视图。我们从整体图形处理器单元流水线的高层视图开始，然后填充细节。我们称这些越来越精确的描述为“近似”，以承认即使在我们最详细的描述中也会省略一些细节。由于当今图形处理器单元的核心组织原则是多线程，我们将这些“近似”围绕上述三个调度循环进行组织。第 3 章

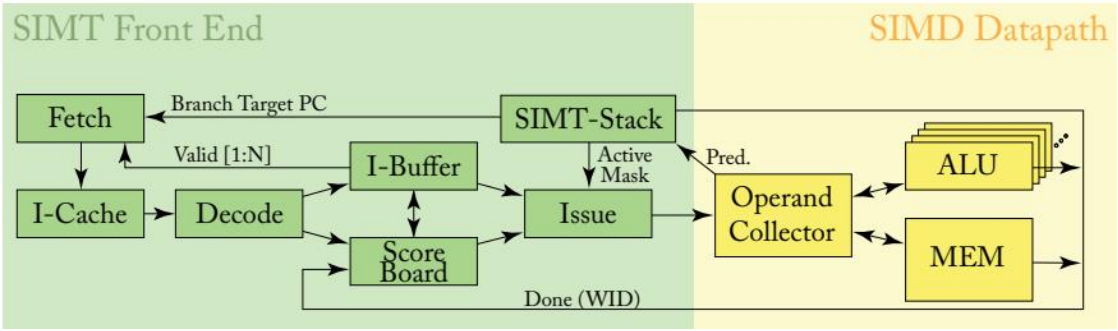


图 3.1　通用 GPGPU 核心的微架构。

我们发现，通过考虑三个越来越精确的“近似循环”来组织本章是方便的，这些

循环逐步考虑到这些调度器循环的细节。

3.1　单循环近似

我们首先考虑具有单个调度器的图形处理器单元。这种对硬件的简化看法与人们仅阅读 CUDA 编程手册中的硬件描述时可能期望硬件所做的并无不同。

为了提高效率，线程被组织成组，NVIDIA 称之为“warps”，AMD 称之为“wavefronts”。因此，调度单位是 warp。在每个周期中，硬件选择一个 warp 进行调度。在单循环近似中，warp 的程序计数器用于访问指令内存，以找到 warp 要执行的下一条指令。取指后，指令被解码，并从寄存器堆中取出源操作数寄存器。在从寄存器堆取源操作数的同时，确定 SIMT 执行掩码值。以下小节描述如何确定 SIMT 执行掩码值，并将其与现代图形处理器单元中也采用的谓词化进行对比。

在执行掩码和源寄存器可用后，执行以单指令多数据的方式进行。只要 SIMT 执行掩码置位，每个线程就在与通道关联的功能单元上执行。与现代中央处理器单元设计一样，功能单元通常是异构的，意味着给定的功能单元仅支持指令的子集。例如，NVIDIA 图形处理器单元包含特殊功能单元（SFU）、加载/存储单元、浮点功能单元、整数功能单元，以及自 Volta 起的 Tensor Core。

所有功能单元名义上包含的通道数与一个线程束中的线程数相同。然而，一些图形处理器单元采用了不同的实现方式，即单个线程束或波前在多个时钟周期内执行。这是通过以更高频率为功能单元提供时钟来实现的，这可以在牺牲能耗增加的情况下实现更高的单位面积性能。为功能单元实现更高时钟频率的一种方法是对其执行进行流水线化或增加其流水线深度。

3.1.1　SIMT 执行掩码

现代图形处理器单元的一个关键特性是 SIMT 执行模型，从功能的角度（尽管不是性能角度）来看，它向程序员呈现的抽象是各个线程完全独立执行。这种编程模型可能仅通过谓词执行来实现。然而，在当前图形处理器单元中，它是通过传统谓词执行与一组谓词掩码栈（我们将称之为 SIMT 栈）的结合来实现的。

SIMT 栈有助于高效处理当所有线程都可以独立执行时出现的两个关键问题。第一个是嵌套控制流。在嵌套控制流中，一个分支控制依赖于另一个分支。第二个问题是在线程束中所有线程都避开某条控制流路径时完全跳过计算。对于复杂的控制流，这可以带来显著的节省。传统上，支持谓词执行的中央处理器单元通过使用多个谓词寄存器来处理嵌套控制流，并且文献中已提出支持跨通道谓词测试。

图形处理器单元采用的 SIMT 栈可以处理嵌套控制流和跳过计算。专利和指令集手册中描述了多种实现方式。在这些描述中，SIMT 栈至少部分由专用于此目的的特殊指令管理。相反，我们将描述学术工作中引入的一个略微简化的版本，该版本假设硬

件负责管理 SIMT 栈。

为了描述 SIMT 栈，我们使用一个示例。图 3.2 展示了包含嵌套在 do-while 循环中的两个分支的 CUDA C 代码，图 3.3 展示了相应的 PTX 汇编。图 3.4（复制了 Fung 等人 [Fung et al., 2007] 中的图 5）展示了假设图形处理器单元每个线程束有四个线程时，此代码如何与 SIMT 栈交互。

图 3.4a 展示了对应于图 3.2 和 3.3 中代码的控制流图（CFG）。如 CFG 顶部节点内的标签“A/1111”所示，最初线程束中的所有四个线程都在执行基本块 A 中的代码，这对应于图 3.2 中第 2 行至第 6 行以及图 3.3 中第 1 行至第 6 行的代码。在执行图 3.3 中第 6 行的分支（对应于图 3.2 中第 6 行的“if”语句）后，这四个线程遵循不同（发散）的控制流。具体来说，如图 3.4a 中标签“B/1110”所示，前三个线程落入基本块 B。这三个线程分支到图 3.3 中的第 7 行（图 3.2 中的第 7 行）。如图 3.4a 中标签“F/0001”所示，执行分支后，第四个线程跳转到基本块 F，这对应于图 3.3 中的第 14 行（图 3.2 中的第 14 行）。

类似地，当在基本块 B 中执行的三个线程到达图 3.3 中第 9 行的分支时，第一个线程发散到基本块 C，而第二个和第三个线程发散到基本块 D。然后，所有三个线程到达基本块 E 并一起执行，如图 3.4a 中标签“E/1110”所示。在基本块 G 中，所有四个线程一起执行。

图形处理器单元硬件如何使线程束内的线程在采用每周期仅允许执行一条指令的 SIMD 数据通路的情况下，沿代码中的不同路径执行?

当前图形处理器单元采用的方法是串行化给定线程束内沿不同路径执行的线程。这在图 3.4b 中示出，其中箭头代表线程。实心箭头表示线程正在执行相应基本块中的代码（由每个矩形顶部的字母指示）。空心箭头表示线程被掩蔽。如图所示，时间向右推进，如底部箭头所示。最初，每个线程都在基本块 B 中执行。然后，在分支之后，前三个线程执行基本块 B 中的代码。注意，此时第四个线程被掩蔽。为了维持 SIMD 执行，第四个线程在不同的时间（在此示例中为几个周期后）执行通过基本块 F 的替代代码路径。

为了实现发散代码路径的这种串行化，一种方法是使用如图 3.4c–e 所示的栈。此栈上的每个条目包含三个条目：重汇聚程序计数器（RPC）、下一条要执行的指令的地址（Next PC）以及活动掩码。

图 3.4c 展示了线程束执行图 3.3 中第 6 行的分支后栈的状态。由于三个线程分支到基本块 B，一个线程分支到基本块 F，因此两个新条目已添加到栈顶（TOS）。线程束执行的下一条指令由栈顶（TOS）条目中的 Next PC 值确定。在图 3.4c 中，此 Next PC 值为 B，它表示基本块 B 中第一条指令的地址。相应的活动掩码条目“1110”表示线程束中只有前三个线程应执行此指令。

线程束中的前三个线程继续执行基本块 B 中的指令，直到它们到达图 3.3 中第 9 行的分支。执行此分支后，它们如前面所述发生发散。此分支发散导致栈发生三处变

```
do {
  t1 = tid*N;       // A
  t2 = t1 + i;
  t3 = data1[t2];
  t4 = 0;
  if( t3 != t4 ) {
    t5 = data2[t2]; // B
    if( t5 != t4 ) {
      x += 1;       // C
    } else {
      y += 2;       // D
    }
  } else {
    z += 3;         // F
  }
  i++;              // G
} while( i < N );
```

图 3.2　说明 SIMT 栈操作的 CUDA C 源代码示例。

```
A:    mul.lo.u32      t1, tid, N;
      add.u32         t2, t1, i;
      ld.global.u32   t3, [t2];
      mov.u32         t4, 0;
      setp.eq.u32     p1, t3, t4;
@p1   bra             F;
B:    ld.global.u32   t5, [t2];
      setp.eq.u32     p2, t5, t4;
@p2   bra             D;
C:    add.u32         x, x, 1;
      bra             E;
D:    add.u32         y, y, 2;
E:    bra             G;
F:    add.u32         z, z, 3;
G:    add.u32         i, i, 1;
      setp.le.u32     p3, i, N;
@p3   bra             A;
```

图 3.3　说明 SIMT 栈操作的 PTX 汇编示例。

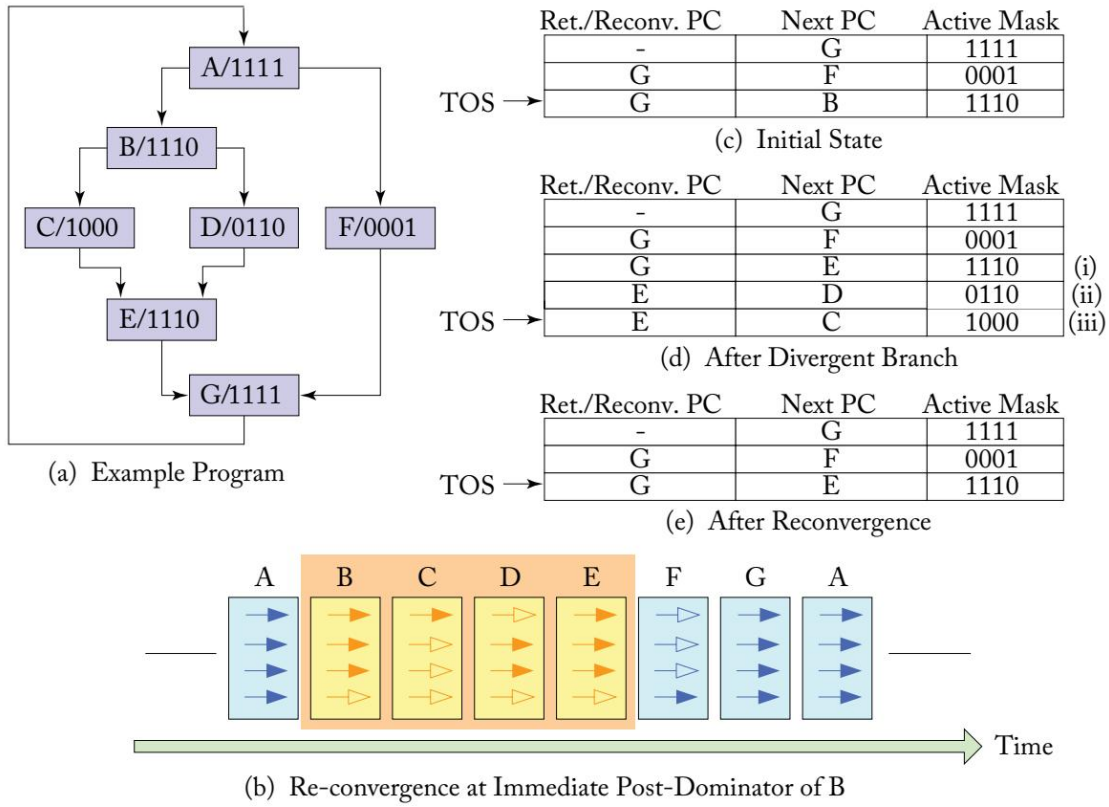


图 3.4 SIMT 栈操作示例（基于 Fung 等人 [2007] 的图 5）。

化。首先，执行分支之前的 TOS 条目的 Next PC 条目（在图 3.4d 中标记为（i））被修改为分支的重汇聚点，即基本块 E 中第一条指令的地址。然后，添加两个条目（在图 3.4d 中标记为（ii）和（iii）），分别对应执行分支后线程束中线程所遵循的每条路径。

重收敛点是指程序中一个位置，在该位置分叉的线程可以被强制以锁步方式继续执行。通常优先选择最近的重收敛点。在给定程序执行中，编译时能够保证分叉线程可以再次以锁步方式执行的最早位置是导致分支分叉的分支的直接后支配点。在运行时，有时可以在程序中更早的位置进行重收敛 [Coon and Lindholm，2008，Diamos et al., 2011，Fung and Aamodt，2011]。

一个有趣的问题是“在分叉分支之后，应该使用什么顺序将条目添加到栈中?”为了将重收敛栈的最大深度降低到与一个 warp 中的线程数成对数关系，最好先将活动线程最多的条目放入栈中，然后再放入活动线程较少的条目 [AMD，2012]。在图 3.4 的（d）部分中，我们遵循了这个顺序，而在（c）部分中我们使用了相反的顺序。

3.1.2 SIMT 死锁与无栈 SIMT 架构

最近，NVIDIA 披露了其即将推出的 Volta GPU 架构的细节 [NVIDIA Corp., 2017]。他们强调的一项变化将涉及分叉下掩码的行为以及这与同步的交互方式。基于栈的 SIMT 实现可能导致一种死锁条件，ElTantawy 和 Aamodt[2016] 称之为“SIMT 死锁”。学术工作描述了用于 SIMT 执行的替代硬件 [ElTantaway et al., 2014]，经过少量修改

[ElTantawy and Aamodt，2016] 后可以避免 SIMT 死锁。NVIDIA 将其新的线程分叉管理方法称为独立线程调度。对独立线程调度的描述表明，他们实现了与上述学术提案类似的行为。下面，我们首先描述 SIMT 死锁问题，然后描述一种避免 SIMT 死锁的机制，该机制与 NVIDIA 对独立线程调度的描述一致，并在 NVIDIA 最近的一项专利申请中公开 [Diamos et al., 2015]。

图 3.5 的左侧给出了一个说明 SIMT 死锁问题的 CUDA 示例，中间部分显示了相应的控制流图。A 行将共享变量 mutex 初始化为零，表示锁是空闲的。在 B 行，warp 中的每个线程执行 atomicCAS 操作，该操作对包含 mutex 的内存位置执行比较并交换操作。atomicCAS 操作是一个编译器内建函数，被翻译为 atom。global。cas PTX 指令。逻辑上，比较并交换首先读取 mutex 的内容，然后将其与第二个输入 0 进行比较。如果 mutex 的当前值为 0，则比较并交换操作将 mutex 的值更新为第三个输入 1。atomicCAS 返回的值是 mutex 的原始值。重要的是，比较并交换对每个线程原子地执行上述逻辑操作序列。因此，同一 warp 内不同线程对任何单个位置的多次 atomicCAS 访问是串行化的。由于图 3.5 中的所有线程访问同一内存位置，只有一个线程会看到 mutex 的值为 0，其余线程将看到值为 1。接下来，在牢记 SIMT 栈的情况下，考虑 atomicCAS 返回后 B 行上的 while 循环会发生什么。不同线程看到不同的循环条件。具体来说，一个线程会想要退出循环，而其余线程会想要留在循环中。退出循环的线程将已到达重收敛点，因此将不再在 SIMT 栈上活动，从而无法执行 C 行上的 atomicExch 操作来释放锁。留在循环中的线程将在 SIMT 栈顶活动，并将无限自旋。由此产生的线程间的循环依赖引入了一种新的死锁形式，ElTantawy 和 Aamodt[2016] 称之为 SIMT 死锁，如果线程在 MIMD 架构上执行，这种死锁将不存在。

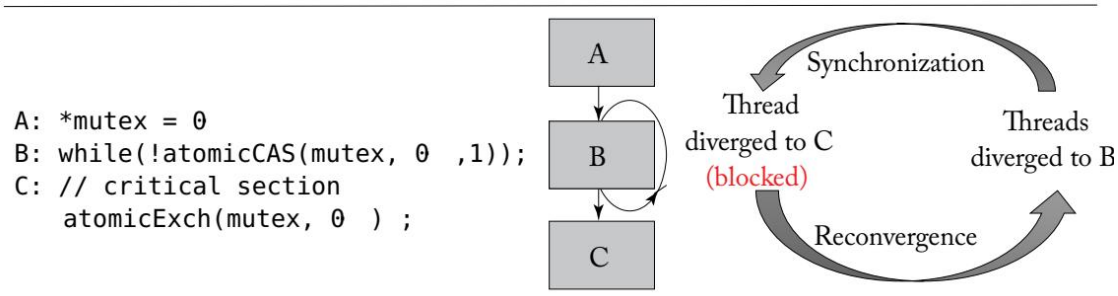


图 3.5　SIMT 死锁示例（基于 ElTantawy 和 Aamodt[2016] 的图 1）。

接下来，我们总结一种无栈分支重收敛机制，类似于 NVIDIA 最近在美国专利申请中描述的机制 [Diamos et al., 2015]。该机制与 NVIDIA 迄今为止对 Volta 重收敛处理机制的描述一致 [Nvidia，2017]。关键思想是用每 warp 收敛屏障替换栈。图 3.6 显示了 NVIDIA 专利申请中描述的每 warp 维护的各种字段，图 3.8

提供了相应的示例来说明收敛屏障的操作。实际上，该提案提供了多路径 IPDOM 的替代实现 [ElTantaway et al., 2014]，这将在第 3.4.2 节中与早期的学术工作一起描述。收敛屏障机制与 Fung 和 Aamodt[2011] 中描述的 warp 屏障概念有一些相似之处。为了帮助解释下面的收敛屏障机制，我们考虑在如图 3.8 所示代码上执行单个 warp，图 3.8

显示了由图 3.7 所示 CUDA 代码产生的控制流图。

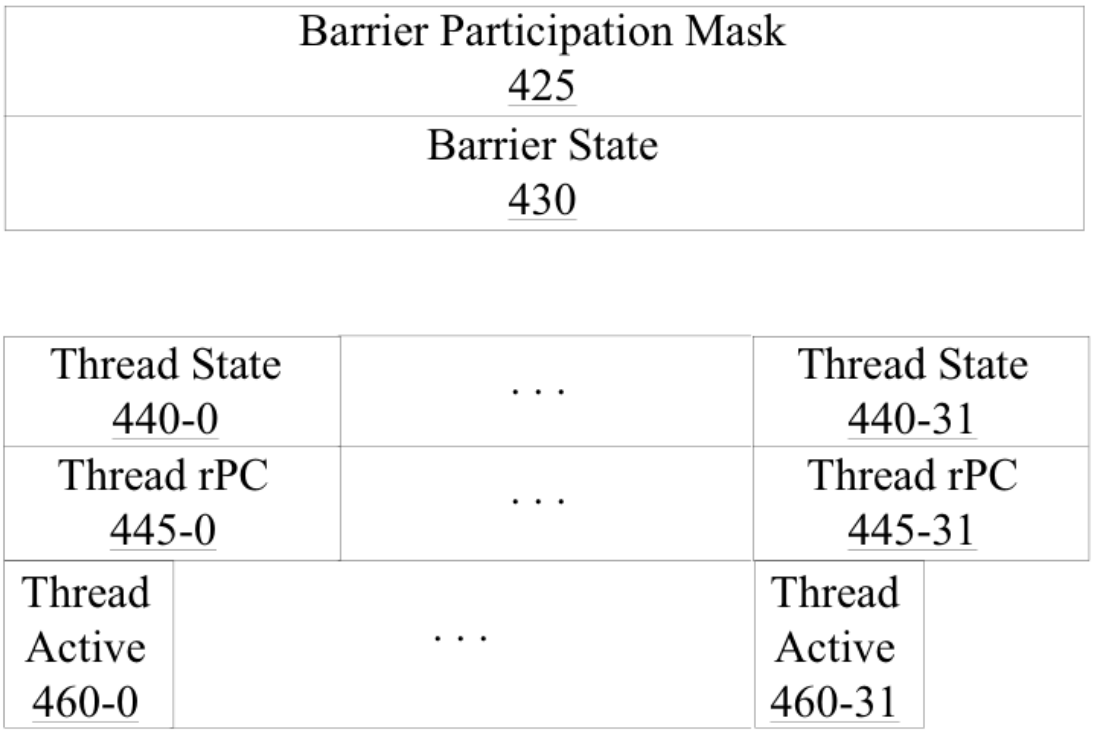


图 3.6 NVIDIA 的无栈收敛屏障式分支发散处理机制（据 Diamos 等，2015）。

```
// id = warp ID
// BB_A Basic Block "A"
if(id%2==0){
    // BB_B
}else{
    // BB_C
    if(id==1){
        // BB_D
    }else{
        // BB_E
    }
    // BB_F
}
// BB_G
```

图 3.7 嵌套控制流代码示例（据 ElTantawy 等，2014）。

接下来，我们描述图 3.6 中的字段。这些字段存储在寄存器中，并由硬件 warp 调度器使用。每个屏障参与掩码用于跟踪给定 warp 中哪些线程参与给定的收敛屏障。一个给定的 warp 可能有多个屏障参与掩码。在常见情况下，由给定屏障参与掩码跟踪的线程将等待彼此在分叉分支之后到达程序中的共同点，从而一起重收敛。为了支持这

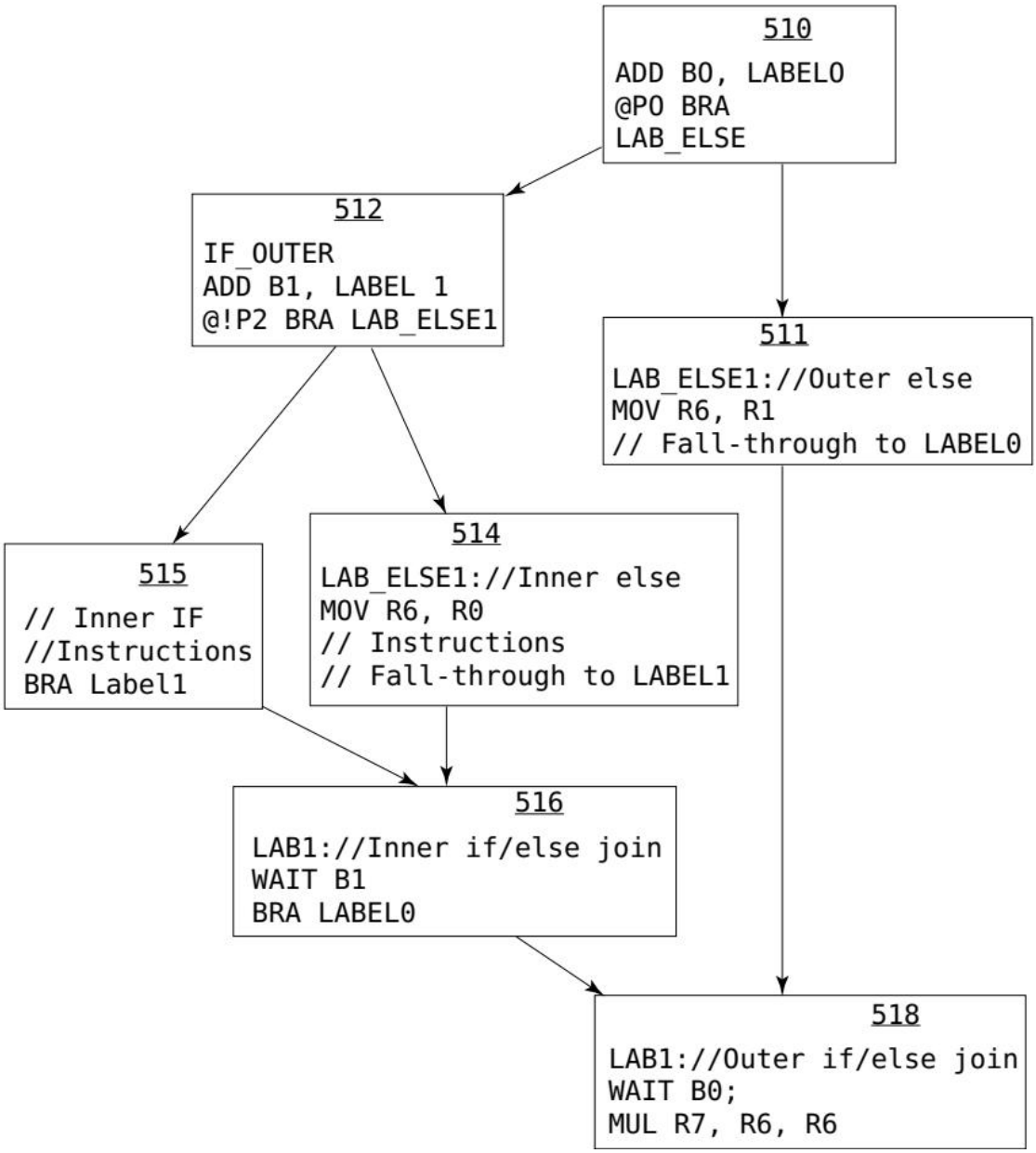


图 3.8　NVIDIA 最近描述的收敛屏障分支分叉处理机制的代码示例（基于 Diamos 等人 [2015] 的图 5B）。

一点，屏障状态字段用于跟踪哪些线程已到达给定的收敛屏障。线程状态跟踪 warp 中的每个线程是否准备好执行、在收敛屏障处阻塞（如果是，是哪一个），或者已经让出。似乎让出状态可能用于使 warp 中的其他线程在某种情况下越过收敛屏障取得进展

　否则将导致 SIMT 死锁。Thread rPC 字段为每个非活跃线程记录下一条要执行的指令地址。Thread Active 字段是一个位，指示 warp 中对应线程是否活跃。

假设一个 warp 包含 32 个线程，则屏障参与掩码为 32 位宽。若某位被置位，表示 warp 中对应线程参与此收敛屏障。当线程执行分支指令（如图 3.8 中基本块 510 和 512

末尾的指令）时会发生分歧。这些分支对应于图 3.7 中的两个“if”语句。warp 调度器使用屏障参与掩码将线程停止在特定的收敛屏障位置，该位置可以是分支的立即后支配点或其他位置。在任何给定时刻，每个 warp 可能需要多个屏障参与掩码以支持嵌套控制流结构，如图 3.7 中的嵌套 if 语句。图 3.6 中的寄存器可以用通用寄存器、专用寄存器或两者的某种组合来实现（专利申请中未说明）。鉴于屏障参与掩码只有 32 位宽，如果每个线程都有一份屏障参与掩码的副本（就像朴素地使用通用寄存器文件存储它时可能发生的那样），那将是冗余的。然而，由于控制流可以嵌套到任意深度，一个给定的 warp 可能需要任意数量的屏障参与掩码，因此对掩码进行软件管理是可取的。

为了初始化收敛屏障参与掩码，使用一种特殊的“ADD”指令。当 warp 执行此 ADD 指令时，所有活跃线程的位在 ADD 指令所指示的收敛屏障中被置位。执行分支后，一些线程可能发生分歧，这意味着下一条要执行的指令地址（即 PC）将不同。当这种情况发生时，调度器将选择具有共同 PC 的线程子集，并更新 Thread Active 字段以启用 warp 中这些线程的执行。学术提案将这种线程子集称为“warp 分裂”[ElTantaway 等，2014，ElTantawy 和 Aamodt，2016，Meng 等，2010]。与基于栈的 SIMT 实现相比，采用收敛屏障实现时，调度器可以自由地在分歧线程组之间切换。这使得当一个 warp 中的某些线程已获取锁而其他线程尚未获取时，线程之间仍能向前推进。

“WAIT”指令用于在 warp 分裂到达收敛屏障时将其停止。如 NVIDIA 专利申请中所述，WAIT 指令包含一个操作数，用于指示收敛屏障的身份。WAIT 指令的作用是将 warp 分裂中的线程添加到该屏障的 Barrier State 寄存器中，并将线程状态改为阻塞。一旦屏障参与掩码中的所有线程都执行了相应的 WAIT 指令，线程调度器就可以将原始 warp 分裂中的所有线程切换为活跃状态，从而保持 SIMD 效率。图 3.8 中的示例有两个收敛屏障 B1 和 B2，其 WAIT 指令位于基本块 516 和 518 中。为了实现 warp 分裂之间的切换，NVIDIA 描述了使用 YIELD 指令以及其他细节，例如对间接分支的支持，我们在本讨论中省略 [Diamos 等，2015]。

图 3.9 展示了基于栈的重收敛时序示例，图 3.10 展示了 NVIDIA Volta 白皮书中描述的独立线程调度的潜在时序。在图 3.10 中，我们可以看到 Volta 将语句 A 和 B 与语句 X 和 Y 交错执行，这与图 3.9 中的行为形成对比。这种行为与上述收敛屏障机制（以及多路径 IPDOM [ElTantaway 等，2014]）一致。最后，图 3.11 展示了无栈架构如何执行图 3.5 中的自旋锁代码以避免 SIMT 死锁。

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```

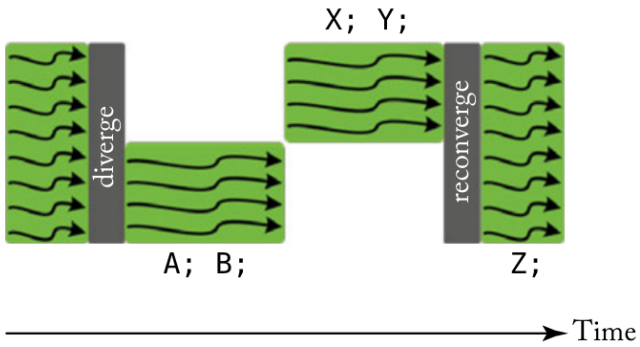


图 3.9　基于栈的重收敛行为示例（基于 Nvidia [2017] 的图 20）。

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
__syncwarp()
```

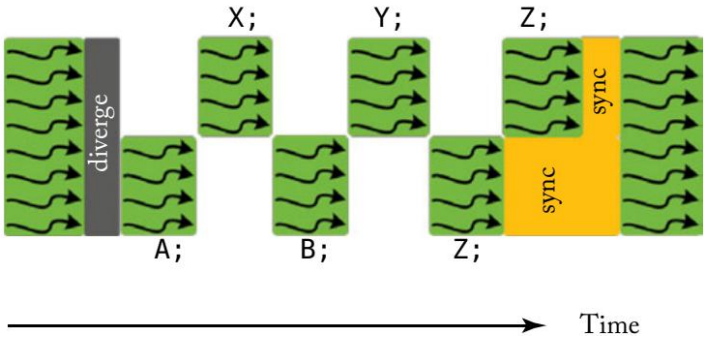


图 3.10　Volta 重收敛行为示例（基于 Nvidia [2017] 的图 23）。

3.1.3　WARP 调度

GPU 中的每个核心包含许多 warp。一个非常有趣的问题是这些 warp 应该按什么顺序调度。为简化讨论，我们假设每个 warp 在被调度时只发射一条指令，并且在该指令完成执行之前，该 warp 没有资格发射另一条指令。我们将在本章后面重新讨论这一假设。

如果内存系统是“理想的”，并在某个固定延迟内响应内存请求，那么理论上可以设计核心支持足够多的 warp，通过细粒度多线程来隐藏这种延迟。在这种情况下，可以认为通过以“轮询”顺序调度 warp，我们可以在给定吞吐量下减少芯片面积。在轮询中，warp 被赋予某种固定顺序，例如按线程标识符递增排序，调度器按此顺序选择 warp。这种调度顺序的一个特性是，它允许每条发射的指令有大致相等的完成时间。如果核心中 warp 的数量乘以每个 warp 的发射时间超过内存延迟，核心中的执行单元将始终保持忙碌。因此，原则上将 warp 数量增加到这一程度可以提高每核心吞吐量。

然而，存在一个重要的权衡：为了使不同的 warp 每个周期都能发射指令，每个线程必须拥有自己的寄存器（这避免了在寄存器和内存之间复制和恢复寄存器状态的需要）。因此，增加每核心的 warp 数量会增加用于寄存器文件存储的芯片面积比例，相对于专用于执行单元的面积比例。对于固定的芯片面积，增加每核心的 warp 数量将减少每芯片的总核心数。在实践中，内存的响应延迟取决于应用的局部性属性以及片外内存访问所遇到的争用程度。那么 32 3. SIMT 核心：指令与寄存器数据流

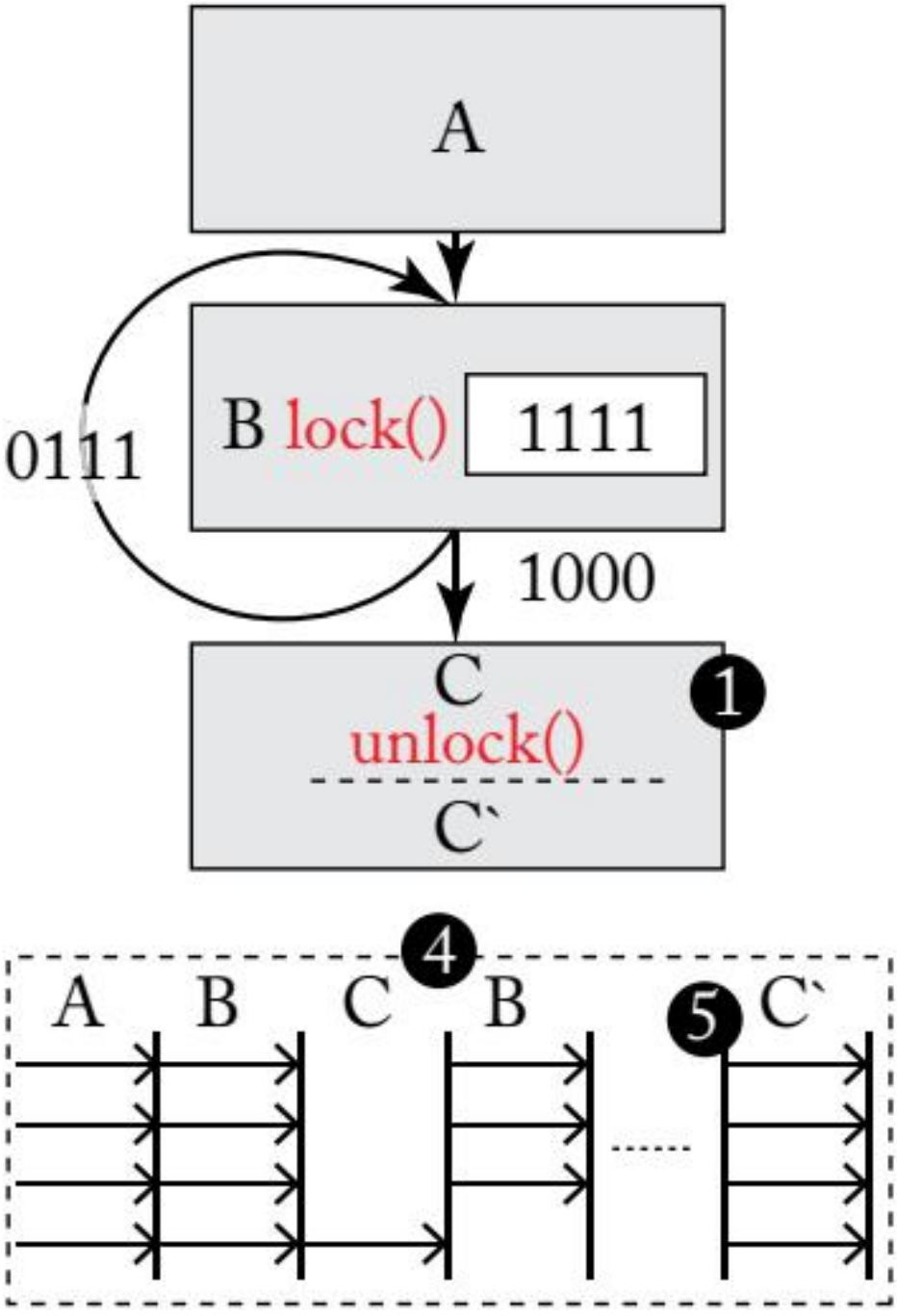


图 3.11 类似于收敛屏障的学术机制在图 3.5 自旋锁代码上的行为示例（基于 ElTantawy 和 Aamodt [2016] 的图 6（a））。

3.2　双循环近似

在考虑 GPU 的内存系统时，调度会起到什么作用？这是过去几年中大量研究的主题，我们将在为 GPU 微架构模型添加更多关于内存系统的细节后回到这个问题。不过，简而言之，局部性属性既可能有利于也可能不利于轮转调度：当不同线程在执行中的相似时间点共享数据时，例如在图形像素着色器中访问纹理映射时，线程取得相同进展是有益的，因为这可以增加在片上缓存中“命中”的内存引用数量，而轮转调度有助于实现这一点 [Lindholm et al., 2015]。类似地，当在时间上接近地访问地址空间中接近的位置时，访问 DRAM 会更高效，这也受到轮转调度的鼓励 [Narasiman et al., 2011]。另一方面，当线程主要访问不相交的数据时（这在更复杂的数据结构中往往发生），让给定线程被重复调度以最大化局部性可能是有益的 [Rogers et al., 2012]。

为了帮助减少每个核心为隐藏长执行延迟而必须支持的 warp 数量，能够在 warp 的先前指令尚未完成时发射后续指令是有帮助的。然而，前面描述的单循环微架构阻止了这一点，因为该设计中的调度逻辑只能访问线程标识符和要发射的下一条指令的地址。具体来说，它不知道 warp 要发射的下一条指令是否依赖于尚未完成执行的先前指令。为了提供这种依赖信息，必须首先从内存中取指以确定存在什么数据和/或结构冒险。为此，GPU 实现了指令缓冲区，在缓存访问之后将指令放入其中。使用单独的调度器来决定指令缓冲区中的若干指令中哪一个应接下来发射到流水线的其余部分。

指令内存实现为一级指令缓存，并由一个或多个二级（通常统一）缓存作为后备。指令缓冲区还可以结合指令未命中状态保持寄存器（MSHR）[Kroft，1981] 来帮助隐藏指令缓存未命中的延迟。在缓存命中或缓存未命中填充之后，指令信息被放入指令缓冲区。指令缓冲区的组织可以采取多种形式。一种特别直接的方法是为每个 warp 存储一条或多条指令。

接下来，让我们考虑如何检测同一 warp 内指令之间的数据依赖。在传统 CPU 架构中，检测指令间依赖有两种传统方法：记分板和保留站。保留站用于消除名称依赖，并引入了在面积和能耗上昂贵的关联逻辑需求。记分板可以设计为支持顺序执行或乱序执行。支持乱序执行的记分板，如 CDC 6600 中使用的，也相当复杂。另一方面，用于

单线程顺序 CPU 的记分板非常简单：每个寄存器在记分板中用一个位表示，每当发射一条将写入该寄存器的指令时，该位被置位。任何想要读取或写入记分板中对应位被置位的寄存器的指令都会被暂停，直到写入该寄存器的指令清除该位。这可以防止写后读和写后写冒险。当与顺序指令发射结合时，这个简单的记分板可以防止读后写冒险，前提是寄存器文件的读取被限制为按顺序发生，这通常在顺序 CPU 设计中是这种情况。由于它是最简单的设计，因此将消耗最少的面积和能量，GPU 实现了顺序记分板。然而，如下文所述，在支持多个 warp 时使用顺序记分板存在挑战。

上述简单顺序记分板设计的第一个问题是现代 GPU 中包含的寄存器数量非常大。每个 warp 最多 128 个寄存器，每个核心最多 64 个 warp，每个核心总共需要 8192 位

来实现记分板。

上述简单顺序记分板设计的另一个问题是，遇到依赖的指令必须反复在记分板中查找其操作数，直到它所依赖的先前指令将其结果写入寄存器文件。在单线程设计中，这几乎不会增加复杂性。然而，在顺序发射的多线程处理器中，来自多个线程的指令可能正在等待较早的指令完成。如果所有此类指令都必须探测记分板，则需要额外的读端口。最近的 GPU 每个核心支持多达 64 个 warp，并且最多 4 个操作数，允许所有 warp 每个周期探测记分板将需要 256 个读端口，这将非常昂贵。一种替代方案是限制每个周期可以探测记分板的 warp 数量，但这限制了可以考虑调度的 warp 数量。此外，如果检查的指令中没有一条是无依赖的，那么即使其他无法检查的指令恰好无依赖，也可能无法发射任何指令。

这两个问题都可以通过 Coon 等人 [2008] 提出的一种设计来解决。该设计并非为每个 warp 的每个寄存器保留一个比特，而是为每个 warp 包含少量条目（根据最近一项研究 [Lashgar et al., 2016] 估计约为 3 或 4 个），其中每个条目是一个寄存器的标识符，该寄存器将由一条已发射但尚未完成执行的指令写入。常规的按序记分板在指令发射和写回时都会被访问。而 Coon 等人的记分板则在指令被放入指令缓冲区时以及指令将其结果写入寄存器文件时被访问。

当一条指令从指令缓存中取出并放入指令缓冲区时，对应 warp 的记分板条目会与该指令的源寄存器和目标寄存器进行比较。这会生成一个简短的位向量，该 warp 的记分板中每个条目对应一个比特（例如 3 或 4 个比特）。如果记分板中的相应条目与该指令的任一操作数匹配，则相应比特被置位。然后，这个位向量会随指令一起复制到指令缓冲区中。直到所有比特都被清除，指令才有资格被指令调度器考虑，这可以通过将位向量的每个比特输入一个或非门来确定。指令缓冲区中的依赖比特会在指令将其结果写入寄存器文件时被清除。如果某个 warp 的所有条目都已用完，则要么所有 warp 的取指停顿，要么该指令被丢弃并必须重新取指。当一条已执行的指令准备写入寄存器文件时，它会清除记分板中分配给它的条目，并同时清除指令缓冲区中来自同一 warp 的任何指令的相应依赖比特。

在双循环架构中，第一个循环选择一个在指令缓冲区中有空间的 warp，查找其程序计数器，并执行指令缓存访问以获取下一条指令。第二个循环在指令缓冲区中选择一条没有未完成依赖的指令，并将其发射到执行单元。

3.3 三循环近似

如前所述，为了隐藏长内存延迟，有必要在每个核心中支持许多 warp；而为了支持 warp 之间的逐周期切换，有必要拥有一个大型寄存器文件，其中为每个正在执行的 warp 包含独立的物理寄存器。例如，在 NVIDIA 近期的 GPU 架构（如 Kepler、Maxwell 和 Pascal 架构）中，这样的寄存器文件包含 256 KB。现在，SRAM 存储器的面积与端口数成正比。寄存器文件的朴素实现需要每个周期发射的每条指令的每个操作数一个端

口。减少寄存器文件面积的一种方法是使用多个单端口存储器组来模拟大量端口。虽然可以通过将这些存储体暴露给指令集架构来实现这样的效果，但在一些 GPU 设计中，似乎使用了一种称为操作数收集器 [Coon et al., 2009，Lindholm et al., 2008b，Lui et al., 2008] 的结构，以更透明的方式实现这一点。操作数收集器实际上形成了第三个调度循环，如下所述。

为了更好地理解操作数收集器所解决的问题，首先考虑图 3.12，它展示了一种用于提供更高寄存器文件带宽的朴素微架构。该图显示了 GPU 指令流水线的寄存器读取阶段，其中寄存器文件由四个单端口逻辑寄存器组构成。实际上，由于寄存器文件非常大，每个逻辑组可能进一步分解为更多数量的物理组（未显示）。逻辑组通过交叉开关连接到暂存寄存器（标记为“流水线寄存器”），这些寄存器在将源操作数传递给 SIMD 执行单元之前对其进行缓冲。仲裁器控制对各个组的访问，并通过交叉开关将结果路由到适当的暂存寄存器。

图 3.13 展示了每个 warp 的寄存器到逻辑组的朴素布局。在该图中，warp 0（w0）的寄存器 r0 存储在组 0 的第一个位置，warp 0 的寄存器 r1 存储在组 1 的第一个位置，依此类推。如果计算所需的寄存器数量大于逻辑组的数量，则分配会回绕。例如，warp 0 的寄存器 r4 存储在组 0 的第二个位置。

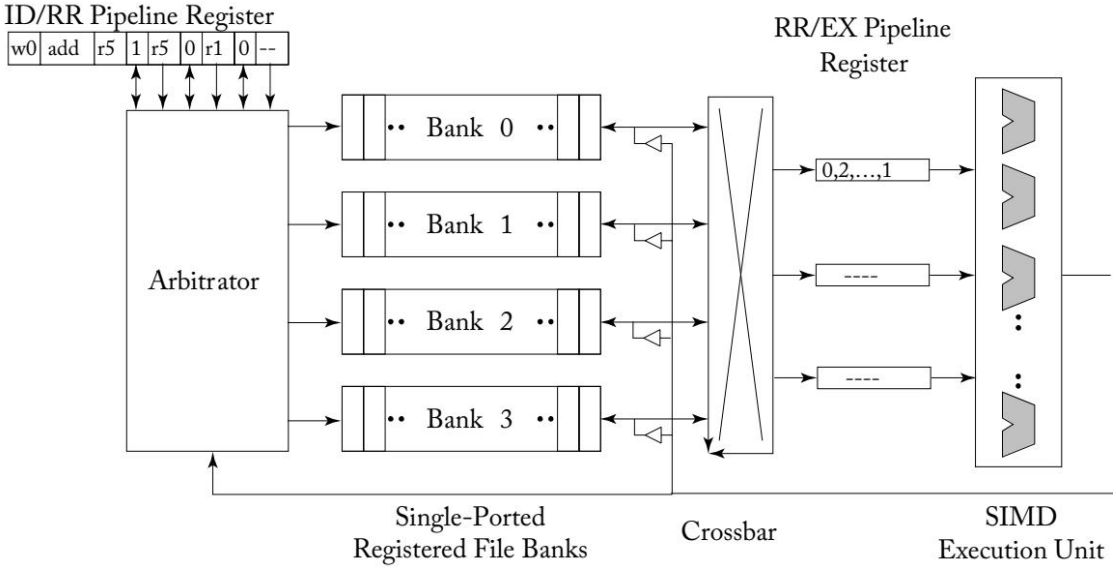


图 3.12　朴素的分组寄存器文件微架构。

图 3.14 展示了一个时序示例，突出了这种微架构如何损害性能。该示例涉及顶部显示的两条指令。第一条指令 i1 是一个多重加法操作，它读取寄存器 r5、r4 和 r6，这些寄存器分别分配在组 1、组 0 和组 2 中（图中用下标表示）。第二条指令 i2 是一个加法指令，它读取寄存器 r5 和 r1，两者都分配在组 1 中。图的中间部分显示了指令发射的顺序。在第 0 周期，warp 3 发射指令 i1；在第 1 周期，warp 0 发射指令 i2；在第 4 周期，warp 1 在因组冲突而延迟后发射指令 i2，如下所述。图的底部部分展示了不同指令对不同组的访问时序。在第 1 周期，来自 warp 3 的指令 i1 能够在第 1 周期读取

Bank 0	Bank 1	Bank 2	Bank 3
...	...	...	...
w1:r4	w1:r5	w1:r6	w1:r7
w1:r0	w1:r1	w1:r2	w1:r3
w0:r4	w0:r5	w0:r6	w0:r7
w0:r0	w0:r1	w0:r2	w0:r3

图 3.13 朴素的分组寄存器布局。

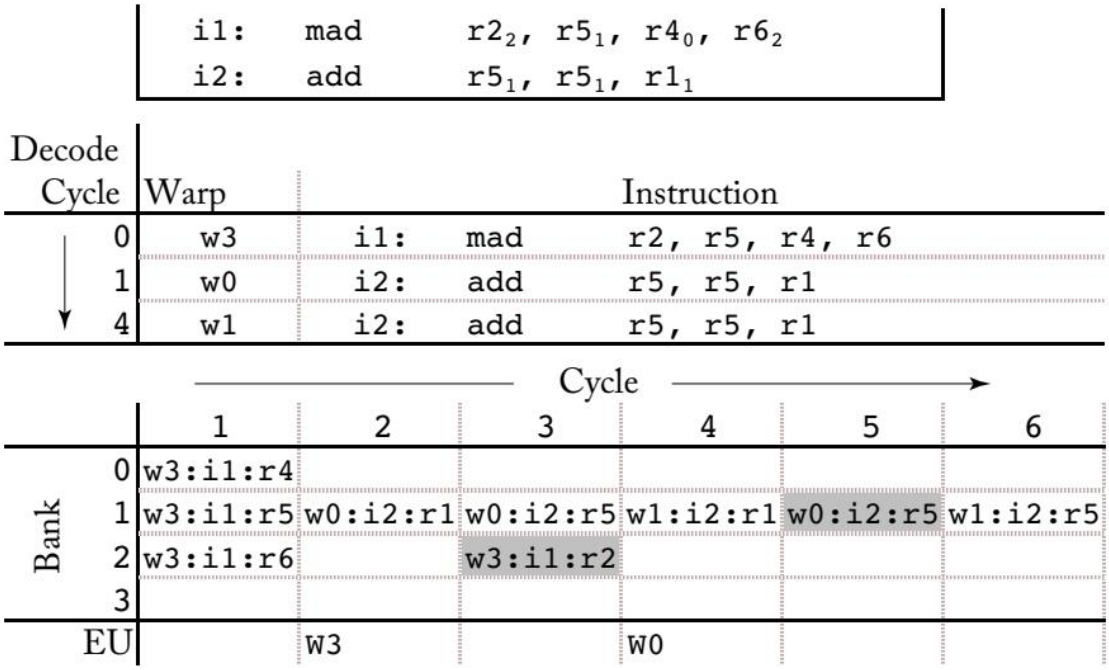


图 3.14 朴素分组寄存器文件的时序。

其所有三个源寄存器，因为它们映射到不同的逻辑组。然而，在第 2 周期，来自 warp 0 的指令 i2 只能读取其两个源寄存器中的一个，因为两者都映射到组 1。在第 3 周期，该指令的第二个源寄存器与来自 warp 3 的指令 i1 的写回并行读取。在第 4 周期，来自 warp 1 的指令 i2 能够读取其第一个源操作数，但无法读取第二个，因为两者同样都映射到组 1。在第 5 周期，来自 warp 1 的指令 i2 的第二个源操作数被阻止从寄存器文件读取，因为该组已被 warp 0 先前发射的指令 i2 的更高优先级写回所访问。最后，在第 6 周期，来自 warp 1 的 i2 的第二个源操作数从寄存器文件读取。总之，三条指令完成读取其源寄存器需要六个周期，在此期间许多组未被访问。

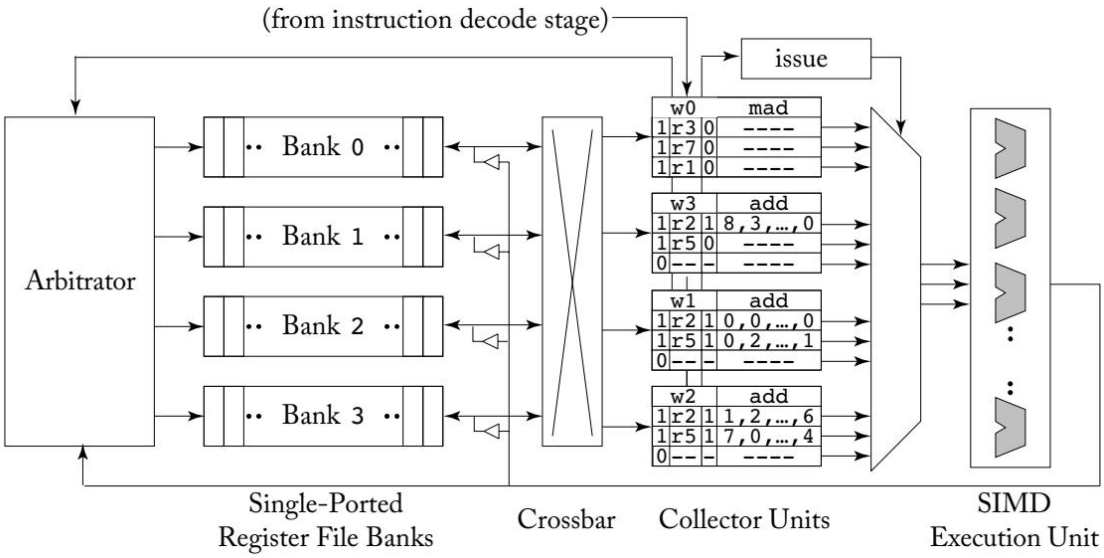


图 3.15　操作数收集器微架构（基于 Tor M. Aamodt 等人的图 6）。

3.3.1　操作数收集器

操作数收集器微架构 [Lindholm 等人，2008b] 如图 3.15 所示。关键变化在于暂存寄存器被替换为收集器单元。当指令进入寄存器读取阶段时，会为其分配一个收集器单元。由于存在多个收集器单元，多条指令可以重叠读取源操作数，这有助于在单个指令的源操作数之间存在存储体冲突时提高吞吐量。每个收集器单元包含执行一条指令所需的所有源操作数的缓冲空间。鉴于多条指令的源操作数数量较多，仲裁器更有可能实现更高的存储体级并行性，从而允许并行访问多个寄存器文件存储体。

操作数收集器利用调度来容忍发生的存储体冲突。这就留下了如何减少存储体冲突数量的问题。图 3.16 展示了 Coon 等人描述的用于帮助减少存储体冲突的修改后的寄存器布局。其思想是将来自不同 warp 的等价寄存器分配到不同的存储体。例如，在图 3.16 中，warp 0 的寄存器 r0 被分配到存储体 0，但 warp 1 的寄存器 r0 被分配到存储体 1。这并未解决单条指令的寄存器操作数之间的存储体冲突。然而，它确实有助于减少来自不同 warp 的指令之间的存储体冲突。特别是当 warp 进展相对均衡时（例如，由于轮询调度或两级调度 [Narasiman 等人，2011]，其中取指组中的各个 warp 按轮询顺序调度）。38 3. SIMT 核心：指令与寄存器数据流

图 3.17 展示了一个时序示例，顶部是一系列加法和乘法加法指令。中间显示了发射顺序。来自 warp 1 至 3 的三个 i1 实例在周期 0 至 2 发射。来自 warp 0 的指令 i2 的一个实例在周期 3 发射。注意，加法指令写入寄存器 r1，对于任何给定的 warp，该寄存器与源寄存器 r5 分配在同一存储体中。然而，与使用图 3.13 中寄存器布局的情况不同，这里不同的 warp 访问不同的存储体，这有助于减少一个 warp 的写回与其他 warp 中读取源操作数之间的冲突。底部显示了由于操作数收集器导致的访问的存储体级时序。在周期 1，来自 warp 1 的寄存器 r2 读取存储体 3。在周期 4，注意来自 warp 1 的寄存器 r1 的写回与来自 warp 3 的寄存器 r5 的读取以及来自 warp 0 的寄存器 r3 的读

Bank 0	Bank 1	Bank 2	Bank 3
...	...	...	...
w1:r4	w1:r5	w1:r6	w1:r7
w1:r0	w1:r1	w1:r2	w1:r3
w0:r4	w0:r5	w0:r6	w0:r7
w0:r0	w0:r1	w0:r2	w0:r3

图 3.16 交错排列的分组寄存器布局。

```
i1:  add   r1, r2, r5
i2:  mad   r4, r3, r7, r1
```

Cycle	Warp	Instruction
0	w1	i1: add $r1_2$, $r2_3$, $r5_2$
1	w2	i1: add $r1_3$, $r2_0$, $r5_3$
2	w3	i1: add $r1_0$, $r2_1$, $r5_0$
3	w0	i2: mad $r4_0$, $r3_3$, $r7_3$, $r1_1$

Cycle →

Bank	1	2	3	4	5	6
0		w2:r2		w3:r5		w3:r1
1			w3:r2			
2		w1:r5		w1:r1		
3	w1:r2		w2:r5	w0:r3	w2:r1	w0:r7
EU			w1	w2	w3	

图 3.17 操作数收集器的时序。

取并行进行。

到目前为止描述的操作数收集器的一个微妙问题是，由于它没有对不同指令何时准备好发射施加任何顺序，它可能允许写后读（WAR）冒险 [Mishkin 等人，2016]。如果来自同一 warp 的两条指令同时存在于操作数收集器中，且第一条指令读取的寄存器是第二条指令将要写入的，就可能发生这种情况。

40 3. SIMT 核心：指令与寄存器数据流　如果第一条指令的源操作数访问遇到重复的存储体冲突，第二条指令有可能在第一条指令读取（正确的）旧值之前向该寄存器写入新值。防止这种 WAR 冒险的一种方法是简单地要求来自同一 warp 的指令按程序顺序离开操作数收集器进入执行单元。Mishkin 等人 [2016] 探索了三种硬件复杂度低的潜在解决方案，并评估了它们的性能影响。第一种是提交时释放的 warpboard，它允许每个 warp 最多执行一条指令。不出所料，他们发现这会对性能产生负面影响，在某些情况下性能降低近两倍。他们的第二个提案是读取时释放的 warpboard，它允许每个 warp 一次只有一条指令在操作数收集器中收集操作数。该方案在他们研究的工作负载上导致最多 10% 的减速。最后，为了允许操作数收集器中的指令级并行，他们提出了一种 bloomboard 机制，使用小型布隆过滤器来跟踪未完成的寄存器读取。与（错误地）允许 WAR 冒险相比，这导致的影响不到几个百分点。另外，Gray 进行的一项分析表明，NVIDIA 的 Maxwell GPU 引入了“读依赖屏障”，由特殊的“控制指令”管理，可用于避免某些指令的 WAR 冒险（见第 2.2.1 节）。

3.3.2　指令重放：处理结构冒险

GPU 流水线中结构冒险的潜在原因有很多。例如，寄存器读取阶段可能用完操作数收集器单元。许多结构冒险的来源与内存系统有关，我们将在下一章更详细地讨论。通常，由 warp 执行的单条内存指令可能需要分解为多个独立操作。这些独立操作中的每一个都可能在一个给定周期内完全利用流水线的一部分。

当指令在 GPU 流水线中遇到结构冒险时会发生什么？在单线程顺序 CPU 流水线中，标准的解决方案是暂停较年轻的指令，直到遇到暂停条件的指令可以继续推进。这种方法在高度多线程的吞吐量架构中可以说不太理想，至少有两个原因。首先，鉴于寄存器文件的大尺寸以及支持完整图形流水线所需的许多流水线阶段，分发暂停信号可能会影响关键路径。流水化暂停周期分发导致需要引入额外的缓冲，增加面积。其次，暂停来自一个 warp 的指令可能导致来自其他 warp 的指令在其后暂停。如果这些指令不需要导致暂停的指令所需的资源，吞吐量可能会受到影响。

为避免这些问题，GPU 实现了一种指令重放机制。指令重放也见于某些 CPU 设计中，当推测性地调度一条依赖于具有可变延迟的较早指令的指令时，它被用作恢复机制。例如，加载操作可能在一级缓存中命中或未命中，但以高频率计时的 CPU 设计可能将一级缓存访问流水线化多达四个时钟周期。一些 CPU 会推测性地唤醒依赖于加载操作的指令，以提高单线程性能。相比之下，GPU 避免推测，因为推测往往浪费能

量并降低吞吐量。相反，GPU 使用指令重放来避免堵塞流水线以及由停顿引起的电路面积和/或时序开销。

为实现指令重放，GPU 可以将指令保留在指令缓冲区中，直到已知它们已完成或指令的所有单独部分都已执行 [Lindholm et al., 2015]。

本节基于 Wilson Fung 的博士论文 [Fung，2015]。

理想情况下，同一 warp 内的线程通过相同的控制流路径执行，以便 GPU 可以在 SIMD 硬件上以锁步方式执行它们。鉴于线程的自主性，当一个 warp 的线程在数据相关分支处发散到不同目标时，该 warp 可能会遇到分支发散。现代 GPU 包含特殊硬件来处理 warp 中的分支发散。第 3.1.1 节描述了基线 SIMT 栈，本书中的基线 GPU 架构使用该栈。基线 SIMT 栈通过串行化不同目标的执行来处理 warp 中的分支发散。虽然基线 SIMT 栈对大多数现有 GPU 应用正确解决了分支发散，但它存在以下缺陷。

较低的 SIMD 效率在存在分支发散的情况下，基线 SIMT 栈串行化每个分支目标的执行。当每个目标被执行时，SIMT 栈仅激活运行该目标的标量线程子集。这导致 SIMD 硬件中的一些通道空闲，降低了整体 SIMD 效率。

不必要的串行化基线 SIMT 栈对每个分支目标的串行化执行并非功能正确性所必需。GPU 编程模型不在 warp 内的标量线程之间强加任何隐式数据依赖——它们必须通过共享内存和屏障显式通信。GPU 可以交错执行发散 warp 的所有分支目标，以利用 SIMD 硬件中的空闲周期。

不充分的 MIMD 抽象通过强制发散的 warp 在编译器定义的再汇聚点重新汇聚，基线 SIMT 栈在每个再汇聚点隐式强加了 warp 范围的同步点。这对许多现有 GPU 应用有效。然而，这种隐式同步可能与其他用户实现的同步机制（如细粒度锁）发生病态交互，导致 warp 死锁。编译器定义的再汇聚点也没有考虑由系统级构造（如异常和中断）引入的控制流发散。

3.4 分支发散的研究方向

面积成本虽然每个 warp 的基线 SIMT 栈的面积需求仅为 32×64 位（或低至 6×64 位），但面积随 GPU 中在飞 warp 的数量而扩展。在分支发散罕见的典型 GPU 应用中，SIMT 栈占用的面积本可用于以其他方式提升应用吞吐量（例如，大缓存、更多 ALU 单元等）。

工业界和学术界都提出了解决上述缺陷的替代方案。各种提案可分类如下：warp 压缩、warp 内发散路径管理、增加 MIMD 能力和复杂度降低。一些提案包含捕获多个类别方面的改进，因此被多次提及。

3.4.1　WARP 压缩

由于 GPU 实现细粒度多线程以容忍长内存访问延迟，每个 SIMT 核心中有许多 warp，总共有数百到数千个标量线程。由于这些 warp 通常运行相同的计算内核，它们可能遵循相同的执行路径，并在同一组数据相关分支处遇到分支发散。因此，发散分支的每个目标可能由大量线程执行，但这些线程分散在多个静态 warp 中，每个 warp 单独处理发散。

在本节中，我们总结了一系列利用这一观察来提高遭受分支发散的 GPU 应用性能的研究。这一系列的提案都涉及新颖的硬件机制，将来自不同静态 warp 的线程压缩成新的动态 warp，以提高这些发散 GPU 应用的整体 SIMD 效率。这里，静态 warp 指的是当标量线程从内核启动中产生时由 GPU 硬件形成的 warp。在我们的基线 GPU 架构中，这种安排在 warp 的整个执行过程中是固定的。将标量线程安排到静态 warp 中是由 GPU 硬件强加的任意分组，对编程模型基本不可见。

动态 Warp 形成。动态 Warp 形成（DWF）[Fung et al., 2007，Fung et al., 2009] 利用这一观察，通过将执行相同指令的这些分散线程重新排列成新的动态 warp。在发散分支处，DWF 可以通过将分散在多个发散静态 warp 中的线程压缩成更少的非发散动态 warp 来提高应用的整体 SIMD 效率。这样，DWF 可以在 SIMD 硬件上捕获 MIMD 硬件的显著部分优势。然而，DWF 要求 warp 在短时间窗口内遇到相同的发散分支。DWF 的这种时间依赖性使其对 warp 调度策略非常敏感。

Fung 和 Aamodt [2011] 的后续工作指出了 DWF 的两个主要性能问题：（1）贪婪的调度策略可能使某些线程饥饿，导致 SIMD 效率降低；（2）DWF 中的线程重组增加了非合并内存访问和共享内存存储体冲突。这些问题导致 DWF 减慢了许多现有的 GPU 应用程序。此外，依赖于静态 warp 中隐式同步的应用程序在使用 DWF 时会执行不正确。

上述问题可以通过改进的调度策略部分解决，该策略有效地将计算内核分为两类区域：发散区域和非发散（一致）区域。发散区域从 DWF 中显著受益，而一致区域没有分支发散，但容易受到 DWF 问题的影响。我们发现，通过强制 DWF 在一致区域将标量线程重新排列回其静态 warp，可以显著减少 DWF 问题的影响。

线程块压缩。线程块压缩（TBC）[Fung and Aamodt，2011] 基于这一见解，观察到将线程不断重新排列到新的动态 warp 中不会产生额外好处。相反，重新排列（或压缩）只需要在发散分支之后、发散区域的开始处以及其重汇聚点（一致区域的开始）之前发生。我们注意到，现有的每 warp SIMT 栈（在第 3.1.1 章中描述）在发散分支的重汇聚点隐式同步发散到不同执行路径的线程，在执行一致区域之前将这些发散线程合并回静态 warp。TBC 扩展了 SIMT 栈以涵盖在同一核心中执行的所有 warp，强制它们在发散分支和重汇聚点同步和压缩，以实现稳健的 DWF 性能优势。然而，在每个发散分支处同步核心内的所有 warp 以进行压缩可能会大大减少可用的线程级并行性（TLP）。GPU 架构依赖于丰富的 TLP 来容忍流水线和内存延迟。

TBC 通过将压缩限制仅在线程块内发生，在 SIMD 效率和 TLP 可用性之间达成妥协。GPU 应用程序通常在一个核心上并发执行多个线程块，以重叠同步和内存延迟。TBC 利用这种软件优化来重叠发散分支处的压缩开销——当一个线程块中的 warp 在发散分支处同步以进行压缩时，其他线程块中的 warp 可以使硬件保持忙碌。它扩展了每 warp SIMT 栈以涵盖线程块中的 warp。warp 调度逻辑使用这个线程块范围的 SIMT 栈来确定线程块中的 warp 何时应同步并压缩成新的 warp 集合。结果是一种更加稳健和简单的机制，它捕获了 DWF 的大部分好处，而没有病态行为。

大 warp 微架构。大 warp 微架构（LWM）[Narasiman et al., 2011] 类似于 TBC，扩展了 SIMT 栈以管理一组 warp 的重汇聚。然而，LWM 不是将压缩限制在分支和重汇聚点，而是要求组内的 warp 以完全锁步执行，以便它可以在每条指令处压缩组。这比 TBC 更多地减少了可用的 TLP，但允许

LWM 使用谓词指令以及无条件跳转来执行压缩。与 TBC 类似，LWM 将运行在同一核心上的 warp 分成多个组，并将压缩限制为仅在组内发生。它还选择了一种更复杂的记分板微架构，以线程粒度跟踪寄存器依赖性。这允许组中的一些 warp 稍微领先于其他 warp 执行，以补偿由于锁步执行而损失的 TLP。

压缩充分性预测器。Rhu 和 Erez [2012] 通过压缩充分性预测器（CAPRI）扩展了 TBC。该预测器识别在每个分支处将线程压缩到少数 warp 的有效性，并且仅在预测压缩将产生益处的分支处同步线程。这回收了由于 TBC 的无益停顿和压缩而损失的 TLP。Rhu 和 Erez [2012] 还表明，类似于单级分支预测器的简单基于历史的预测器足以实现高精度。

warp 内压缩。Vaidya 等人 [2013] 提出了一种低复杂度的压缩技术，有利于在较窄的硬件单元上执行多个周期的宽 SIMD 执行组。他们的基本技术将单个执行组划分为多个与硬件宽度匹配的子组。遭受发散的 SIMD 执行组可以通过跳过完全空闲的子组在窄硬件上运行得更快。为了创建更多完全空闲的子组，他们提出了一种混洗机制，在发散时将元素压缩到更少的子组中。

同时 warp 交织。Brunie 等人 [2012] 提出了同时分支和 warp 交织（SBI 和 SWI）。他们扩展了 GPU SIMT 前端以支持每周期发出两条不同的指令。他们通过将 warp 加宽到原始大小的两倍来补偿这种增加的复杂性。SWI 将一个遭受发散的 warp 的指令与另一个发散 warp 的指令共同发出，以填补分支发散留下的空白。

对寄存器文件微架构的影响 为了避免在 SIMT 核心之间引入额外的通信流量，硬件压缩方案通常发生在 SIMT 核心本地。由于压缩后的线程都位于共享同一寄存器文件的同一核心上，因此可以通过更灵活的寄存器文件设计来执行压缩而不移动其架构状态 [Fung et al., 2007]。

如本章前文所述，GPU 寄存器文件采用大型单端口 SRAM 存储体实现，以最大化面积效率。同一 warp 中线程的寄存器存储在同一个 SRAM 存储体的连续区域中，从而可以通过单个宽端口一起访问。这样既能实现高带宽的寄存器文件访问，又能分摊

寄存器文件访问控制硬件的开销。硬件 warp 压缩会创建动态 warp，这些 warp 可能不遵循这种寄存器排列方式。Fung 等人 [2007] 提出了一种更灵活的寄存器文件设计，采用具有窄端口的 SRAM 存储体。该设计使用更多的 SRAM 存储体来维持相同的带宽。

动态微内核。Steffen 和 Zambreno[2010] 利用动态微内核提高了 GPU 上光线追踪的 SIMD 效率。程序员获得了一些原语，用于将数据依赖循环中的迭代分解为连续的微内核启动。这种分解本身并不能提高并行性，因为每次迭代都依赖于前一次迭代的数据。相反，启动机制通过将剩余的活动线程压缩到少数几个 warp 中，改善了同一核心内不同线程之间的负载不均衡。它与其他硬件 warp 压缩技术的不同之处在于，压缩会连同线程的架构状态一起迁移线程，并使用每核心的暂存存储器作为暂存区。

第 3.4.1 节总结了一系列在软件中实现 warp 压缩的研究，这些研究不需要更灵活的寄存器文件设计。相反，这些方案引入了额外的内存流量，以将线程从一个 SIMT 核心迁移到另一个 SIMT 核心。

软件中的 Warp 压缩　在现有的 GPU 上，提高应用程序 SIMD 效率的一种方法是通过软件 warp 压缩——使用软件根据线程/工作项的控制流行为对其进行分组。重新分组涉及在内存中移动线程及其私有数据，可能会引入显著的内存带宽开销。下面我们重点介绍几项关于软件压缩技术的工作。

条件流 [Kapasi 等人，2000] 将这一概念应用于流计算。它将具有潜在发散控制流的流处理器计算内核拆分为多个内核。在发散分支处，内核根据每个数据元素的分支结果将其数据流拆分为多个流。每个流随后由单独的内核处理，并在控制流发散结束时合并回来。

Billeter 等人 [2009] 提出使用并行前缀和来实现 SIMD 流压缩。流压缩将具有各种任务的元素流重新组织为相同任务的紧凑子流。该实现利用了 GPU 片上暂存存储器的访问灵活性来实现高效率。Hoberock 等人 [2009] 提出了一种用于光线追踪的延迟着色技术，该技术使用流压缩来提高具有许多材质类别的复杂场景中像素着色的 SIMD 效率。每个材质类别都需要其独特的计算。组合每个材质类别计算的像素着色器在 GPU 上运行效率低下。流压缩将击中具有相似材质类别物体的光线分组，使 GPU SIMD 硬件能够高效地为这些像素执行着色器。

Zhang 等人 [2010] 提出了一种运行时系统，该系统动态地将线程重新映射到不同的 warp 中，以提高 SIMD 效率以及内存访问的空间局部性。该运行时系统采用流水线系统，由 CPU 执行动态重映射，GPU 对重映射后的数据/线程执行计算。

Khorasani 等人 [2015] 提出了集体上下文收集（Collective Context Collection, CCC），这是一种编译器技术，可转换给定的具有潜在分支发散惩罚的 GPU 计算内核

以提高其在现有 GPU 上的 SIMD 效率。CCC 专注于每个线程在每一步执行不规则计算量的计算内核，例如通过不规则图进行广度优先搜索。CCC 不是为每个线程分配一个节点（或其他应用中的任务），而是首先转换计算内核，使每个线程处理多个节

点，并且节点到 warp（注意：不是线程）的分配在内核启动之前确定。然后，CCC 转换计算内核，使 warp 中的每个线程可以将任务的上下文卸载到存储在共享内存中的 warp 专用栈。在当前任务集上经历低 SIMD 效率的 warp 可以将任务卸载到栈中，并使用这些卸载的任务来填充在处理后续任务集时空闲的线程。实际上，CCC 通过将来自多个 warp 的任务分组到更少的 warp 集中，然后通过存储在快速片上共享内存中的 warp 专用栈将发散任务压缩到每个 warp 内的更少迭代中，从而执行“warp 压缩”。

Warp 内线程分配的影响 在本书中研究的基线 GPU 架构中，具有连续线程 ID 的线程被静态融合在一起形成 warp。关于线程到 warp 或 warp 中通道的静态分配，学术研究很少。这种默认的连续映射对于大多数工作负载都很有效，因为相邻线程倾向于访问相邻数据，从而改善内存合并。然而，一些研究已经考察了替代方案。

SIMD 通道排列。Rhu 和 Erez[2013b] 观察到，线程 ID 到 warp 中连续线程的连续映射对于本节前面描述的 warp 压缩技术来说不是最优的。大多数 warp 压缩和形成工作的一个关键限制是，当线程被分配到新的 warp 时，它们不能被分配到不同的通道，否则它们的寄存器文件状态将不得不移动到向量寄存器中的不同通道。Rhu 和 Erez 观察到，程序的结构会使某些控制流路径偏向某些 SIMD 通道。这种偏向使得实现压缩更加困难，因为走相同路径的线程往往位于相同的通道，从而阻止这些线程合并在一起。Rhu 和 Erez 提出了几种不同的线程映射排列，以消除这些程序性偏向并显著提高压缩率。

Warp 内循环压缩。Vaidya 等人 [2013] 利用 SIMD 数据通路宽度并不总是等于 warp 宽度这一事实。例如，在 NVI[2009] 中，SIMD 宽度为 16，但 warp 大小为 32。这意味着一个 32 线程的 warp 需要 2 个核心周期来执行。Vaidya 等人 [2013] 观察到，当发生分歧时，如果一条指令屏蔽了连续一个 SIMD 宽度的线程，那么该指令可以仅在一个周期内发射，跳过被屏蔽的通道。他们将这种技术称为循环压缩。然而，如果被屏蔽的线程不连续，基本技术不会带来任何性能提升。为了解决这个问题，他们提出了一种交织循环压缩，重新排列哪些线程位于哪些通道，以创造更多循环压缩的机会。

Warp 标量化。其他工作，如 Yang 等人 [2014] 的工作，认为当 warp 中的线程操作相同数据时，SIMT 编程模型效率低下。许多解决方案提出在流水线中包含一个标量单元，用于编译器或程序员可以预先识别为标量的工作。AMD 的 Graphics Core Next（GCN）架构为此包含了一个标量流水线。更多细节见第 3.5 节。

3.4.2 Warp 内分歧路径管理

虽然带有直接后支配重汇合点的 SIMT 栈可以处理具有任意控制流的分支分歧，但它在多个方面还可以进一步改进。

1. 分歧到不同分支目标的线程可以交错执行，以利用 SIMD 硬件中的空闲周期。

2. 虽然分歧分支的直接后支配点是确定的汇合点，但分歧到不同分支目标的线程

可能能够在分歧分支的直接后支配点之前汇合。

以下小节重点介绍了几项试图在这两个方面改进 SIMT 栈的工作。

多路径并行　当 warp 在分支处发生分歧时，线程被分成多个组，称为 warp 分裂。每个 warp 分裂由遵循相同分支目标的线程组成。在基线单路径 SIMT 栈中，来自同一 warp 的 warp 分裂逐个执行，直到该 warp 分裂到达其重汇合点。这种串行化适合相对简单的硬件实现，但并非功能正确性所必需。warp 中的线程拥有独立的寄存器，并通过内存操作和同步操作（如屏障）显式通信。换句话说，来自同一 warp 的每个 warp 分裂都可以并行执行。我们将这种执行模式称为多路径执行模式。

虽然不同的 warp 分裂可能不会在同一周期在同一硬件上执行（毕竟它们运行不同的指令），但它们可以像多个 warp 在同一数据通路上交错执行一样，在同一硬件上交错执行。这样，多路径执行模式提升了应用程序中可用的线程级并行性（TLP），以容忍内存访问延迟。尽管 SIMD 效率没有提高，但多路径执行提升了内存受限应用程序的整体性能，在这些程序中，SIMT 核心有大量空闲周期可以用有用的工作填充。

例 3.1 展示了一个可能受益于多路径执行的简短计算内核。在这个例子中，两个分支目标中的代码路径都包含从内存加载。在单路径 SIMT 栈中，块 B 和 C 各自串行执行，直到相应的 warp 分裂到达块 D（重汇合点），即使 warp 分裂因等待内存数据而停顿。这会使整个 warp 停顿，在数据通路中引入空闲周期，如图所示

在图 3.18 中，这些空闲周期必须由其他 warp 的工作填充。使用多路径执行，块 B 和 C 的 warp 分裂可以交错执行，消除由内存访问引入的这些空闲周期。

```
Algorithm 3.1 Example of multi-path parallelism with branch divergence.

X = data[i]; // block A
if (X > 3)
    result = Y[i] * i; // block B
else
    result = Z[i] + i; // block C
return result; // block D
```

动态 Warp 细分。Meng 等人 [2010] 提出动态 warp 细分（DWS）是第一个利用多路径执行带来的 TLP 提升的方案。DWS 通过 warp 分裂表扩展 SIMT 栈，将分歧的 warp 细分为并发的 warp 分裂。每个执行分歧分支目标的 warp 分裂可以并行执行，以回收因内存访问导致的硬件空闲。当 warp 中只有部分线程在 L1 数据缓存中命中时，也会在内存分歧处创建 warp 分裂。DWS 不是等待所有线程获取数据，而是分裂 warp，允许在缓存中命中的 warp 分裂先行执行，可能为未命中的线程预取数据。

双路径执行。Rhu 和 Erez[2013a] 提出了双路径 SIMT 栈（DPS），通过限制每个 warp 只能执行两个并发的 warp 分裂，解决了 DWS 的一些实现缺陷。虽然这种限制使 DPS 只能获得完整 DWS 优势的一部分，但它带来了简单得多的硬件设计。DPS 只需要用一组额外的 PC 和活动掩码扩展基线 SIMT 栈，以编码额外的 warp 分裂。只有

Algorithm 3.1 Example of multi-path parallelism with branch divergence.

```
X = data[i];          // block A
if( X > 3 )
  result = Y[i] * i;  // block B
else
  result = Z[i] + i;  // block C
return result;        // block D
```

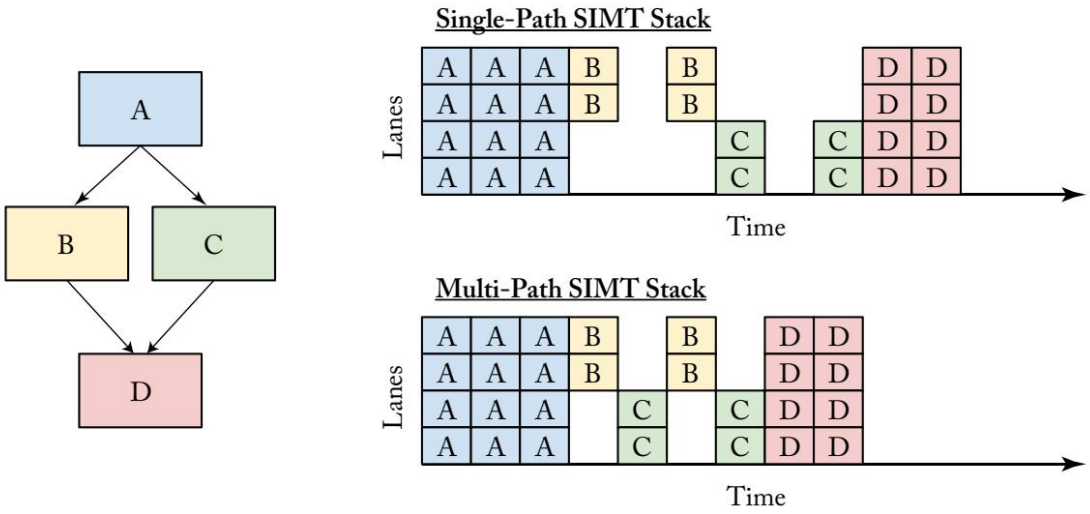


图 3.18 分支分歧处的多路径执行。

warp 栈顶条目处的两个 warp 分裂并行执行；同一 warp 中的其他所有 warp 分裂都暂停，直到其条目到达栈顶。DPS 还扩展了记分板，以独立跟踪每个 warp 分裂的寄存器依赖关系。这使得双路径执行模型能够比使用基线记分板的 DWS 实现更大的 TLP。

多路径执行。ElTantaway 等人 [2014] 通过多路径执行模型（MPM）消除了双路径限制。MPM 用两个表替换 SIMT 栈：一个 warp 分裂表维护分歧 warp 的 warp 分裂集合，一个重汇合表同步所有具有相同重汇合点的 warp 分裂。

在发散分支处，重收敛表中会创建一个新条目，记录该发散分支的重收敛点（即其直接后支配节点）。在 warp 分裂表中会创建多个（通常为两个）条目，每个 warp 分裂对应一个。每个 warp 分裂条目维护该 warp 分裂的当前 PC、其活动掩码、重收敛 PC（RPC）以及指向重收敛表中对应条目的 R 索引。warp 分裂表中的每个 warp 分裂都可供执行，直到其 PC == RPC。此时，相应的重收敛表条目会更新，以反映来自该 warp 分裂的线程已到达重收敛点。当所有待处理线程都到达重收敛点后，重收敛表条目被释放，并创建一个新的 warp 分裂条目，使重收敛后的线程从 RPC 开始处于活动状态。

MPM 还扩展了记分板，以跟踪每个线程的寄存器依赖关系，而无需为每个线程完全复制记分板（否则会因显著的面积开销而使 MPM 不切实际）。这是一个关键扩展，使得 warp 分裂能够真正独立地执行——如果没有该扩展，一个 warp 分裂的寄存器依赖可能会被误认为是同一 warp 中另一个 warp 分裂的依赖。

MPM 进一步扩展了机会性提前重收敛，从而提升了非结构化控制流的 SIMD 效率（见第 3.4.2 节）。

DWS 以及本节讨论的其他技术与第 3.4.1 节讨论的 warp 压缩技术是正交的。例如，TBC 中的块级 SIMT 栈可以通过 DWS 进行扩展，以提升可用的 TLP。

更好的收敛　基于后支配节点（PDOM）栈的重收敛机制 [Fung et al., 2007，Fung et al., 2009] 使用通过统一算法识别的重收敛点，而不是通过将源代码中的控制流惯用法转换为指令来识别 [AMD，2009，Coon and

Lindholm，2008，Levinthal and Porter，1984]。选择发散分支的直接后支配节点作为重收敛点，该节点是程序中发散线程保证重收敛的最早位置。在某些情况下，线程可以在更早的位置重收敛，如果硬件能够利用这一点，将提高 SIMD 效率。我们认为这一观察促使了近期 NVIDIA GPU 中引入 break 指令 [Coon and Lindholm，2008]。

示例 3.2 中的代码（来自 [Fung and Aamodt，2011]）展示了这种更早的重收敛。它产生了图 3.19 中的控制流图，其中边标记了各个标量线程遵循该路径的概率。块 F 是 A 和 C 的直接后支配节点，因为 F 是从 A（或 C）开始的所有路径首次重合的位置。在基线机制中，当 warp 在 A 处发散时，重收敛点被设置为 F。然而，从 C 到 D 的路径很少被遵循，因此在大多数情况下线程可以在 E 处更早地重收敛。

```
Algorithm 3.2 Example for branch reconvergence earlier than immediate post-dominator.
while (i < K) {
    X = data[i]; // block A
    if (X == 0)
    result[i] = Y; // block B
    else if (X == 1) // block C
    break; // block D
    i++; // block E
}
return result[i]; // block F
```

可能收敛点。Fung 和 Aamodt [2011] 提出用可能收敛点扩展 SIMT 栈。该扩展为每个 SIMT 栈条目添加两个新字段：一个用于可能收敛点的 PC（LPC），另一个（LPos）是一个指针，记录当分支的可能收敛点不同于直接后支配节点时创建的特殊可能收敛条目的栈位置。每个分支的可能收敛点可以通过控制流分析或性能分析信息（可能在运行时收集）来识别。Fung 和 Aamodt [2011] 的提案将可能收敛点限制为最接近的封闭后向分支，以捕捉循环内“break”语句的影响 [Coon and Lindholm，2008]。

当 warp 在具有可能收敛点的分支处发散时，三个条目被压入 SIMT 栈。第一个条目是 LPC 条目，为该分支的可能收敛点创建。另外两个条目分别用于分支的跳转和顺序执行，如基线机制中那样创建。这些其他条目中的 LPC 字段填充为发散分支的可能收敛点，LPos 字段填充为 LPC 条目的栈位置。LPC 条目的 RPC 设置为发散分支的直接后支配节点，即确定的重收敛点，以便该条目中的线程将重收敛到确定的重收敛点。

当 warp 使用 SIMT 栈顶条目执行时，它将其 PC 与 RPC 字段（如同基线 SIMT 栈

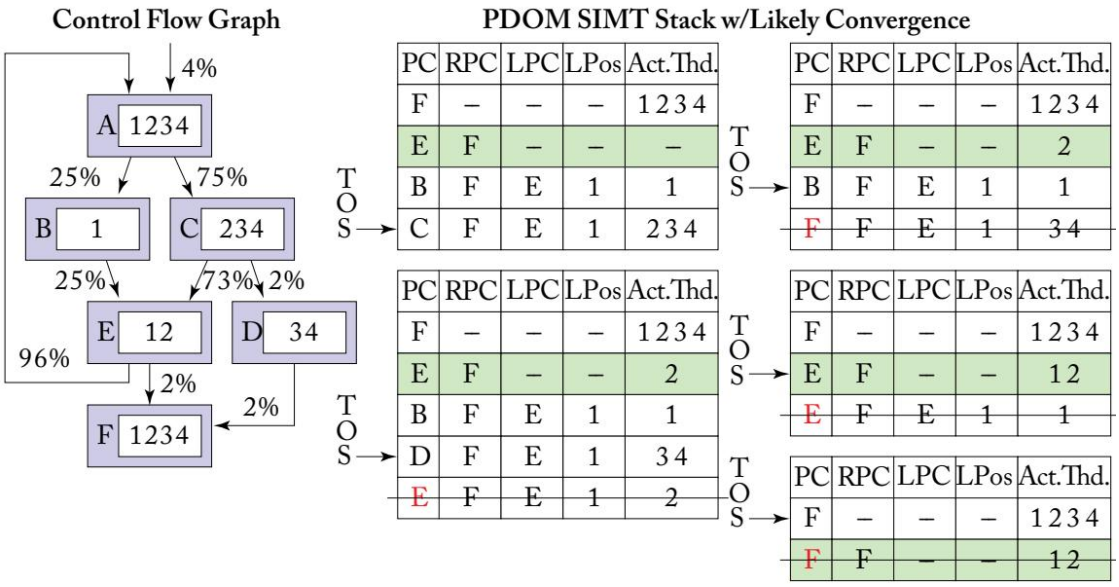


图 3.19 直接后支配节点之前的更早重收敛点。可能收敛点捕捉了 E 处的这种更早重收敛。

一样）以及 LPC 字段进行比较。如果 PC == LPC，则弹出 SIMT 栈，并将该弹出条目中的线程合并到 LPC 条目中。否则，如果 PC == RPC，则直接弹出 SIMT 栈——RPC 条目已在其活动掩码中记录了这些线程。当 LPC 条目到达 SIMT 栈顶时，它像任何其他 SIMT 栈条目一样执行，或者如果其活动掩码为空则直接弹出。

线程前沿。Diamos 等人 [2011] 完全摒弃了 SIMT 栈，转而提出通过线程前沿在发散后重收敛线程。支持线程前沿的编译器根据拓扑顺序对内核中的基本块进行排序。这样，在较高 PC 处执行的线程永远不会跳转到较低 PC 的指令。循环通过将循环出口放在循环体末尾来处理。通过这种排序的代码布局，发散的 warp 最终将通过优先处理具有较低 PC 的线程（允许它们赶上）来重收敛。

与具有直接后支配节点重收敛的 SIMT 栈相比，通过线程前沿进行重收敛对于非结构化控制流的应用产生更高的 SIMD 效率。多表达式条件语句的求值语义和异常的使用都可能生成具有非结构化控制流的代码。用可能收敛点扩展的 SIMT 栈

汇聚点可以在非结构化控制流的应用上产生类似的 SIMD 效率提升；然而，SIMT 栈中的每个条目可能只有有限数量的可能汇聚点，而线程边界方法则没有这种限制。

机会性提前重汇聚。ElTantaway 等人 [2014] 提出了机会性提前重汇聚（OREC），无需任何额外的编译器分析，即可提升具有非结构化控制流的 GPU 应用的 SIMD 效率。OREC 基于同一论文中引入的多路径（MP）SIMT 栈（见第 3.4.2 节）。MP SIMT 栈使用一个单独的 warp 分裂表，保存当前可供执行的 warp 分裂集合。在发散分支处，使用分支目标 PC 和发散分支的重汇聚 PC 创建新的 warp 分裂。使用 OREC 时，硬件不是简单地将这些新的 warp 分裂插入 warp 分裂表，而是搜索 warp 分裂表，寻找具有相同起始 PC 和 RPC 的现有 warp 分裂。如果存在这样的 warp 分裂，硬件在重汇聚表中创建一个提前重汇聚点，以在原始 RPC 之前汇聚两个 warp 分裂。提前重汇聚点在特定 PC 处同步两个 warp 分裂，使得即使现有 warp 分裂已经沿着发散路径前进，它

们也可以被合并。在 ElTantaway 等人 [2014] 中，提前重汇聚点是现有 warp 分裂的下一个 PC。

3.4.3　添加 MIMD 能力

以下提案通过纳入有限数量的 MIMD 能力，提高了 GPU 与发散控制流的兼容性。所有这些提案都提供两种操作模式：

• SIMD 模式，前端发出一个指令，在 warp 中的所有线程上执行；或者 MIMD 模式，前端为发散 warp 中的每个线程发出不同的指令。当 warp 未发散时，它以 SIMD 模式执行，以捕捉 warp 中线程所展现的控制流局部性，其能效与传统 SIMD 架构相当。当 warp 发散时，它切换到 MIMD 模式。warp 在此模式下的运行效率较低，但性能损失低于传统 SIMD 架构上的损失。

向量线程架构。向量线程（VT）架构 [Krashinsky 等人，2004] 结合了 SIMD 和 MIMD 架构的各个方面，旨在兼取两者之长。VT 架构具有一组通道，连接到一个公共的 L1 指令缓存。在 SIMD 模式下，所有通道直接从 L1 指令缓存接收指令以进行锁步执行，但每个通道可以切换到 MIMD 模型，以自己的节奏运行，从其 L0 缓存获取指令。Lee 等人 [2011] 最近与传统 SIMT 架构（如 GPU）的比较表明，VT 架构在规则并行应用上具有相当的效率，而在不规则并行应用上效率要高得多。

时间 SIMT。时间 SIMT[Keckler 等人，2011，Krashinsky，2011] 允许每个通道以 MIMD 方式执行，类似于 VT 架构。然而，它不是在所有通道上以锁步方式运行一个 warp，而是通过单个通道对 warp 的执行进行时间多路复用，每个通道运行一组单独的 warp。时间 SIMT 通过为整个 warp 只取一次指令来实现 SIMD 硬件的效率。这在时间上分摊了控制流开销，而传统 SIMD 架构在空间上跨多个通道分摊相同的开销。

可变 Warp 大小架构。可变 Warp 大小（VWS）架构 [Rogers 等人，2015] 包含多个（例如 8 个）切片，每个切片包含一个取指和解码单元，因此每个切片可以同时执行不同的指令，类似于 VT 和时间 SIMT。VWS 不是通过窄数据路径对大型 warp 进行时间多路复用，而是每个切片由窄（4 宽）warp 组成。这些窄 warp 然后被分组为称为帮（gang）的更大执行实体。每个帮包含来自每个切片的一个 warp。

在没有分支发散的应用中，帮中的 warp 以锁步方式执行，从共享取指单元和共享 L1 指令缓存获取指令。当遇到分支发散（或内存发散）时，帮分裂成多个帮。新的帮可以进一步分裂，直到每个 warp 都在自己的帮中。此时，这些单 warp 帮通过切片的取指单元和私有 L0 指令缓存，在各自的切片上单独执行。这些分裂的帮通过比较各个帮的 PC 的硬件，机会性地合并回原始帮。如果它们全部匹配，则重建原始帮。Rogers 等人 [2015] 还提出在第一个发散分支的立即后支配点插入帮级同步屏障。

本书还评估了每个切片中 L0 指令缓存容量对性能的影响，以及与共享 L1 指令缓存带宽的关系。在非帮模式下，切片中的 L0 缓存可能同时向 L1 缓存请求指令，造成带宽瓶颈。他们的评估表明，即使对于发散应用，256 字节的 L0 缓存也可以过滤掉对

共享 L1 缓存的大部分请求。因此，L1 缓存仅需基线 SIMT 架构的 $2\times$ 带宽即可弥补大部分带宽不足。

同时分支交织。Brunie 等人 [2012] 在线程块压缩发表后提出了同时分支和 warp 交织（SBI 和 SWI）。他们扩展了 GPU SIMT 前端，以支持每周期发出两条不同的指令。SBI 在遇到分支发散时，从同一 warp 共同发出指令。同时执行发散分支的两个目标可以显著消除其性能损失。

3.4.4 低复杂度的高效分支发散管理

基线 SIMT 栈对每个 warp 的面积需求仅为 32×64 位（在 AMD GCN 中采用的优化下可低至 6×64 位）。虽然与 SIMT 核心中的寄存器文件相比，这一面积很小，但该面积直接随 GPU 中在飞 warp 的数量以及每个 warp 的线程数成比例增长。此外，在分支发散罕见的典型 GPU 应用中，SIMT 栈占用的面积本可用于以其他方式提升应用吞吐量。许多提案用替代机制取代 SIMT 栈，这些机制共享的资源在 warp 未遇到任何分支发散时可用于其他用途。

标量寄存器文件中的 SIMT 栈。AMD GCN [AMD，2012] 具有一个由 warp 中所有线程共享的标量寄存器文件。其寄存器可用作谓词寄存器，以控制 warp 中每个线程的活动。当编译器检测到计算内核中可能存在发散分支时，会使用该标量寄存器文件在软件中模拟 SIMT 栈。GCN 架构具有特殊指令来加速 SIMT 栈的模拟。

一种最小化支持最坏情况发散所需标量寄存器数量的优化是优先执行活动线程数较少的目标。这使得最坏情况发散可由 $\log_2(\#threads\ per\ warp)$ 个标量寄存器支持，远少于基线 SIMT 栈所需的条目数。此外，当计算内核没有潜在发散分支时，编译器可将为 SIMT 栈保留的标量寄存器用于其他标量计算。

线程前沿。如第 3.4.2 节所述，Diamos 等人 [2011] 用线程前沿取代 SIMT 栈。使用线程前沿时，每个线程在寄存器文件中维护自己的 PC，并且代码经过拓扑排序，使得分支的重汇聚点始终具有更高的 PC。当 warp 发散时，它总是优先执行所有线程中 PC 最低的线程。这组线程被称为 warp 的线程前沿。优先执行前沿中的线程会隐式地强制程序中更靠前的所有线程在分支的重汇聚点等待合并。

由于仅在计算内核包含潜在发散分支时才需要每线程 PC，编译器只需在这些计算内核中分配 PC 寄存器。在其他计算内核中，额外的寄存器存储可用于提高 warp 占用率，增加每个 SIMT 核心可维持的 warp 数量，以更好地容忍内存延迟。

无栈 SIMT。Asanovic 等人 [2013] 提出用 syncwarp 指令扩展时序 SIMT 架构。在该提案中，warp 中的线程在计算内核的收敛区域执行时以锁步方式运行，编译器保证 warp 在这些区域永远不会发散。在发散分支处，warp 中的每个线程利用时序 SIMT 架构中的 MIMD 能力，以其私有 PC 遵循自己的控制流路径。

编译器在发散分支的重汇聚点放置一条 syncwarp 指令。这强制发散 warp 中的所有线程在进入计算内核的另一个收敛区域之前，在重汇聚点同步。

虽然该机制无法捕获嵌套发散分支可能的重汇聚，但它仍是一种更廉价的替代方案，对于很少出现分支发散的 GPU 应用，可提供与基线 SIMT 栈相当的性能。该论文引入了一种结合收敛与变体分析的方法，使编译器能够确定任意计算内核中符合标量化 and/or 仿射变换条件的操作。在无栈 SIMT 的背景下，同样的分析使编译器能够确定任意计算内核中的收敛和发散区域。

1. 编译器首先假设所有基本块都是线程不变的。

2. 它将所有依赖于线程 ID 的指令、原子指令以及对易失性内存的内存指令标记为线程变体。

3. 然后迭代地将所有依赖于线程变体指令的指令也标记为线程变体。

4. 基本块中控制依赖于线程变体分支指令的所有指令也是线程变体的。本质上，线程变体分支的直接后支配点之后的指令可以保持线程不变，只要它们不因其他条件被标记为线程变体。

该分析使编译器能够检测每个 warp 中所有线程一致执行的分支。由于这些分支不会导致 warp 发散，编译器无需为这些分支插入检测动态发散的代码，也无需在其直接后支配点插入 syncwarp 指令以强制重汇聚。

谓词化。在将完整的 SIMT 栈纳入架构之前，具有可编程着色器的 GPU 一直通过谓词化支持着色器程序中的有限控制流结构，就像传统向量处理器一样。谓词化在现代 GPU 中仍然作为一种低开销方式处理简单的 if 分支，避免 SIMT 栈的压栈和出栈开销。在 NVIDIA 的实现中，每条指令扩展了一个额外操作数字段以指定其谓词寄存器。谓词寄存器本质上是专用于控制流的标量寄存器。

Lee 等人 [2014b] 提出了一种线程感知的谓词化算法，将谓词化的应用扩展到任意控制流，性能与 NVIDIA 的 SIMT 栈相当。线程感知的谓词化算法在每个分支处用谓词节点扩展标准控制流依赖图（CDG）。然后可根据每个基本块的控制流依赖计算其所需的谓词化，并在不破坏功能行为的前提下进一步严格优化。论文随后描述了两种

基于这种线程感知 CDG 的优化，以及他们先前工作 [Asanovic et al., 2013] 中的收敛性和方差分析。

- 静态分支一致性优化在编译器能够保证分支可以在整个 warp 中一致地执行时应用，这由收敛性分析推断得出。在这种情况下，编译器可以将谓词生成替换为统一分支指令。
- 运行时分支一致性优化在其他情况下应用。编译器发出共识分支（cbranch.ifnone），这些分支仅在给定空谓词（即所有线程禁用）时执行。这允许 warp 跳过具有空谓词的代码——这是 SIMT 栈提供的关键优势。这种方法与先前针对向量处理器的努力（如 BOSCC）不同，因为它依赖于结构分析来确定此优化的候选者。虽然谓词和 SIMT 栈在根本上以相似的能耗和面积成本提供相同的功能，但 Lee 等人 [2014b] 强调了两种方法之间的以下权衡。
- 由于不同的分支目标由不同的谓词寄存器保护，编译器可以调度来自不同分支目标的指令，交错执行不同的分支目标以利用线程级并行性（TLP），否则这需要更高级的

硬件分支发散管理。• 谓词倾向于增加寄存器压力，进而降低 warp 占用率并带来整体性能损失。这是因为保守的寄存器分配无法为分支的两侧重用寄存器。它无法稳健地证明来自不同分支目标的指令在寄存器中操作的是非独占的通道集。两种提出的优化插入的统一和共识分支指令缓解了这个问题。• 谓词可能以多种方式影响动态指令计数。在某些情况下，检查统一分支的开销显著增加了动态指令计数。或者，不执行检查意味着某些路径在空谓词掩码下执行。在其他情况下，它移除了维护 SIMT 栈所需的 push/pop 指令。最后，论文提出了新指令以减少谓词的开销。

• 对于函数调用和间接分支，他们提出了一种新的 find_unique 指令，通过循环串行执行每个分支目标/函数。• cbranch.ifany（除了现有的共识分支指令 cbranch.ifnone 和 cbranch.ifall 之外）将有助于减少动态统一分支检测引入的指令计数开销。

3.5 标量化与仿射执行的研究方向

如第 2 章所述，GPU 计算 API（如 CUDA 和 OpenCL）具有类似 MIMD 的编程模型，允许程序员在 GPU 上启动大量标量线程。虽然这些标量线程中的每一个都可以遵循其独特的执行路径并可能访问任意内存位置，但在常见情况下，它们都遵循一小部分执行路径并执行类似的操作。GPU 线程之间的收敛控制流在大多数（如果不是全部）现代 GPU 上通过 SIMT 执行模型得到利用，其中标量线程被分组为在 SIMD 硬件上运行的 warp（见第 3.1.1 节）。

本节总结了一系列通过标量化和仿射执行进一步利用这些标量线程相似性的研究。这些研究的关键见解在于观察执行相同计算内核的线程之间的值结构 [Kim et al., 2013]。两种类型的值结构，统一和仿射，在示例 3.3 的计算内核中进行了说明。

统一变量在计算内核中，对于每个线程具有相同常量值的变量。在算法 3.3 中，变量 a 以及字面量 THRESHOLD 和 Y_MAX_VALUE 在计算内核中的所有线程中都具有统一值。统一变量可以存储在单个标量寄存器中，并由计算内核中的所有线程重用。

仿射变量在计算内核中，对于每个线程，其值是线程 ID 的线性函数的变量。在算法 3.3 中，变量 y[idx] 的内存地址可以表示为线程 ID threadIdx.x 的仿射变换：

$$\text{addr}(y[idx]) = \text{addr}(y[0]) + \text{sizeof}(int) \times threadIdx.x$$

这种仿射表示可以存储为一对标量值，即基址和步长，这比完全展开的向量紧凑得多。

关于如何在 GPU 中检测和利用统一或仿射变量，有多种研究提案。本节的其余部分从这两个方面总结了这些提案。

3.5.1　统一或仿射变量的检测

在 GPU 计算内核中检测统一或仿射变量的存在有两种主要方法：编译器驱动的检测和通过硬件的检测。

编译器驱动的检测　检测 GPU 计算内核中统一或仿射变量存在的一种方法是通过特殊的编译器分析来实现。这是可能的，因为现有的 GPU 编程模型 CUDA 和 OpenCL 已经提供了让程序员声明变量为

算法 3.3 计算内核中标量和仿射操作的示例（来自 [Kim et al., 2013]）。

```
__global__ void vsadd( int y[], int a )
{
    int idx = threadIdx.x;
    y[idx] = y[idx] + a;
    if ( y[idx] > THRESHOLD )
    y[idx] = Y_MAX_VALUE;
}
```

在整个计算内核中为常量，以及为线程 ID 提供特殊变量。编译器可以执行控制依赖性分析，以检测完全依赖于常量和线程 ID 的变量，并将它们标记为统一/仿射。仅对统一/仿射变量进行操作的操作然后成为标量化的候选者。

AMD GCN [AMD，2012] 依赖编译器来检测可以由专用标量处理器存储和处理的统一变量和标量操作。

Asanovic 等人 [2013] 提出了一种结合收敛与变体分析的方案，使编译器能够确定任意计算内核中符合标量化条件和/或仿射变换条件的操作。计算内核收敛区域内的指令可以转换为标量/仿射指令。在计算内核从发散区域到收敛区域的任何转换处，编译器会插入一条 syncwarp 指令，以处理两个区域之间由控制流引起的寄存器依赖关系。Asanovic 等人 [2013] 采用这种分析为 Temporal-SIMT 架构 [Keckler 等人，2011，Krashinsky，2011] 生成标量操作。

解耦仿射计算（Decoupled Affine Computation，DAC）[Wang and Lin，2017] 依赖类似的编译器分析来提取可解耦到单独 warp 的标量和仿射候选。Wang 和 Lin[2017] 通过发散仿射分析增强了该过程，目标是从计算内核起始处提取一直是仿射的指令串。这些仿射指令串从主内核解耦到一个仿射内核中，该仿射内核通过硬件队列向主内核提供数据。

硬件检测　与编译器驱动的检测相比，在硬件中检测均匀/仿射变量具有两个潜在优势。

1. 这使得标量化和仿射执行能够应用于原始 GPU 指令集架构。它省去了与硬件协同开发专用标量化编译器的工作。

2. 硬件检测发生在计算内核执行期间。因此，它能够检测动态出现的均匀/仿射变量，而这些变量会被静态分析遗漏。

基于标签的检测。Collange 等人 [2010] 引入了一种基于标签的检测系统。在该系统中，每个 GPU 寄存器都扩展了一个标签，指示该寄存器包含均匀、仿射还是通用向量值。在计算内核启动时，包含线程 ID 的寄存器的标签被设置为仿射状态。从常量内存或共享内存的单一位置广播值的指令将目标寄存器的标签设置为均匀状态。在内核执行期间，寄存器的状态根据表 3.1 中的简单规则跨算术指令从源操作数传播到目标操作数。虽然这种基于标签的检测硬件开销很小，但它往往偏于保守——例如，它保守地将均匀变量和仿射变量之间乘法的输出判定为向量变量。

architecture [Kim et al., 2013] extends the tag-based det
t al. [2010] with better support for branches. Affine branche
en affine operands, is resolved via the scalar datapath if one
et al. [2013] also introduce a *lazy expansion* scheme, where
nto full vector registers after a divergent branch or a predicate

表 3.1 均匀状态（U）、仿射状态（A）和向量状态（V）在基本运算中的传播规则（据 Collange 等，2010）。

FG-SIMT 架构 [Kim 等人，2013] 扩展了 Collange 等人 [2010] 的基于标签的检测机制，增强了对分支的支持。仿射分支，即比较仿射操作数之间的分支，如果其中一个操作数是均匀的，则通过标量数据通路解决。Kim 等人 [2013] 还引入了一种惰性扩展方案，其中仿射寄存器在发散分支或谓词指令之后被惰性扩展为完整向量寄存器。这种扩展是必需的，以允许发散 warp 中的一部分线程更新其在目标寄存器中的槽位，同时保持其他槽位不变——这维持了 SIMT 执行语义。与更简单的、急切的扩展方案（在第一个发散分支后扩展每个仿射寄存器）相比，惰性扩展方案消除了许多不必要的扩展。

写回时比较。Gilani 等人 [2013] 引入了一种更激进的机制，通过在向量指令的每次写回时比较 warp 中所有线程的寄存器值来检测均匀变量。在检测到均匀变量时，检测逻辑将写回重定向到标量寄存器文件，并更新内部表以记住该寄存器的状态。随后对该寄存器的使用被重定向到标量寄存器文件。所有操作数来自标量寄存器文件的指令在单独的标量流水线上执行。

Lee 等人 [2015] 使用了类似的检测方案。他们不是简单的均匀检测器，而是在寄存器写回阶段增加了一个寄存器值压缩器，该压缩器使用 Pekhimenko 等人 [2012] 引入的算法将传入的值向量转换为基-增量-立即数（BDI）元组。

Wong 等人 [2016] 引入了 Warp 近似（Warp Approximation），这是一个在 warp 内利用近似计算的框架，其特点也是在寄存器写回时进行检测。检测器计算写回寄存器文件的向量中所有值之间的最小 d 相似度（两个给定值共享 d 个最高有效位）。d 相似度高于阈值的寄存器被标记为相似，然后用于确定后续依赖指令中近似执行的资格。

与 Lee 等人 [2015] 的提案类似，G-Scalar[Liu 等人，2017] 在寄存器写回阶段具有寄存器值压缩器，但压缩器采用更简单的算法，仅提取 warp 中所有通道所有值共用的

公共字节。如果所有字节都是公共的，则该寄存器包含均匀变量。任何仅对均匀变量操作的指令都可以被标量化。

G-Scalar 还扩展了寄存器值压缩器，以检测在分支发散下符合标量执行条件的操作。所有先前的提案在 warp 一发散时就退回到向量执行。Liu 等人 [2017] 观察到，在分支发散下的许多指令中，活动通道的操作数值是均匀的。使用这些部分均匀寄存器的指令实际上符合标量执行条件。然后，他们扩展了寄存器值压缩器，使用特殊逻辑仅检查来自活动通道的值。这大大增加了各种 GPU 计算工作负载中标量指令的数量。注意，在发散情况下，写入的寄存器不会被压缩。

3.5.2　利用 GPU 中的均匀或仿射变量

GPU 的设计可以通过多种方式利用计算内核中值结构的存在。

压缩寄存器存储　均匀和仿射变量的紧凑表示允许它们以少得多的位存储在寄存器文件中。回收的存储空间可用于维持更多在途 warp，从而在相同寄存器文件资源下提高 GPU 对内存延迟的容忍度。

标量寄存器文件。许多提案/设计利用 GPU 中的均匀或仿射变量，为标量/仿射值配备了专用寄存器文件。

- AMD GCN 架构具有一个标量寄存器文件, 可由标量流水线和向量流水线共同访问。
- FG-SIMT 架构 [Kim 等人，2013] 将均匀/仿射值存储在单独的仿射 SIMT 寄存器文件（ASRF）中。ASRF 记录每个寄存器的状态（仿射/均匀/向量），允许控制逻辑检测可在控制处理器上直接执行的符合条件的操作。
- Gilani 等人 [2013] 的动态均匀检测提案将动态检测到的均匀值存储到专用标量寄存器文件中。部分寄存器文件访问。Lee 等人 [2015] 对写回寄存器文件的寄存器应用基址、增量、立即数（BDI）压缩。压缩后的寄存器在作为源操作数读回时被解压缩回正常向量。在此方案中，每个压缩寄存器仍占用与未压缩寄存器相同的存储槽，但仅占用寄存器 bank 的一个子集，因此读取寄存器的压缩表示消耗更少的能量。

Warp 近似架构 [Wong 等人，2016] 通过仅访问与通过相似性检测选出的代表线程对应的 lane，减少了寄存器读/写能耗。

类似地，G-Scalar[Liu 等人，2017] 具有压缩寄存器，这些寄存器仅占用为未压缩寄存器分配的 bank 的一个子集，以减少寄存器读取的能量。

专用仿射 Warp。解耦仿射计算（DAC）[Wang 和 Lin，2017] 将所有编译器提取的仿射变量缓冲在专用仿射 warp 的寄存器中。该仿射 warp 与其他非仿射 warp 一样共享相同的向量寄存器文件存储，但仿射 warp 使用每个单独寄存器条目的各个 lane 来存储基址以及不同非仿射 warp 的增量。

标量化操作 除了高效存储之外，具有均匀或仿射变量的操作可以被标量化。不是通过 SIMD 数据通路在 warp 中的所有线程上重复相同的操作，标量操作可以在单个标量数据通路中完成一次，在此过程中消耗的能量少得多。通常，如果算术操作的输入操作数仅由均匀或仿射变量组成，则该操作可以被标量化。

专用标量流水线。AMD 的 GCN 架构具有一个专用标量流水线，执行编译器生成的标量指令。FG-SIMT 架构 [Kim 等人，2013] 具有一个控制处理器，能够直接执行动态检测到的仿射操作，而无需调用 SIMD 数据通路。

在两种实现中，标量流水线还处理 SIMD 流水线的控制流和谓词化。这种解耦意味着许多系统相关功能（例如，与主机处理器的通信）也可以卸载到标量流水线，从而将 SIMD 数据通路从实现完整指令集的负担中解放出来。

时钟门控 SIMD 数据通路。Warp 近似架构 [Wong 等人，2016] 和 G-Scalar[Liu 等人，2017] 都在 SIMD 数据通路的一个

lane 上执行动态检测到的标量指令。当这种情况发生时，其他 lane 被时钟门控以减少动态功耗。

这种方法消除了在专用标量数据通路上支持完整指令集的重复工作，或者必须筛选出要在标量数据通路上实现的子集。例如，G-Scalar[Liu 等人，2017] 可以以相对较低的开销标量化由特殊功能单元支持的指令。

聚合到仿射 Warp。解耦仿射计算（DAC）[Wang 和 Lin，2017] 将来自多个 warp 的仿射操作聚合到每个 SIMT 核心的单个仿射 warp 中。该仿射 warp 与其他 warp 一样在 SIMD 数据通路上执行，但执行的每条指令同时操作多个 warp 的仿射表示。

内存访问加速 当均匀或仿射变量用于表示内存操作（加载/存储）的地址时，内存操作触及的内存位置是高度可预测的——每个连续位置之间相隔已知的步长。这允许进行各种优化。例如，具有已知步长的内存位置的内存合并比任意随机位置的合并简单得多。仿射变量还可以用于表示单条指令的大块传输，而不是通过加载/存储指令的循环。

FG-SIMT 架构 [Kim 等人，2013] 在控制处理器中具有特殊的地址生成单元，用于将具有仿射地址的内存访问扩展为实际地址。由于仿射地址在线程之间具有固定步长，将这些仿射内存访问合并到缓存行可以用更简单的硬件完成。

解耦仿射计算（DAC）[Wang 和 Lin，2017] 也具有类似的优化，以利用仿射内存访问中的固定步长。此外，它使用仿射 warp 在其他非仿射 warp 之前执行，为这些 warp 预取数据。预取的数据存储在 L1 缓存中，稍后由相应的非仿射 warp 通过特殊的出队指令取回。

3.6　寄存器文件架构的研究方向

现代 GPU 采用大量硬件线程（warp），将其执行复用到少得多（但仍然很大）的 ALU 数量上，以容忍流水线和内存访问延迟。为了允许 warp 之间快速高效地切换，GPU 使用硬件 warp 调度器并将所有硬件线程的寄存器存储在片上寄存器文件中。在许多 GPU 架构上，这些寄存器文件的容量相当大，有时甚至超过最后一级缓存的容量，这是由于 GPU 中使用的宽 SIMD 数据通路，以及容忍数百个周期的内存访问延迟所需的 warp 数量。例如，NVIDIA 的 Fermi GPU 可以维持超过 20，000 个在途线程，并且具有 2 MB 的总寄存器容量。

为了最小化寄存器文件存储所占用的面积，GPU 上的寄存器文件通常通过低端口数的 SRAM bank 来实现。这些 SRAM bank 被并行访问，以提供维持宽 SIMD 流水线上指令以峰值吞吐量运行所需操作数带宽。如本章前面所述，一些 GPU 使用操作数收集器来协调来自多条指令的操作数访问，以最小化 bank 冲突惩罚。

访问这些大型寄存器文件在每次访问时消耗大量动态能量，并且其大尺寸也导致高静态功耗。在 NVIDIA GTX280 GPU 上，近 10% 的总 GPU 功耗由寄存器文件消耗。这提供了明确的动机来创新 GPU 寄存器文件架构以减少其能耗。因此，近年来在这个主题上有大量的研究论文。本节的剩余部分总结了几个旨在实现这一目标的研究提案。

3.6.1　分层寄存器文件

Gebhart 等人 [2011b] 观察到，在一组真实世界的图形和计算工作负载中，指令产生的值中高达 70% 只被读取一次，并且只有 10% 被读取超过两次。为了捕捉大多数寄存器值的这种短生命周期，他们提出用寄存器文件缓存（RFC）扩展 GPU 上的主寄存器文件。这形成了寄存器文件的分层结构，并显著降低了对主寄存器文件的访问频率。

在这项工作中，RFC 通过 FIFO 替换策略为每条指令的目的操作数分配一个新条目。未命中 RFC 的源操作数不会被加载到 RFC 上，以减少对已经很小的 RFC 的污染。默认情况下，从 RFC 逐出的每个值都会写回主寄存器文件。然而，由于其中许多值永远不会再被读取，Gebhart 等人 [2011b] 用编译时生成的静态活跃性信息扩展了仅硬件的 RFC。在指令编码中添加一个额外的位，以指示消费寄存器值的最后一条指令。最后一次被读取的寄存器在 RFC 中被标记为死亡。在逐出时，它不会被写回主寄存器文件。

为了进一步减小 RFC 的大小，Gebhart 等人 [2011b] 将其与两级 warp 调度器结合。这个两级 warp 调度器将执行限制在一个活动 warp 池中，该池仅由每个 SIMT 核心中 warp 的一小部分组成。这项工作考虑每个 SIMT 核心总共 32 个 warp 中的 4-8 个 warp 的活动 warp 池。RFC 仅保存来自活动 warp 的值，因此更小。在长延迟操作（如全局内存加载或纹理获取）时，warp 从活动池中移除。当这种情况发生时，warp 的 RFC 条目被刷新，为第二级调度器激活的不同 warp 释放空间。

编译时管理的寄存器文件层次结构。Gebhart 等人 [2011a] 进一步用最后结果文件

（LRF）扩展了这个寄存器文件层次结构，该文件仅缓冲每个活动 warp 的最后一条指令产生的寄存器值。这项工作还用编译时管理的操作数寄存器文件（ORF）替换了硬件管理的 RFC。值进出 ORF 的移动由编译器显式管理。这消除了 RFC 所需的标签查找。编译器在大多数 GPU 工作负载中对寄存器使用模式有更全面的视图，使其能够做出更优的决策。这项工作还扩展了两级 warp 调度器，以便编译器指示何时可以将 warp 从活动池中切换出去。这是必需的，以协调 ORF 的内容与 warp 的活跃性，在 warp 被切换出去之前将所有活跃数据从 ORF 移回主寄存器文件。

3.6.2 休眠状态寄存器文件

Abdel-Majeed 和 Annavaram[2013] 提出了一种三模态寄存器文件设计，以减少大型 GPU 寄存器文件的泄漏功耗。三模态寄存器文件中的每个条目可以在 ON、OFF 和 Drowsy 模式之间切换。ON 模式是正常操作模式; OFF 模式不保留寄存器的值; Drowsy 模式保留寄存器的值，但条目在访问前需要唤醒到 ON 模式。在这项工作中，所有未分配的寄存器处于 OFF 模式，所有已分配的寄存器在每次访问后立即进入休眠状态。该策略利用了 GPU 上由于细粒度多线程而导致的同一寄存器连续访问之间的长延迟，允许寄存器文件中的寄存器大部分时间处于休眠模式。GPU 中的长流水线也意味着从休眠状态唤醒寄存器的额外延迟不会引入显著的性能损失。

3.6.3 寄存器文件虚拟化

Tarjan 和 Skadron[2011] 观察到，在等待内存操作时，GPU 线程中的活跃寄存器数量往往很小。对于某些 GPU 应用，他们声称高达 60% 的寄存器未被使用。他们提出通过使用寄存器重命名来虚拟化物理寄存器，将物理寄存器文件的大小减少多达 50%，或将并发执行线程的数量加倍。在所提出的机制中，线程开始执行时没有分配寄存器，并且随着指令被解码，物理寄存器被分配给目的寄存器。Tarjan 和 Skadron 进一步建议，可以通过采用编译器分析来确定寄存器的最后一次读取来增强物理寄存器的释放。他们提出了“最终读取注释”，并建议添加“每个操作数一个位以指示是否是最后一次读取”，并指出这可能需要指令编码中的额外位。

Jeon 等人 [2015] 量化了通过将寄存器溢出到内存来减少 GPU 寄存器文件大小的影响。他们发现，通过采用溢出将寄存器文件大小减少 50%，平均执行时间增加了 73%。他们回顾了早期在采用乱序执行的 CPU 上使用寄存器重命名时提前回收物理寄存器的提案。

执行。他们提出通过添加“元数据指令”来高效编码物理寄存器何时可以被回收，并使用寄存器生命周期活跃性分析生成这些指令，从而解决添加“最终读注解”所需的额外位的问题。他们的一项重要观察是，在确定何处可以安全回收物理寄存器时，必须考虑分支发散（Kloosterman 等人 [2017] 进一步阐述了这一点）。对于 128 KB 的寄存器文件，Jeon 等人的重命名技术的直接实现需要 3.8 KB 的重命名硬件。他们表明，

通过不对生命周期长的寄存器进行重命名，这一开销可以降低到 1 KB。为了利用这一机会，他们提出仅对逻辑寄存器号大于编译器确定阈值的寄存器使用重命名。Jeon 等人进一步提出使用重命名来实现寄存器文件子阵列的功率门控。他们评估了其通过寄存器重命名支持寄存器文件虚拟化的详细提案的有效性，表明确实可以在不损失性能的情况下将寄存器文件大小减少 50%。

3.6.4　分区寄存器文件

Abdel-Majeed 等人 [2017] 引入了试点寄存器文件（Pilot Register File），它将 GPU 寄存器文件划分为快速寄存器文件（FRF）和慢速寄存器文件（SRF）。FRF 使用常规 SRAM 实现，而 SRF 使用近阈值电压（NTV）SRAM 实现。与常规 SRAM 相比，NTV SRAM 具有低得多的访问能量以及低得多的泄漏功耗。作为交换，对 NTV SRAM 的访问延迟要慢得多，通常需要几个周期（而不是常规 SRAM 的一个周期）。在这项工作中，SRF 显著大于 FRF。每个 warp 在 FRF 中有 4 个条目。关键是用 FRF 来服务大多数访问，以补偿 SRF 的缓慢。访问 SRF 的额外延迟由操作数收集器处理。FRF 通过 FinFET 背栅控制进一步增强了低功耗模式。这允许不活跃 warp 的 FRF 切换到低功耗模式。这使得 FRF 能够获得两级调度器的好处，而无需显式地将 warp 调度进或调度出活跃池。

这项工作与分层寄存器文件的不同之处在于, 不同分区持有独占的寄存器集合, 并且分区在 warp 的整个生命周期内保持不变。Abdel-Majeed 等人 [2017] 没有使用编译器来确定要放入 FRF 的寄存器集合，而是在每次内核启动时采用一个试点 CTA 来分析最常用的寄存器。这组高使用率的寄存器被记录在一个查找表中，该表可供内核启动后的每个后续 warp 访问。

3.6.5　RegLess

Kloosterman 等人 [2017] 引入了 RegLess，旨在消除寄存器文件并用操作数暂存缓冲区取而代之。该论文观察到，在相对较短的时间跨度内，访问的寄存器数量只占寄存器文件总容量的一小部分。例如，在 100 个周期的时段内，他们评估的许多应用在使用 GTO 或两级 warp 调度器时访问的寄存器不到

2048 KB 寄存器文件的 10%。为了利用这一观察，RegLess 使用编译器算法将内核执行划分为区域。区域是单个基本块内的连续指令。区域之间的边界选择为限制活跃寄存器的数量。使用区域注解，容量管理器（CM）确定哪些 warp 有资格被调度。当 warp 开始执行新区域的指令时，该区域中使用的寄存器从全局内存中分配的后备存储区（可能缓存在 L1 数据缓存中）被带入操作数暂存单元（OSU）。OSU 本质上是一个由八个 bank 组成的缓存，提供足够的带宽每周期服务两条指令。为了避免在访问 OSU 中的数据时停顿，CM 在发出区域中的第一条指令之前预加载寄存器。为了管理预加载过程，CM 为每个 warp 维护一个状态机，指示下一个区域所需的寄存器是否存在于

OSU 中。为了减少 OSU 与内存层次结构之间产生的内存流量，RegLess 采用了利用仿射值的寄存器压缩技术（见第 3.5 节）。

Kloosterman 等人对其提案进行了详细评估，包括 Verilog 综合以及提取 RegLess 引入的硬件单元的寄生电容和电阻值。他们的评估表明，512 条目的 OSU 与 2048 KB 寄存器文件相比可以实现略好的性能，同时仅占用 25% 的空间，并将整体 GPU 能耗降低 11%。

第四章　内存系统

本章探讨 GPU 的内存系统。GPU 计算内核通过加载和存储指令与内存系统交互。传统图形应用与多个内存空间交互，如纹理、常量和渲染表面。虽然 GPGPU 编程 API（如 CUDA）中可以访问这些内存空间，但本章我们将重点介绍 GPGPU 编程中使用的内存空间，特别是实现它们所需的微架构支持。

CPU 通常包括两个独立的内存空间：寄存器文件和内存。现代 GPU 在逻辑上将内存进一步细分为本地内存空间和全局内存空间。本地内存空间是每个线程私有的，通常用于寄存器溢出，而全局内存用于多个线程之间共享的数据结构。此外，现代 GPU 通常实现程序员管理的暂存器内存，在协作线程数组中一起执行的线程之间共享访问。包含共享地址空间的一个动机是，在许多应用中，程序员知道在计算的给定步骤需要访问哪些数据。通过一次性将所有数据加载到共享内存中，他们可以重叠长延迟的片外内存访问，并在对这些数据进行计算时避免长延迟的内存访问。更重要的是，在给定时间内 GPU 与片外内存之间可以传输的字节数（DRAM 带宽）相对于同一时间内可以执行的指令数来说很小。此外，在片外内存和 GPU 之间传输数据所消耗的能量比从片上内存访问数据所消耗的能量高几个数量级。因此，从片上内存访问数据可以获得更高的性能并节省能量。

我们将对内存系统的讨论分为两部分，以反映内存划分为驻留在 GPU 核心内部的部分和连接到片外 DRAM 芯片的内存分区中的部分。

4.1　第一级内存结构

本节描述 GPU 上的第一级缓存结构，重点介绍统一的 L1 数据缓存和暂存器“共享内存”，以及它们如何与核心流水线交互。我们还简要讨论 L1 纹理缓存的典型微架构。我们之所以讨论纹理缓存——它在 GPU 计算应用中的使用有限——是因为它提供了一些关于 GPU 与 CPU 差异的见解和直觉。最近的一项专利描述了如何统一纹理缓存和 L1 数据缓存（例如，在 NVIDIA 的第 4 章

Maxwell 和 Pascal GPU 中）[Heinrich et al., 2017]。我们将此设计的讨论推迟到先考虑纹理缓存的组织方式之后。GPU 中第一级内存结构的一个有趣方面是它们在遇到冒险时如何与核心流水线交互。如第 3 章所述，流水线冒险可以通过重放指令来处理。本章扩展了之前关于重放的讨论，重点关注内存系统中的冒险。

4.1.1 暂存器内存和 L1 数据缓存

在 CUDA 编程模型中，“共享内存”指的是一个相对较小的内存空间，预期具有低延迟，但可由给定 CTA 内的所有线程访问。在其他架构中，这种内存空间有时被称为暂存器内存 [Hofstee，2005]。访问此内存空间的延迟通常与寄存器文件访问延迟相当。实际上，早期的 NVIDIA 专利将 CUDA“共享内存”称为全局寄存器文件 [Acocella and Goudy，2010]。在 OpenCL 中，此内存空间被称为“本地内存”。从程序员的角度来看，使用共享内存时需要考虑的一个关键方面，除了其容量有限之外，是潜在的存储体冲突。共享内存实现为静态随机存取存储器（SRAM），并在一些专利中描述为每个通道一个存储体，每个存储体有一个读端口和一个写端口 [Minkin et al., 2012]。每个线程可以访问所有存储体。当多个线程在给定周期内访问同一个存储体，并且这些线程希望访问该存储体中的不同位置时，就会发生存储体冲突。在详细考虑共享内存的实现方式之前，我们先看看 L1 数据缓存。

L1 数据缓存在缓存中维护全局内存地址空间的一个子集。在某些架构中，L1 缓存仅包含未被内核修改的位置，这有助于避免因 GPU 缺乏缓存一致性而带来的复杂性。从程序员的角度来看，访问全局内存时的一个关键考虑因素是给定 warp 内不同线程访问的内存位置之间的关系。如果 warp 中的所有线程访问的位置都落在单个 L1 数据缓存块内，并且该块不在缓存中，那么只需要向更低级别的缓存发送一个请求。这种访问被称为“合并的”。如果 warp 内的线程访问不同的缓存块，则需要生成多个内存访问。这种访问被称为非合并的。程序员试图避免存储体冲突和非合并访问，但为了简化编程，硬件允许两者存在。

图 4.1 展示了 Minkin 等人 [2012] 描述的 GPU 缓存组织。图中所示的设计实现了统一的共享内存和 L1 数据缓存，这是 NVIDIA Fermi 架构引入的功能，也存在于 Kepler 架构中。图中央是一个 SRAM 数据阵列⑤，可以配置 [Minkin et al., 2013] 为部分用于共享内存的直接映射访问，部分作为组相联缓存。该设计通过在处理存储体冲突和 L1 数据缓存未命中时使用重放机制，支持与指令流水线的非停顿接口。为了帮助解释这种缓存架构的操作，我们首先考虑共享内存访问的处理方式，然后考虑合并的缓存命中，最后考虑缓存未命中和非合并访问。对于所有情况，内存访问请求首先从指令流水线内的加载/存储单元发送到 L1 缓存①。内存访问请求由一组内存地址组成，每个线程在 warp 中对应一个地址，以及操作类型。

共享内存访问操作 对于共享内存访问，仲裁器确定 warp 内请求的地址是否会导致存储体冲突。如果请求的地址会导致一个或多个存储体冲突，仲裁器将请求分成两部分。第一部分包括 warp 中一部分线程的地址，这些地址没有存储体冲突。原始请求的这一部分被仲裁器接受，由缓存进一步处理。第二部分包含与第一部分中的地址发生存储体冲突的地址。原始请求的这一部分返回到指令流水线，必须再次执行。这种后续执行称为“重放”。原始共享内存请求的重放部分存储在哪里存在权衡。虽然通过从指令缓冲区重放内存访问指令可以节省面积，但访问大型寄存器文件会消耗能量。对

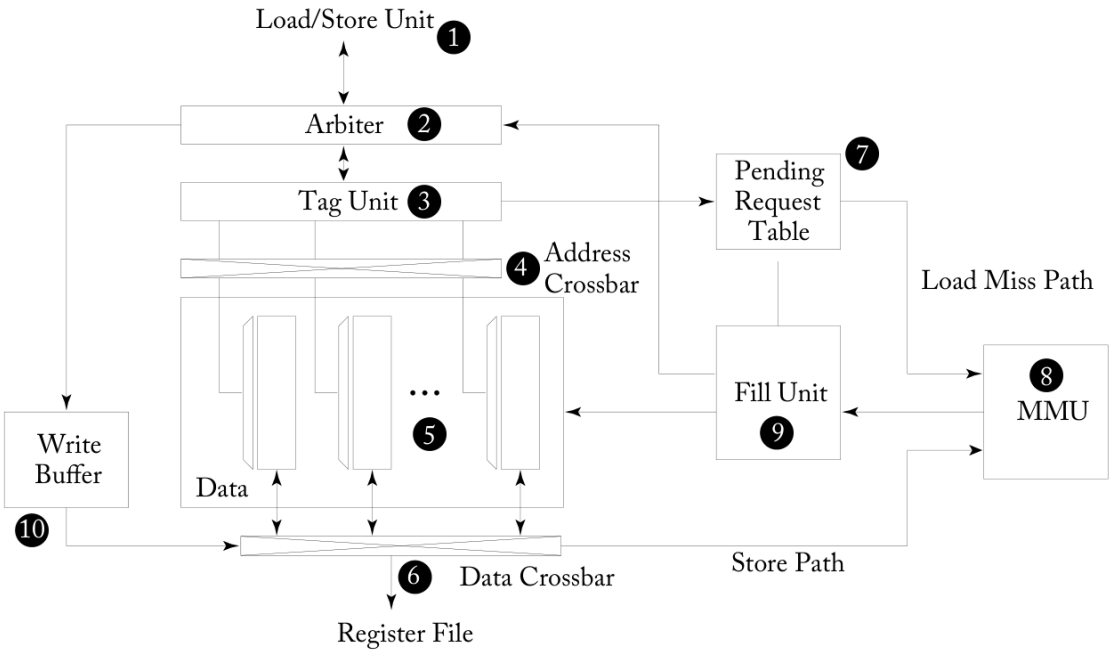


图 4.1　统一的 L1 数据缓存和共享内存 [Minkin et al., 2012]。

于能效而言，更好的替代方案可能是在加载/存储单元中提供有限的缓冲用于重放内存访问指令，并避免当该缓冲区中的空闲空间开始耗尽时从指令缓冲区调度内存访问操作。在考虑重放请求会发生什么之前，让我们先考虑已接受的内存请求部分是如何处理的。

共享内存请求中被接受的部分会绕过标签单元③内部的标签查找，因为共享内存是直接映射的。当接受一个共享内存加载请求时，仲裁器会在指令流水线中调度一个对寄存器堆的回写事件，因为在没有存储体冲突的情况下，直接映射内存查找的延迟是恒定的。标签单元确定每个线程的请求映射到哪个存储体，以便控制地址交叉开关④，该交叉开关将地址分发到数据阵列⑤内的各个存储体。数据阵列⑤内的每个存储体都是 32 位宽，并且有自己的译码器，允许独立访问每个存储体中的不同行。数据通过数据交叉开关⑥返回到相应线程的通道，以便存储在寄存器堆中。只有与 warp 中活跃线程对应的通道才会向寄存器堆写入值。

假设共享内存查找的延迟为单周期，那么共享内存请求中重放的部分可以在前一个被接受部分的下一周期访问 L1 缓存仲裁器。如果这个重放部分遇到存储体冲突，它会被进一步细分为接受部分和重放部分。

缓存读操作　接下来，让我们考虑对全局内存空间的加载是如何处理的。由于全局内存空间中只有一部分被缓存在 L1 中，标签单元需要检查数据是否存在于缓存中。虽然数据阵列高度分体化，以便各个 warp 能够灵活访问共享内存，但对全局内存的访问被限制为每周期一个缓存块。这一限制有助于减少相对于缓存数据量的标签存储开销，同时也是标准 DRAM 芯片标准接口的结果。在 Fermi 和 Kepler 中，L1 缓存块大小为 128 字节，在 Maxwell 和 Pascal 中进一步分为四个 32 字节的扇区 [Liptay，1968]

[NVIDIA Corp。]。32 字节的扇区大小对应于最近图形 DRAM 芯片（例如 GDDR5）在单次访问中可读取的最小数据大小。每个 128 字节的缓存块由 32 个存储体中同一行的 32 位条目组成。

加载/存储单元①计算内存地址并应用合并规则，将 warp 的内存访问分解为各个合并访问，然后将其送入仲裁器②。如果资源不足，仲裁器可能拒绝请求。例如，如果访问映射到的缓存组中的所有路都忙，或者待处理请求表⑦（如下所述）中没有空闲条目。假设有足够的资源来处理未命中，仲裁器请求指令流水线在对应于缓存命中的固定周期数后调度对寄存器堆的回写。同时，仲裁器还请求标签单元③检查该访问实际上导致缓存命中还是未命中。在缓存命中的情况下，数据阵列⑤的相应行在所有存储体中被访问，数据⑥返回到指令流水线中的寄存器堆。与共享内存访问的情况一样，只有与活跃线程对应的寄存器通道被更新。

当访问标签单元时，如果确定请求触发缓存未命中，仲裁器通知加载/存储单元它必须重放该请求，并同时将请求信息发送到待处理请求表（PRT）⑦。待处理请求表提供的功能与 CPU 缓存内存系统中传统未命中状态保持寄存器 [Kroft，1981] 所支持的功能类似。NVIDIA 专利中描述了至少两个版本的待处理请求表 [Minkin et al., 2012, Nyland et al., 2011]。与图 4.1 所示 L1 缓存架构相关的版本看起来有些类似于传统的 MSHR。用于数据缓存的传统 MSHR 包含缓存未命中的块地址，以及块偏移和当块填充到缓存时需要写入的相关寄存器的信息。对同一块的多个未命中通过记录多个块偏移和寄存器来支持。图 4.1 中的 PRT 支持合并对同一块的两个请求，并记录通知指令流水线需要重放哪个延迟内存访问所需的信息。

图 4.1 所示的 L1 数据缓存是虚拟索引和虚拟标记的。与现代 CPU 微架构大多采用虚拟索引/物理标记的 L1 数据缓存相比，这可能令人惊讶。CPU 使用这种组织方式是为了避免在上下文切换时刷新 L1 数据缓存的开销 [Hennessy and Patterson, 2011]。虽然 GPU 实际上在每个 warp 发射的每个周期都执行上下文切换，但 warp 是同一应用程序的一部分。基于页面的虚拟内存在 GPU 中仍然是有利的，即使它限于一次运行一个操作系统应用程序，因为它有助于简化内存分配并减少内存碎片。在 PRT 中分配条目后，内存请求被转发到内存管理单元（MMU）8 进行虚拟到物理地址转换，然后通过交叉开关互连到相应的内存分区单元。正如将在第 4.3 节中详细阐述的，内存分区单元包含一组 L2 缓存以及内存访问调度器。除了关于要访问哪个物理内存地址以及读取多少字节的信息外，内存请求还包含一个“subid”，当内存请求返回到核心时，可用于查找 PRT 中包含请求相关信息的条目。

一旦加载的内存请求响应返回到核心，它由 MMU 传递到填充单元⑨。填充单元进而使用内存请求中的 subid 字段在 PRT 中查找有关请求的信息。这包括填充单元可以通过仲裁器②传递给加载/存储单元的信息，以重新调度加载，然后通过将行放入数据阵列⑤后锁定该行，保证加载在缓存中命中。

缓存写操作　图 4.1 中的 L1 数据缓存可以同时支持写直达和写回策略。因此，对全局内存的存储指令（写操作）可以通过多种方式处理。具体的存储

写入的地址空间决定了该写入是按写直达还是写回方式处理。在许多 GPGPU 应用中, 对全局内存的访问通常具有非常差的时间局部性, 因为内核通常被编写为线程在退出前将数据写入一个大数组。对于此类访问，采用不写分配的写直达策略 [Hennessy and Patterson，2011] 可能是合理的。相比之下，用于将寄存器溢出到栈的局部内存写入可能表现出良好的时间局部性，随后的加载操作使得采用写分配的写回策略 [Hennessy and Patterson，2011] 具有合理性。

要写入共享内存或全局内存的数据首先被放入写数据缓冲区（WDB）⑩。对于非合并访问或当某些线程被屏蔽时，只写入缓存块的一部分。如果该块存在于缓存中，数据可以通过数据交叉开关⑥写入数据阵列。如果数据不在缓存中，则必须首先从 L2 缓存或 DRAM 内存中读取该块。完全填充缓存块的合并写入如果使缓存中任何陈旧数据的标签失效，则可以绕过缓存。

注意，图 4.1 中描述的缓存组织不支持缓存一致性。例如，假设在 SM 1 上执行的线程读取内存位置 A，该值被存储在 SM 1 的 L1 数据缓存中，然后在 SM 2 上执行的另一个线程写入内存位置 A。如果 SM 1 上的任何线程随后在内存位置 A 被从 SM 1 的 L1 数据缓存中逐出之前读取它，它将获得旧值而不是新值。为了避免这个问题，从 Kepler 开始的 NVIDIA GPU 只允许用于寄存器溢出和栈数据的局部内存访问或只读全局内存数据被放置在 L1 数据缓存中。最近的研究探索了如何在 GPU 上实现一致的 L1 数据缓存 [Ren and Lis，2017，Singh et al., 2013] 以及明确定义 GPU 内存一致性模型的需求 [Alglave et al., 2015]。

4.1.2　L1 纹理缓存

NVIDIA 近期的 GPU 架构将 L1 数据缓存和纹理缓存合并以节省面积。为了更好地理解这种缓存如何工作，首先有必要了解一些独立纹理缓存的设计。这里涵盖的细节应有助于为如何开发吞吐处理器的微架构提供额外的直觉。这里的大部分讨论基于 Igehy 等人 [1998] 的论文，该论文旨在填补关于工业纹理缓存设计如何容忍缓存缺失的长片外延迟的文献空白。近期的工业 GPU 专利 [Minken et al., 2010，Minken and Rubinstein, 2003] 描述了密切相关的设计。由于本书的重点不是图形学，我们仅简要总结促使引入纹理缓存的纹理操作。

在 3D 图形中，希望使场景看起来尽可能逼真。为了以实时渲染所需的高帧率实现这种真实感，图形 API 采用了一种称为纹理映射的技术 [Catmull，1974]。在纹理映射中，将称为纹理的图像应用于 3D 模型中的表面，以使表面看起来更逼真。例如，纹理可以用于为场景中的桌子赋予天然木材的外观。为了实现纹理映射，渲染管线首先确定纹理图像内一个或多个采样点的坐标。这些采样点称为纹素。然后使用坐标来查找包含纹素的内存位置的地址。由于屏幕上的相邻像素映射到相邻纹素，并且通常会

对附近纹素的值进行平均，因此纹理内存访问中存在显著的局部性，可以被缓存利用[Hakura and Gupta，1997]。

图 4.2 展示了 Igehy 等人 [1998] 描述的 L1 纹理缓存的微架构。与第 4.1.1 节中描述的 L1 数据缓存相比，标签阵列②和数据阵列⑤被一个 FIFO 缓冲区③隔开。这个 FIFO 的动机是隐藏可能需要从 DRAM 服务的缺失请求的延迟。本质上，纹理缓存的设计假设缓存缺失会频繁发生，并且工作集大小相对较小。为了保持标签阵列和数据阵列的尺寸较小，标签阵列本质上领先于数据阵列运行。标签阵列的内容反映了在大致等于缺失请求往返内存的时间之后数据阵列的未来状态。虽然与容量和缺失处理资源有限的常规 CPU 设计相比吞吐量有所提高，但缓存命中和缺失都经历大致相同的延迟。

具体来说，图 4.2 中所示的纹理缓存操作如下。加载/存储单元①将计算出的纹素地址发送到标签阵列②进行查找。如果访问命中，则将指向数据阵列中数据位置的指针连同完成纹理操作所需的任何其他信息一起放入片段 FIFO③尾部的条目中。当条目到达片段 FIFO 的头部时，控制器④使用该指针从数据阵列⑤中查找纹素数据，并将其返回到纹理过滤单元⑥。虽然未详细展示，但对于双线性和三线性过滤（mipmap 过滤）等操作，每个片段（即屏幕像素）实际上有四次或八次纹素查找。纹理过滤单元将这些纹素组合以产生单个颜色值，该值通过寄存器文件返回到指令流水线。

如果在标签查找期间发生缓存缺失，标签阵列通过缺失请求 FIFO⑧发送内存请求。缺失请求 FIFO 将请求发送到存储系统的较低层级⑨。GPU 存储系统中的 DRAM 带宽利用率可以通过使用内存访问调度技术 [Eckert, 2008, 2015] 来提高，这些技术可以乱序服务内存请求以减少行切换惩罚。为了确保数据阵列⑤的内容反映标签阵列②的时延状态，数据必须按顺序从存储系统返回。这是通过使用重排序缓冲区⑩来实现的。

4.1.3 统一纹理与数据缓存

在 NVIDIA 和 AMD 近期的 GPU 架构中，数据和纹理值的缓存通过统一的 L1 缓存结构实现。为了以最直接的方式实现这一点，只有能够保证只读的数据值才会被缓存在 L1 中。对于遵循此限制的数据，纹理缓存硬件几乎可以不加修改地使用，只需更改寻址逻辑。这种设计在近期的一项专利中有所描述 [Heinrich 等人，

2017]。在 AMD 的 GCN GPU 架构中，所有向量内存操作都通过纹理缓存处理[AMD，2012]。

4.2 片上互连网络

为了提供 SIMT 核心所需的大量内存带宽，高性能 GPU 通过内存分区单元（见第 4.3 节）并行连接到多个 DRAM 芯片。内存流量通过地址交织分布在内存分区单元之间。一项 NVIDIA 专利描述了以 256 字节或 1，024 字节为粒度在最多 6 个内存分区之间平衡流量的地址交织方案 [Edmondson 和 Van Dyke，2011]。

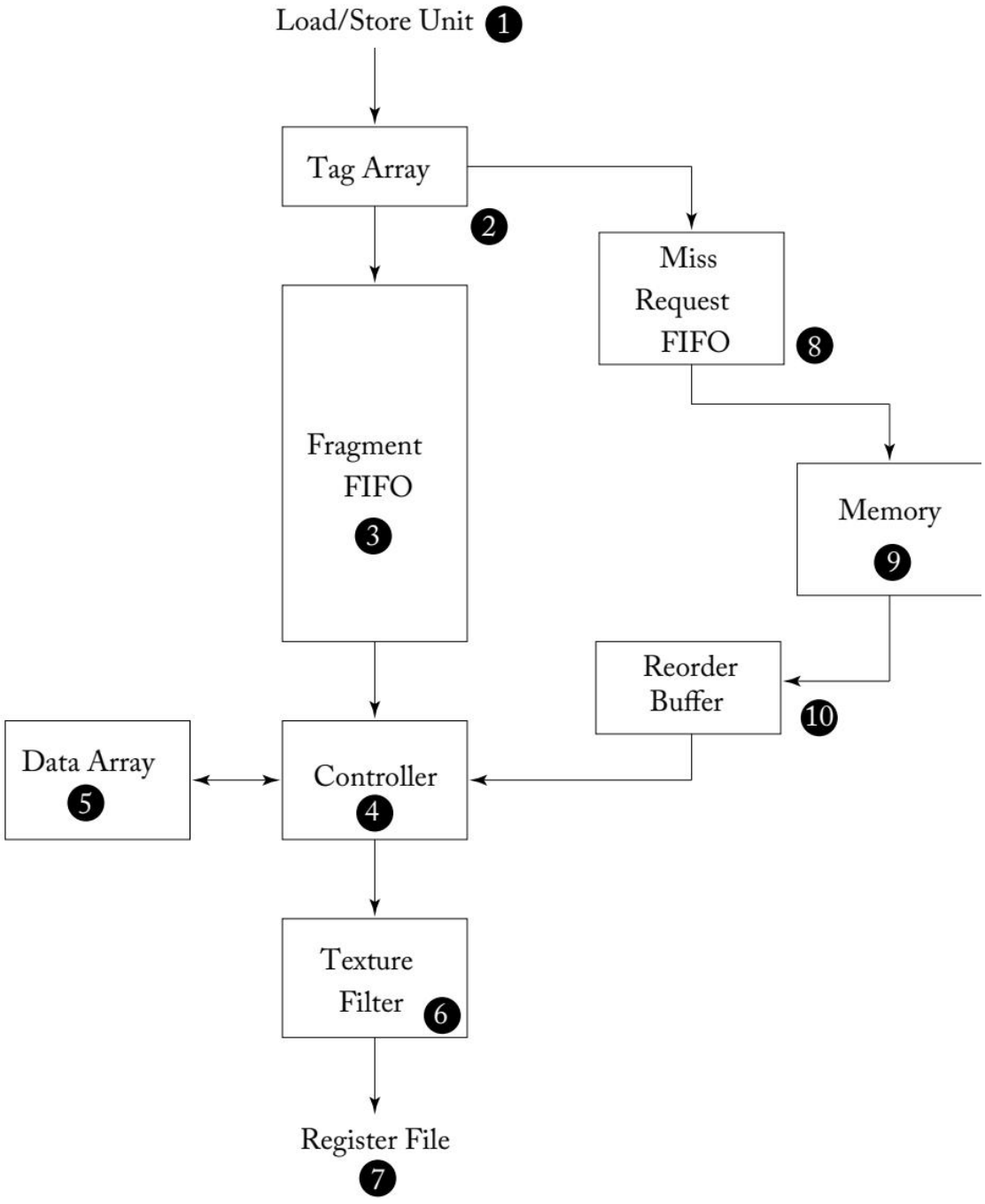


图 4.2　L1 纹理缓存（部分基于 [Igehy 等人，1998] 中的图 2）。

SIMT 核心通过片上互连网络连接到内存分区单元。NVIDIA 近期专利中描述的片上互连网络是交叉开关 [Glasco 等人，2013，Treichler 等人，2015]。AMD 的 GPU 有时被描述为使用环形网络 [Shrout，2007]。

4.3 内存分区单元

下面，我们描述对应于几项近期 NVIDIA 专利的内存分区单元的微架构。从历史背景来看，这些专利是在 NVIDIA Fermi GPU 架构发布前大约一年提交的。如图 4.3 所示，每个内存分区单元包含二级（L2）缓存的一部分，以及一个或多个内存访问调度器（也称为“帧缓冲区”或 FB）和光栅操作（ROP）单元。L2 缓存包含图形和计算数据。内存访问调度器对内存读写操作进行重排序，以减少访问 DRAM 的开销。ROP 单元主要用于图形操作，如 alpha 混合，并支持图形表面的压缩。ROP 单元还支持 CUDA 编程模型中的原子操作。这三个单元紧密耦合，下面将详细描述。

4.3.1 L2 缓存

L2 缓存设计包含多项优化，以提高 GPU 单位面积的整体吞吐量。每个内存分区内的 L2 缓存部分由两个切片组成 [Edmondson 等人，2013]。每个切片包含独立的标签和数据阵列，并按顺序处理传入请求 [Roberts 等人，2012]。为了匹配 GDDR5 中 32 字节的 DRAM 原子大小，切片内的每个缓存行有四个 32 字节的扇区。缓存行被分配用于存储指令或加载指令。为了在合并写入完全覆盖每个扇区的常见情况下优化吞吐量，在写未命中时不会先从内存读取数据。这与标准计算机体系结构教科书中通常描述的 CPU 缓存工作方式有很大不同。我们检查的专利中没有描述如何处理未完全覆盖扇区的非合并写入，但两种解决方案是存储字节级有效位和完全绕过 L2。为了减少内存访问调度器的面积，正在写入内存的数据在等待调度时缓存在 L2 的缓存行中。

4.3.2 原子操作

如 Glasco 等人 [2012] 所述，ROP 单元包含用于执行原子和归约操作的功能单元。访问同一内存位置的一系列原子操作可以流水线化，因为 ROP 单元包含本地 ROP 缓存。原子操作可用于实现不同线程块中运行线程之间的同步。

4.3.3 内存访问调度器

为了存储大量数据，GPU 采用特殊的动态随机存取存储器（DRAM），如 GDDR5。DRAM 将单个位存储在小电容器中。例如，要从这些电容器读取值，首先将一行位（称为页）读入一个称为行缓冲区的小型内存结构。为了完成此操作，连接各个存储电容器到行缓冲区的位线（其本身具有电容）必须首先预充电到 0 和电源电压之间的一半电压。在激活操作期间，当电容器通过访问晶体管连接到字线时，位线的电压会因电荷

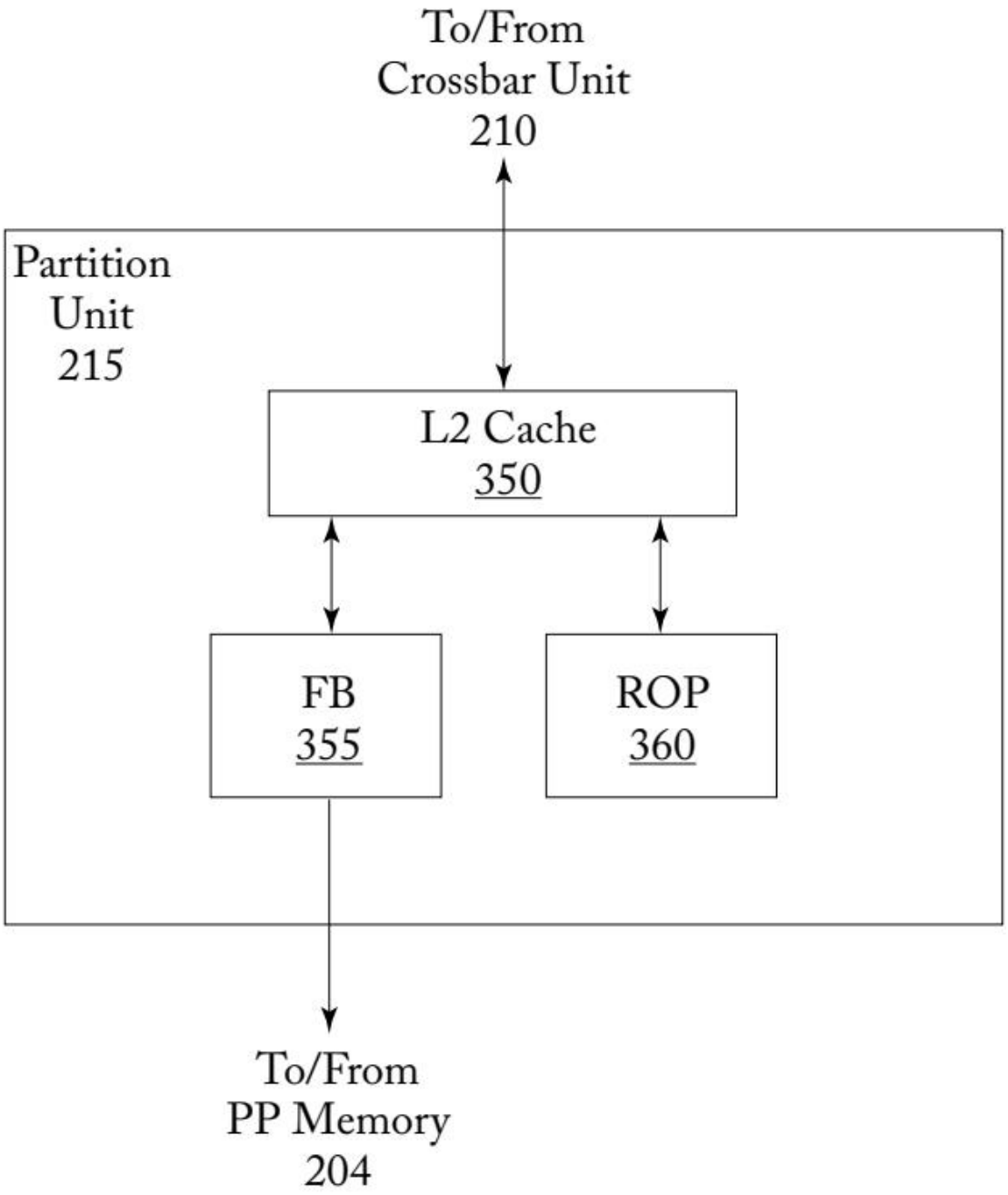


图 4.3　内存分区单元概览（基于 Edmondson 等人 [2013] 中的图 3B）。

从位线流入或流出存储单元而略微上升或下降。然后，灵敏放大器放大这个微小变化，直到读出干净的逻辑 0 或 1。将值读入行缓冲区的过程会刷新存储在电容器中的值。预充电和激活操作会引入延迟，在此期间无法从 DRAM 阵列读取或写入数据。为了减轻这些开销，DRAM 包含多个存储体，每个存储体都有自己的行缓冲区。然而，即使有多个 DRAM 存储体，在访问数据时通常也无法完全隐藏行切换的延迟。这导致了内存访问调度器的使用 [Rixner 等人，2000，Zuravleff 和 Robinson，1997]，它们对 DRAM 内存访问请求进行重排序，以减少数据在行缓冲区和 DRAM 单元之间移动的次数。

为了实现对 DRAM 的访问，GPU 中的每个内存分区可能包含多个内存访问调度器 [Keil 和 Edmondson，2012]，将其包含的 L2 缓存部分连接到片外 DRAM。最简单

的方法是让 L2 缓存的每个切片拥有自己的内存访问调度器。每个内存访问调度器包含独立的逻辑，用于对从 L2 缓存发送的读请求和写请求（“脏数据通知”）进行排序 [Keil 等人，2012]。为了将同一 DRAM 存储体中同一行的读请求分组在一起，采用了两个独立的表。第一个称为读请求排序器，是一个按内存地址访问的组相联结构，将给定存储体中同一行的所有读请求映射到单个指针。该指针用于在第二个称为读请求存储的表中查找各个读请求的列表。

4.4 GPU 内存系统的研究方向

4.4.1 内存访问调度与互连网络设计

Yuan 等人 [2009] 探索了运行用 CUDA 编写的 GPU 计算应用的 GPU 的内存访问调度器设计。他们观察到，由单个流多处理器（SM）生成的请求具有行缓冲区局部性。如果序列中附近出现的请求访问同一 DRAM 存储体中的同一 DRAM 行，则称对给定内存分区的内存请求序列具有行缓冲区局部性。然而，当一个 SM 的内存请求被发送到内存分区时，它们会与来自其他 SM 发送到同一内存分区的请求混合在一起。结果是进入内存分区的请求序列的行缓冲区局部性降低。Yuan 等人 [2009] 提出通过修改互连网络以保持行缓冲区局部性来降低内存访问调度的复杂性。他们通过引入仲裁策略来实现这一点，这些策略优先授予包含来自同一 SM 或具有相似行-存储体地址的内存请求的数据包。

Bakhoda 等人 [2010, 2013] 探索了 GPU 的片上互连网络设计。这种互连将流多处理器连接到内存分区。他们认为，随着 SM 数量的增加，将有必要采用诸如网格之类的可扩展拓扑。他们探索了片上网络设计如何影响系统吞吐量，并发现

许多 CUDA 应用的吞吐量对互连延迟相对不敏感。他们分析了互连流量，发现它具有多对少对多的模式。他们提出了一种更受限的可扩展拓扑，由“半路由器”组成，利用这种流量模式降低了路由器的面积成本。

4.4.2 缓存有效性

Bakhoda 等人 [2009] 研究了为全局内存访问添加 L1 和/或 L2 缓存对使用其 GPGPU-Sim 模拟器模拟的 CUDA GPU 的影响，并表明虽然一些应用受益，但其他应用则没有。

Jia 等人 [2012] 的后续工作通过在 NVIDIA Fermi GPU 硬件上启用或禁用缓存来表征缓存的有效性，并发现了类似的结果。观察到通过 L1 缓存将数据读入暂存器共享内存的应用不会从启用 L1 缓存中受益。即使排除此类应用，Jia 等人 [2012] 也观察到仅凭缓存命中率不足以预测缓存是否会提高性能。他们发现，相反，有必要考虑缓存对 L2 缓存（例如内存分区）请求流量的影响。在他们研究的 Fermi GPU 上，L1 缓存不是分段的，因此启用缓存在未命中时可能导致更大的 128 字节片外内存访问。在内

存带宽受限的应用中，这种额外的片外内存流量可能导致性能下降。Jia 等人 [2012] 引入了三种局部性的分类：warp 内、块内和跨指令。当单个 warp 内不同线程执行的单个加载的内存读取访问访问同一缓存块时，发生 warp 内局部性。当同一线程块中不同 warp 的线程执行的单个加载的内存读取访问访问同一缓存块时，发生块内局部性。当同一线程块中线程执行的不同加载指令的内存读取访问访问同一缓存块时，发生跨指令局部性。Jia 等人 [2012] 引入了一种使用此分类的编译时算法，以帮助推断启用缓存何时对单个加载指令有帮助。

4.4.3　内存请求优先级和缓存绕过

在上述表征研究 [Jia 等人，2012] 以及 Rogers 等人 [2012] 的工作（证明了 warp 调度可以提高缓存有效性，见第 5.1.2 节）之后，Jia 等人 [2014] 提出了 GPU 的内存请求优先级和缓存绕过技术。相对于线程数而言相联度较低的缓存可能会遭受显著的冲突未命中 [Chen 和 Aamodt, 2009]。Jia 等人 [2014] 指出，用 CUDA 编写的几个 GPGPU 应用包含数组索引，当使用标准模缓存集索引函数 [Hennessy 和 Patterson, 2011, Patterson 和 Hennessy，2013] 时，这会导致来自单个 warp 的单个内存请求产生映射到同一缓存集的未命中。Jia 等人 [2014] 称之为 warp 内争用。假设在检测到未命中时分配缓存空间 1，并且未命中状态保持寄存器的数量有限 2，warp 内争用可能导致内存流水线停顿。3 为了解决 warp 内争用，Jia 等人 [2014] 提出在发生未命中且由于相联度停顿而无法分配缓存块时绕过 L1 数据缓存。当缓存集中的所有块都被保留以提供将由未完成的缓存未命中提供的数据空间时，就会发生相联度停顿。

Jia 等人 [2014] 还研究了他们所谓的跨 warp 争用。这种形式的缓存争用发生在一个 warp 驱逐另一个 warp 引入的数据时。为了解决这种形式的争用，Jia 等人 [2014] 建议采用一种他们称为“内存请求优先级缓冲区”（MRPB）的结构。与 CCWS [Rogers 等人，2012] 一样，MRPB 通过修改对缓存的访问顺序以增加局部性来减少容量未命中。然而，与通过线程调度间接实现此目的的 CCWS 不同，MRPB 试图在线程调度后通过改变单个内存访问的顺序来增加局部性。

MRPB 在 L1 数据缓存之前实现内存请求重排序。MRPB 的输入是在执行内存请求合并后，在指令发射流水线阶段生成的内存请求。MRPB 的输出将内存请求馈送到 L1 缓存。在内部，MRPB 包含几个并行的先进先出（FIFO）队列。缓存请求使用“签名”分发到这些 FIFO。在他们评估的几种选项中，他们发现最有效的签名是使用“warp ID”（一个介于 0 到流多处理器上可运行的最大 warp 数之间的数字）。MRPB 采用“排空策略”来确定接下来从哪个 FIFO 选择内存请求来访问缓存。在探索的几种选项中，最佳版本是一种简单的固定优先级方案，其中每个队列被分配一个静态优先级，包含请求的最高优先级队列首先被服务。

详细评估表明，使用 MRPB 的旁路与重排序联合机制在 64 路 16 KB 缓存上实现了 4% 的几何平均加速。Jia 等人 [2014] 还与 CCWS 进行了一些比较，显示出更大的改进。顺带一提，我们注意到 Rogers 等人 [2012] 的评估采用了一种基线架构，该架构

使用了更复杂的组索引哈希函数[1]来减少相联停顿的影响。此外，Nugteren 等人 [2014] 的后续工作致力于逆向工程 NVIDIA Fermi 架构中实际使用的组索引哈希函数的细节，并发现它使用了异或运算（这也有助于减少此类冲突）。

与 Rogers 等人 [2013] 类似，Jia 等人 [2014] 表明，他们这种对程序员透明的性能改进方法可以缩小使用缓存的简单代码与使用暂存器共享内存的高度优化代码之间的差距。

Arunkumar 等人 [2016] 根据静态指令中存在的内存发散程度，探讨了旁路和改变缓存行大小的影响。他们利用观察到的重用距离模式和内存发散度来预测旁路和最优缓存行大小。

Lee 和 Wu[2016] 提出了一种基于控制环的缓存旁路方法，试图在运行时逐条指令地预测重用行为。缓存行的重用行为受到监控。如果特定程序计数器加载的缓存行没有经历足够的重用，则该指令的访问将被旁路。

4.4.4 利用线程束间异构性

Ausavarungnirun 等人 [2015] 提出了一系列针对 GPU 共享 L2 缓存和内存控制器的改进，以缓解不规则 GPU 应用中的内存延迟发散。这些技术统称为内存发散校正（MeDiC），它们利用了同一内核中不同线程束之间内存延迟发散程度存在异构性的观察结果。根据与共享 L2 缓存的交互方式，内核中的每个线程束可以被表征为全部/大部分命中、全部/大部分缺失或均衡。作者证明，对于非全部命中的线程束，收益甚微，因为大部分命中的线程束必须等待最慢的访问返回才能继续执行。他们还证明，L2 缓存处的排队延迟可能产生非平凡的性能影响，并且对于所有非全部命中的线程束，通过让所有请求（即使是可能命中的请求）旁路 L2 缓存，可以缓解这种影响。这通过减少排队延迟降低了全部命中线程束的访问延迟。除了自适应旁路技术外，他们还提出了对缓存替换策略和内存控制器调度器的修改，以尽量减少被检测为全部命中线程束的延迟。他们还证明，即使对于全部命中的线程束，L2 缓存体之间的排队延迟差异也可能导致额外的、可能避免的排队延迟，因为 L2 体之间的排队延迟存在不平衡。

作者提出的微架构机制由四个组件组成：（1）线程束类型检测模块——将 GPU 中的线程束分类为五种潜在类型之一：全部缺失、大部分缺失、均衡、大部分命中或全部命中；（2）线程束类型感知旁路逻辑模块，决定请求是否应旁路 L2 缓存；（3）线程束类型感知插入策略，确定 L2 中的插入在 LRU 栈中的放置位置；（4）线程束类型感知内存调度器，对 L2 缺失/旁路请求发送到 DRAM 的顺序进行排序。

检测机制通过基于间隔采样每个线程束的命中率（总命中数/访问数）来运作。基于该比率，线程束采用上述五种分类之一。确定这些分类边界的确切命中率针对每个工作负载动态调整。在分类间隔期间，没有请求旁路缓存，以便对每个线程束 L2 特性

[1]参见 https：//github。com/tgrogers/ccws-2012/blob/master/simulator/ccws_gpgpu-sim/distribution/src/gpgpu-sim/gpu-cache。cc 中的 cache_config：：set_index_hashed。

的相位变化做出反应。

旁路机制位于 L2 缓存之前，接收标记有生成它们的线程束类型的内存请求。该机制试图消除来自全部缺失线程束的访问，并将大部分缺失线程束转换为全部缺失线程束。该模块简单地将标记为来自全部缺失和大部分缺失线程束的所有请求直接发送到内存调度器。

MeDiC 的缓存管理策略通过改变从 DRAM 返回的请求在 L2 的 LRU 栈中的放置位置来运作。由大部分缺失线程束请求的缓存行被插入到 LRU 位置，而所有其他请求被插入到传统的 MRU 位置。

最后，MeDiC 修改基线内存请求调度器，使其包含两个内存访问队列：一个用于全部命中和大部分命中线程束的高优先级队列，以及一个用于均衡、大部分缺失和全部缺失线程束的低优先级队列。内存调度器简单地优先处理高优先级队列中的所有请求，而非低优先级队列中的任何请求。

4.4.5　协同缓存旁路

Xie 等人 [2015] 探索了选择性启用缓存旁路以提高缓存命中率的潜力。他们采用性能分析来确定 GPGPU 应用中每个静态加载指令是否具有良好的局部性、较差的局部性或中等的局部性。他们相应地标记指令。标记为具有良好局部性的加载操作被允许使用 L1 数据缓存。标记为具有较差局部性的加载操作总是被旁路。标记为中等局部性的加载指令采用一种自适应机制，其工作原理如下。该自适应机制在线程块粒度上运作。对于给定的线程块，执行的所有中等局部性加载被统一处理。它们要么使用 L1，要么旁路。该行为在线程块启动时基于一个阈值确定，该阈值使用考虑 L1 缓存命中和流水线资源冲突的性能指标在线调整。他们的评估表明，这种方法比静态线程束限制显著提高了缓存命中率。

4.4.6　自适应缓存管理

Chen 等人 [2014b] 提出了协同缓存旁路和线程束限流的方法，利用线程束限流和缓存旁路两者的优势来提高对缓存高度敏感的应用程序的性能。所提出的机制在运行时检测缓存争用和内存资源争用，并相应地协调限流和旁路策略。该机制通过现有的 CPU 缓存旁路技术——保护距离来实现缓存旁路，保护距离防止缓存行在若干次访问中被逐出。当缓存行插入缓存时，会为其分配一个保护距离，计数器跟踪缓存行的剩余保护距离。一旦剩余保护距离达到 0，该缓存行就不再受保护，可以被逐出。当新的内存请求试图将新行插入到没有未受保护行的组中时，该内存请求会绕过缓存。

保护距离是全局设置的，其最优值因工作负载而异。在这项工作中，Chen 等人 [2014b] 扫描了静态保护距离，并证明 GPU 工作负载对保护距离值相对不敏感。

4.4.7 缓存优先级

Li 等人 [2015] 观察到，线程束限流优化了 L1 缓存命中率，但可能使其他资源（如片外带宽和 L2 缓存）显著未得到充分利用。他们提出了一种向线程束分配令牌的机制，以确定哪些线程束可以将行分配到 L1 缓存中。额外的“非污染线程束”不会获得令牌，因此虽然它们可以执行，但不允许从 L1 中逐出数据。这导致了一个优化空间，其中可以调度的线程束数量（W）和拥有令牌的线程束数量（T）都可以设置为小于可执行的最大线程束数量。他们表明，静态选择 W 和 T 的最优值比采用静态线程束限制的 CCWS 提高了 17%。

基于这一观察，Li 等人 [2015] 探索了两种学习 W 和 T 最佳值的机制。第一种方法基于在保持高线程级并行性的同时提高缓存命中率的思想。在这种称为 dynPCALMTLP 的方法中，采样期运行内核时将 W 设置为最大线程束数，然后在不同的 SIMT 核心上改变 T。然后选择实现最大性能的 T 值。这使得性能与 CCWS 相当，而面积开销显著减少。第二种方法称为 dynPCALCCWS，最初使用 CCWS 设置 W，然后使用 dynPCALMTLP 确定 T。然后它监控共享结构的资源使用情况，以动态增加或减少 W。与 CCWS 相比，这带来了 11% 的性能提升。

4.4.8 虚拟内存页面放置

Agarwal 等人 [2015] 考虑了在异构系统中跨多种物理内存类型支持缓存一致性的影响，这些内存包括容量优化型和带宽优化型内存。由于带宽优化的 DRAM 在成本和能耗上比容量优化的 DRAM 更昂贵，未来的系统可能会同时包含两者。Agarwal 等人 [2015] 观察到当前的操作系统页面放置策略（如 Linux 中部署的策略）没有考虑内存带宽的非均匀性。他们研究了一种未来系统，其中 GPU 可以以低延迟（100 个核心周期的惩罚）访问低带宽/高容量的 CPU 内存。他们的实验使用修改版的 GPGPU-Sim 3.2.2，并配置了额外的 MSHR 资源来模拟更近期的 GPU。

通过这种设置，他们首先发现对于内存带宽受限的应用程序，通过同时使用 CPU 和 GPU 内存来增加总内存带宽，有显著的机会获得性能提升。他们发现内存延迟受限较少的 GPGPU 应用程序则并非如此。在页面被均匀访问且带宽优化型内存的容量不受限制的假设下，他们表明按照各区域可用内存带宽的比例将页面分配给内存区域是最优的。假设带宽受限内存的容量不是问题，他们发现一种简单的策略——以与内存带宽成比例的概率随机将页面分配给带宽优化型或容量优化型内存——在实际的 GPGPU 程序中是有效的。然而，当带宽优化型内存容量不足以满足应用需求时，他们发现有必要细化页面放置以考虑访问频率。

为了细化页面放置，他们提出了一个系统，涉及使用修改版的 NVIDIA 开发者工具 nvcc 和 ptxas 实现的性能分析过程，以及扩展现有的 CUDA API 以包含页面放置提示。使用性能分析引导的页面放置提示可以获得 oracle 页面放置算法约 90% 的收益。他们将页面迁移策略留待未来工作。

4.4.9　数据放置

Chen 等人 [2014a] 提出了 PORPLE，一种可移植的数据放置策略，由规范语言、源到源编译器和自适应运行时数据放置器组成。他们利用的观察结果是，由于 GPU 上存在各种类型的内存，程序员难以确定哪些数据应放置在何处，并且这通常无法从一个 GPU 架构移植到下一个。PORPLE 的目标是可扩展、输入自适应，并且普遍适用于规则和不规则数据访问。他们的方法依赖于三种解决方案。

第一种解决方案是一种内存规范语言，以帮助实现可扩展性和可移植性。内存规范语言根据访问这些空间被串行化的条件，描述了 GPU 上所有各种形式的内存。例如，对相邻全局数据的访问是合并的，因此是并发访问的，但对同一共享内存存储体的访问必须串行化。

第二种解决方案是一个名为 PORPLE-C 的源到源编译器，它将原始 GPU 程序转换为与放置无关的版本。编译器在内存访问周围插入守卫，选择与预测的最佳数据放置相对应的访问。

最后，为了预测哪种数据放置最为优化，他们使用 PORPLE-C 通过代码分析来寻找静态访问模式。当静态分析无法确定访问模式时，编译器会生成一个函数，该函数跟踪运行时访问模式并尝试做出预测。这个函数在 CPU 上运行一小段时间，帮助在启动内核之前确定最佳的基于 GPU 的数据放置。在这项工作的范围内，系统只处理数组的放置，因为它们是 GPU 内核中最常用的数据结构。

PORPLE 中用于做出放置预测的轻量级模型会根据内存的序列化条件生成事务数量的估计。对于具有缓存层次结构的内存，它使用缓存的复用距离估计

命中率。当多个数组共享一个缓存时，每个数组所分配的缓存量的估计基于数组大小的缓存线性划分。

4.4.10　多芯片模块 GPU

Arunkumar 等人 [2017] 指出，摩尔定律的放缓将导致 GPU 性能增长放缓。他们提出通过在多芯片模块上使用较小的 GPU 模块构建大型 GPU 来扩展性能规模（见图 4.4）。他们证明，通过结合远程数据的本地缓存、考虑局部性的 CTA 调度到模块以及首次触摸页面分配，可以实现单个大型（且无法实现的）单片 GPU 性能的 10%。根据他们的分析，这比在相同工艺技术下使用最大的可实现单片 GPU 可能达到的性能高出 45%。

注释

- 1 GPGPU-Sim 中的默认设置，用于避免协议死锁。
- 2 与有限的待处理请求表条目集一致——见第 4.1.1 节。
- 3 Jia 等人 [2014] 使用的 GPGPU-Sim 3.2.0 版本未对第 3.3.2 节和第 4.1 节中描述

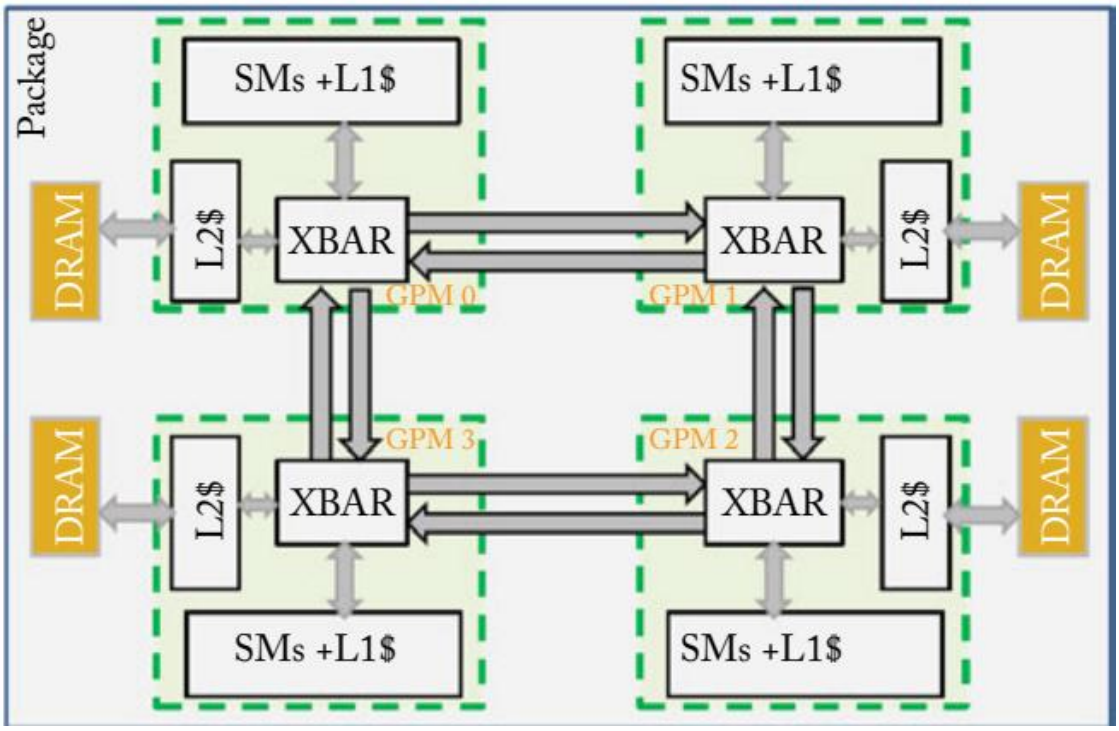


图 4.4 多芯片模块 GPU（基于 Arunkumar 等人 [2017] 的图 3）。

的指令重放进行建模。

第五章　GPU 计算体系结构的跨领域研究

本章详细介绍了 GPGPU 架构中的几个研究方向，这些方向无法完全归入前面专注于 GPU 架构特定部分的章节。5.1 节探讨了 GPU 中线程调度的工作。5.2 节关注替代编程方法，5.4 节考察了异构 CPU/GPU 计算的工作。

5.1　线程调度

当代 GPU 与 CPU 的根本区别在于它们依赖于大规模并行性。无论程序如何指定（例如，使用 OpenCL、CUDA、OpenACC 等），没有大量软件定义并行性的工作负载不适合 GPU 加速。GPU 采用多种机制来聚合和调度所有这些线程。GPU 上线程的组织和调度主要有三种方式。

线程到线程束的分配由于 GPU 使用 SIMD 单元来执行由 MIMD 编程模型定义的线程，因此必须将线程融合在一起，以线程束的形式进行锁步执行。在本书研究的基线 GPU 架构中，具有连续线程 ID 的线程被静态融合在一起形成线程束。第 3.4.1 节总结了关于线程束内替代线程排列以实现更好线程束压缩的研究提案。

线程块到核心的动态分配与 CPU 中线程可以一次一个地分配给硬件线程不同，在 GPU 中，工作以批量方式分配给 GPU 核心。这个工作单元由多个线程束组成，形式为线程块。在我们的基线 GPU 中，线程块按轮询顺序分配给核心。核心的资源（如线程束槽、寄存器文件和共享内存空间）以线程块粒度进行订阅。由于每个线程块关联大量状态，当代 GPU 不会抢占其执行。线程块中的线程运行至完成，然后其资源才能分配给另一个线程块。

逐周期调度决策在线程块分配给 GPU 核心后，一组细粒度硬件调度器在每个周期决定哪些线程束第 5 章

取指令，哪些线程束发出指令执行，以及何时为每个发出的指令读/写操作数。

多内核调度线程块级和逐周期调度决策可以在同一 GPU 上并发运行的内核内部和不同内核之间进行。传统的内核调度限制一次只能有一个内核在 GPU 上活动。然而，NVIDIA 的 Streams 和 HyperQ 调度机制的引入使得并发内核的运行成为可能。这种情况在某些方面类似于 CPU 上的多道程序设计。

5.1.1　线程块到核心分配的研究

当内核启动时，每个内核启动内的线程被分组为线程块。GPU 范围的线程块调度机制根据资源可用性将每个线程块分配给一个 SIMT 核心。每个核心有固定数量的暂

存器内存（在 CUDA 中称为共享内存，在 OpenCL 中称为本地内存）、寄存器数量、线程束槽和线程块槽。在内核启动时，每个线程块的所有这些参数都是已知的。最明显的线程块调度算法是以轮询方式将线程块分配给核心，以最大化涉及的核心数量。线程块持续调度，直到每个核心中至少有一个资源耗尽。注意，一个内核可能由比 GPU 一次能运行的更多的线程块组成。因此，内核中的一些线程块在其他线程块执行时甚至可能不在 GPU 上运行。一些研究技术探讨了线程块调度空间中的权衡。

线程块级别的节流。Kayiran 等人 [2013] 提出对分配给每个核心的线程块数量进行节流，以减少线程过度订阅导致的内存系统争用。他们开发了一种算法，监控核心空闲周期和内存延迟周期。该算法首先为每个核心仅分配其最大线程块数的一半，然后监控空闲周期和内存延迟周期。如果某个核心主要因等待内存而停滞，则不再为其分配更多线程块，并可能暂停现有线程块以阻止它们发射指令。该技术实现了一种粗粒度的并行节流机制，限制了内存系统干扰并提升了整体应用性能，尽管同时活跃的 CTA 数量减少了。

动态调整 GPU 资源。Sethia 和 Mahlke[2014] 提出了 Equalizer，一种硬件运行时系统，动态监控资源争用并调整线程数量、核心频率和内存频率，以同时改善能耗和性能。该系统基于四个参数做出决策：（1）SM 中活跃 warp 的数量；（2）等待内存数据的 warp 数量；（3）准备执行算术指令的 warp 数量；（4）准备执行内存指令的 warp 数量。利用这些参数，它首先决定在 SM 上保持活跃的 warp 数量，然后基于该值和其他三个计数器的值（它们分别作为内存争用、计算强度和内存强度的代理），决定如何最佳地调整核心和内存系统的频率。

Equalizer 有两种操作模式：节能模式和性能增强模式。在节能模式下，它通过缩减未充分利用的资源来节省能量，以最小化能耗，同时减轻对性能的影响。在性能增强模式下，Equalizer 提升瓶颈资源，以节能的方式提高性能。

他们通过考察改变内存频率、计算频率和并发运行线程数所带来的性能和能耗权衡，将一组来自 Rodinia 和 Parboil 的工作负载表征为计算密集型、内存密集型、缓存敏感型或未饱和型。如果目标是最小化能耗（而不牺牲性能），那么计算密集型内核应以较低的内存频率运行，而内存密集型内核应以较低的 SIMT 核心频率运行。这有助于减少系统在基线速率下未充分利用时不必要消耗的能量。

Equalizer 基于间隔对频率和并发性做出决策。该技术为每个 SIMT 核心添加了监控硬件，根据前面列出的四个计数器做出本地决策。每个 SIMT 核心在本地决定该周期内三个输出参数（CTA 数量、内存频率和计算频率）的取值。它通知全局工作分配引擎该 SM 应使用的 CTA 数量，如果 SIMT 核心需要更多工作，则发射新块。如果 SM 应以较少的 CTA 运行，则暂停核心上的一些 CTA。在决定要运行的 CTA 数量后，每个 SIMT 核心向全局频率管理器提交内存/计算电压目标，全局频率管理器基于多数函数设置芯片级频率。

本地决策通过观察等待执行内存指令的 warp 数量和等待执行 ALU 指令的 warp 数量来做出。如果等待内存的 warp 数量大于一个 CTA 中的 warp 数量，则减少该 SIMT

核心上运行的 CTA 数量，这可能有助于缓存敏感型工作负载的性能。如果准备发射内存（或 ALU）指令的 warp 数量大于一个 CTA 中的 warp 数量，则认为该 SIMT 核心是内存（或计算）密集型的。如果等待内存（或计算）的 warp 数量少于一个 CTA 中的 warp 数量，但超过一半的活跃 warp 在等待且等待内存的 warp 不超过两个，则该 SIMT 核心仍可被视为 ALU 或内存受限。在这种情况下，核心上活跃的 CTA 数量增加一，并根据等待计算的 warp 更多还是等待内存的 warp 更多来确定 SIMT 核心是计算受限还是内存受限。

一旦 SIMT 核心做出本地决策，内存和核心的频率将根据 Equalizer 所处的操作模式按 $\pm 15\%$ 进行调整。

5.1.2　逐周期调度决策的研究

逐周期调度的早期特征分析。Lakshminarayana 和 Kim[2010] 在无硬件管理缓存的早期 GPU 背景下探索了多种 warp 调度策略，并表明，对于每个 warp 执行对称（均衡）动态指令计数的应用，基于公平性的 warp 和 DRAM 访问调度策略能提升性能。该策略在他们研究所用的常规 GPU 工作负载上效果良好，因为 warp 间的常规内存请求既在核心内被合并，又更好地利用了 DRAM 行缓冲区局部性。该论文还描述了其他几种 warp 调度策略，包括 ICOUNT，该策略最初由 Tullsen 等人 [1996] 为同时多线程 CPU 提出。ICOUNT 旨在通过优先调度进展最快的 warp（或线程）来提高系统吞吐量。Lakshminarayana 和 Kim[2010] 表明，在其早期无缓存的 GPU 上，对于早期的常规工作负载，仅优先调度少数 warp 通常不会提升性能。

两级调度。Gebhart 等人 [2011c] 引入了两级调度器的使用以提高能效。他们的两级调度器将核心中的 warp 分为两个池：一个是在下一周期考虑调度的活跃 warp 池，另一个是不予考虑的非活跃 warp 池。当 warp 遇到编译器识别的全局或纹理内存依赖时，会从活跃池转移到非活跃池，并以轮转方式从非活跃池返回活跃池。每周期从较小的 warp 池中选择，减少了 warp 选择逻辑的规模和能耗。

Narasiman 等人 [2011] 提出的两级调度器侧重于通过允许线程组在不同时间到达相同的长延迟操作来提升性能。这有助于确保一个取指组内的缓存和行缓冲区局部性得以维持。系统随后可以通过在取指组之间切换来隐藏长延迟操作。相比之下，缓存感知的线程束调度（见下文）侧重于根据线程束内局部性的丢失程度，自适应地限制系统可维持的多线程数量，从而提升性能。

缓存感知的线程束调度。Rogers 等人 [2012] 将 GPU 内核中存在的内存局部性归类为线程束内局部性（即一个线程束的加载随后引用其自身的数据）或线程束间局部性（即一个线程束与其他线程束共享数据）。他们证明，线程束内局部性是缓存敏感型工作负载中最常见的局部性形式。基于这一观察，他们提出了一种缓存感知的波前调度（CCWS）机制，通过根据内存系统反馈限制主动调度的线程束数量来利用这种局部性。

在更少的线程束之间进行主动调度，使每个单独的线程束能够占用更多的缓存空间，并减少 L1 数据缓存的争用。特别是，当具有局部性的工作负载正在剧烈颠簸缓存时，就会发生限制。为了检测这种颠簸，CCWS 引入了一种丢失局部性检测机制，该机制基于 L1 数据缓存的替换牺牲者标签。

图 5.1 展示了 CCWS 的高层微架构。每次从缓存中逐出时，牺牲者的标签被写入线程束私有的牺牲者标签数组。每个线程束都有自己的牺牲者标签数组，因为 CCWS 只关心检测线程束内局部性。在随后的每次缓存缺失时，都会探测缺失线程束的牺牲者标签数组。如果在牺牲者标签中找到该标签，则说明某些线程束内局部性已经丢失。CCWS 假设，如果该线程束对 L1 数据缓存有更多的独占访问权，它可能本来能够命中该行，因此可能从限制中受益。

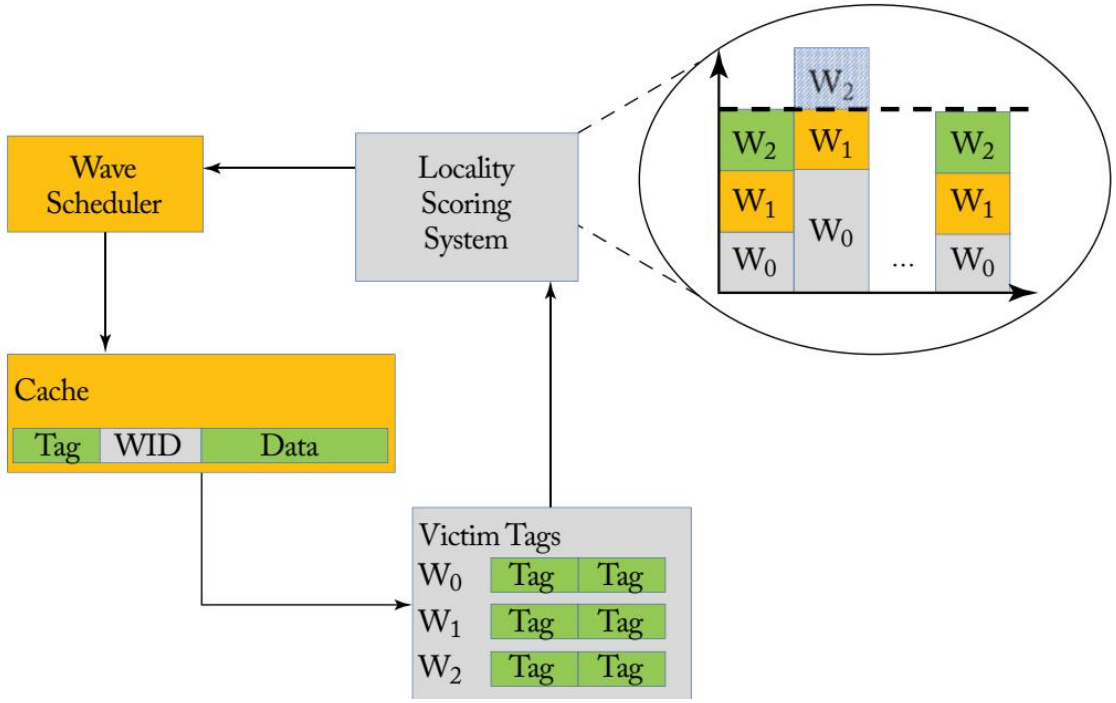


图 5.1 缓存感知的波前调度微架构。

为了反映这种局部性的丢失，会向调度系统发送一个信号。发射调度器使用局部性评分系统来近似系统中每个线程束已丢失的局部性量，这近似于每个线程束需要多少额外的缓存容量。局部性评分系统中的所有线程束都被分配一个初始分数，假设所有线程束需要相同的缓存容量且不发生限制（图 5.1 中的堆叠条）。随着时间的推移和检测到局部性丢失，各个线程束的分数会增加。在图 5.1 的示例中，线程束 0 经历了局部性丢失，其分数已增加。其分数的增加将线程束 3 推过了阈值，这将阻止它发出 L1 数据缓存请求，有效地限制了核心上主动调度的线程束数量。随着时间的推移，如果没有局部性丢失，则线程束 0 的分数会降低，直到线程束 2 能够降到阈值以下并能够再次发出内存请求。

CCWS 进一步通过比较各种调度机制与缓存替换策略，证明了缓存命中率对调度决策的敏感性。该论文表明，线程束调度器可用的决策空间远大于替换策略所做的相

对受限的决策空间。论文进一步证明，使用 LRU 替换策略的 CCWS 方案可以比先前的调度机制（即使它们使用 Belady 最优缓存替换策略）更好地提高缓存命中率。

Rogers 等人 [2013] 提出了发散感知的线程束调度（DAWS），它通过更准确地估计每个线程束的缓存占用来扩展 CCWS。DAWS 利用了 GPU 工作负载中大多数线程束内局部性发生在循环中的事实。DAWS 为循环中的线程束创建每个线程束的缓存占用估计。DAWS 根据每个线程束预测的循环占用，抢先限制循环中的线程束数量。DAWS 还根据每个线程束经历的控制流发散程度调整其缓存占用估计。线程束中已离开循环的线程不再对占用估计做出贡献。DAWS 进一步探索了 GPU 的可编程性方面，证明通过更智能的线程束调度器，未对内存传输进行优化（例如，使用共享内存而不是缓存）的基准测试可以非常接近同一基准测试的 GPU 优化版本。

预取感知的线程束调度。Jog 等人 [2013b] 探索了 GPU 上的预取感知线程束调度器。他们基于两级调度机制构建调度器，但从非连续线程束形成取指组。这种策略增加了 DRAM 中的 bank 级并行性，因为预取器不会对连续的访问查询同一个 DRAM bank。他们进一步扩展了这一想法，根据线程束组分配来操纵预取器。通过为其他组中的线程束预取数据，他们可以改善行缓冲区局部性，并在预取请求和数据需求之间提供间隔。

CTA 感知的调度。Jog 等人 [2013a] 提出了一种 CTA 感知的线程束调度器，它也建立在两级调度器的基础上，通过选择性地组合 CTA 来形成取指组。他们利用几个基于 CTA 的特性来提高性能。他们采用限制优先级技术来限制核心中活动线程束的数量，类似于其他限制调度器。结合限制，他们利用不同核心上 CTA 之间的跨 CTA 页面局部性。在仅局部性感知的 CTA 调度器下，连续的 CTA 通常会同时访问同一个 DRAM bank，从而降低 bank 级并行性。他们将其与预取机制结合，以改善 DRAM 行局部性。

调度对分支发散缓解技术的影响。Meng 等人 [2010] 提出了动态 warp 细分（DWS），当一些 lane 在缓存中命中而另一些 lane 未命中时，将 warp 拆分。该方案允许在缓存中命中的单个标量线程即使其 warp 同伴未命中也能取得进展。DWS 通过允许先行线程更早地发起其未命中，并为那些落后的线程创造预取效果，从而提高性能。DWS 试图通过提高数据加载到缓存的速率来改善 warp 内局部性。

Fung 等人 [2007] 探讨了 warp 调度策略对其动态 warp 形成（DWF）技术有效性的影响。DWF 试图通过在同一 warp 中的标量线程在分支指令上走不同路径时动态创建新 warp 来缓解控制流发散。他们提出了五种调度器并评估了它们对 DWF 的影响。

Fung 和 Aamodt[2011] 还提出了三种线程块优先级机制来补充他们的线程块压缩（TBC）技术。这些优先级机制试图将同一 CTA 内的线程调度在一起。他们的方法类似于 Narasiman 等人 [2011] 提出的两级调度的同期工作，不同之处在于线程块是一起调度的，而不是取指组。

第 3.4 节包含 DWS、DWF 和 TBC 的更详细总结。

调度与缓存重新执行。Sethia 等人 [2015] 引入了 Mascar，它试图在内存密集型工

作负载中更好地重叠计算与内存访问。Mascar 由两种相互交织的机制组成。

• 一种内存感知 warp 调度器（MAS），当核心中的 MSHR 和 L1 未命中队列条目被过度订阅时，优先执行单个 warp。这种优先级排序有助于即使工作负载不包含数据局部性时也能提高性能，因为它使在有序核心上执行的 warp 能够更快地到达其计算操作，从而实现优先 warp 的计算与其他 warp 的内存访问的重叠。• 一种缓存访问重新执行（CAR）机制，通过在有数据在缓存中的 warp 因低局部性访问阻塞内存流水线而无法发射时，允许 L1 数据缓存在未命中下命中，从而帮助避免 L1 数据缓存抖动。MAS 有两种操作模式：等优先级（EP）和内存访问优先级（MAP）模式。系统根据 L1 MSHR 和内存未命中队列的满度在 EP 和 MP 之间切换。一旦这些结构几乎满，系统就切换到 MP 模式。MAS 包含两个队列，一个用于内存 warp（尝试发射内存指令的 warp），一个用于计算 warp（尝试发射其他类型指令的 warp）。在每个队列内，warp 按贪婪然后最旧顺序调度。对内存依赖指令的跟踪通过增强记分板来完成，以指示输出寄存器何时基于加载被填充。当观察到工作负载平衡且内存系统未被过度订阅时，调度器以 EP 模式运行。在 EP 模式下，调度机制优先考虑内存 warp。由于内存系统未被过度订阅，预计尽早发起内存访问将提高性能。当以 MAP 模式运行时，调度器优先考虑计算 warp，以更好地将可用计算与瓶颈内存系统重叠。只允许一个内存 warp，即“所有者 warp”，发射内存指令，直到它到达依赖于待处理内存请求的操作。

除了调度机制外，Sethia 等人 [2015] 还表明，内存密集型内核的 IPC 峰值比例远低于计算密集型内核。他们说明，在内存密集型应用中，大量周期花费在 SIMT 核心的加载存储单元因过度内存访问导致的内存背压而停顿上。当 LSU 停顿时，有相当一部分时间就绪 warp 的数据在 L1 数据缓存中，但由于 LSU 被其他 warp 的内存请求积压，这些 warp 无法发射。缓存访问重新执行（CAR）机制试图通过在 LSU 流水线一侧提供一个缓冲区来存储停顿的内存指令，并允许其他指令发射到 LSU，从而解决这种行为。只有当 LSU 未停顿且没有新请求要发射时，才会处理重新执行队列中的请求，除非重新执行队列已满，在这种情况下，重新执行队列中的访问被优先处理，直到队列中有空间释放。

当重新执行队列与内存感知调度器结合时，需要特别小心，因为重新执行队列中的请求可能来自优先所有者 warp 之外的 warp。当以 MAP 模式运行时，非所有者 warp 从重新执行队列发送到 L1 的请求在 L1 未命中时会被进一步延迟。特别是，当来自非所有者 warp 的请求在 L1 中未命中时，该请求不会被转发到 L2 缓存，而是被重新插入到重新执行队列的尾部。

5.1.3 多内核调度研究

在 GPU 上支持抢占。Park 等人 [2015] 解决了在 GPU 上支持抢占式多任务的挑战。它采用更宽松的幂等性定义，以允许在线程块内刷新计算。更宽松的幂等性定义涉及检测从线程执行开始是否一直是幂等的。他们的提案 Chimera 为每个线程块动态选择三种方法之一来实现上下文切换：

- 完整的上下文保存/存储；• 等待线程块执行完毕；
- 如果线程块可以安全地从头重新启动，则直接停止，而不保存任何上下文。

每种上下文切换技术在切换延迟和对系统吞吐量的影响之间提供了不同的权衡。为了实现 Chimera，一种算法估计当前正在运行的线程块子集，这些线程块可以在对系统吞吐量影响最小的情况下停止，同时满足用户指定的上下文切换延迟目标。

5.1.4　细粒度同步感知调度

ElTantawy 和 Aamodt [2018] 探讨了在运行涉及细粒度同步的代码时，warp 调度的影响。他们使用真实的 GPU 硬件证明，当线程自旋等待锁时会产生显著的开销。他们指出，如果简单地推迟包含未能获取锁的线程的 warp 的执行，可能会阻止或减缓同一 warp 中已持有锁的其他线程的进展。他们提出了一种硬件结构，用于动态识别哪些循环涉及自旋锁，而基于堆栈的重汇聚 [ElTantawy and Aamodt，2016] 的使用使这一任务更具挑战性。该结构使用包含程序计数器最低有效位的路径历史以及谓词寄存器更新的单独历史来准确检测在锁上自旋的循环。为了减少争用并提高性能，他们提出，当被识别为执行自旋循环的 warp 在执行自旋循环的后向分支时，如果该 warp 中持有锁的任何线程已经释放了这些锁，则降低这些 warp 的优先级。他们发现，与 Lee 和 Wu [2014] 相比，这分别将性能提高了 1.5 倍，能耗降低了 1.6 倍。

细粒度工作队列。Kim 和 Batten [2014] 提出在 GPU 的每个 SIMT 核心中添加一个细粒度硬件工作列表。他们利用了这一观察：不规则 GPGPU 程序通常在以数据驱动的方式用软件实现时表现最佳，其中工作动态生成并在线程之间平衡，而不是采用拓扑方法，即启动固定数量的线程——其中许多线程不做有用工作。数据驱动方法有可能提高工作效率和负载均衡，但如果没有大量的软件优化，可能会性能不佳。本文提出了一种片上硬件工作列表，支持核心内和核心间的负载均衡。他们使用线程等待机制，并基于间隔重新平衡线程生成的任务。他们在 lonestar GPU 基准测试套件中不规则应用的各种实现上评估了他们的硬件机制，这些实现使用了拓扑和数据驱动的工作分配。

核心内硬件工作列表解决了数据驱动软件工作列表的两个主要问题：（1）当线程推出生成的工作时内存系统中的争用，以及（2）由于基于线程 ID 静态划分工作而导致的负载均衡不佳。不依赖静态划分的软件实现会在推入和拉取时遭受内存争用。静态划分工作解决了拉取争用。硬件工作列表分布在多个结构中，减少了争用。它通过在线程空闲之前动态地将生成的工作重新分配给线程来改善负载均衡。作者在 ISA 中添加了特殊指令，用于从硬件队列推入和拉取。核心中的每个 lane 被分配一个小的单端口 SRAM，用作给定 lane 使用和生成的工作 ID 的存储。

5.2 表达并行性的其他方式

本文提出了基于间隔和需求驱动（仅在推/拉请求时重新分配）的工作再分配方法，并深入评估了前者。基于间隔的方法基于简单阈值或更复杂的排序基础重新分配工作。阈值方法将工作量超过阈值的 lane 分类为贪婪（工作过多），将工作量低于阈值的 lane 分类为贫困（工作不足）。然后通过排序过程将工作从贪婪的 bank 重新分配给贫困的 bank。基于排序的技术更复杂，但实现了更好的负载均衡，因为所有贫困的 bank 也可以向其他贫困的 bank 捐赠工作。他们的技术还包括一个全局排序机制，可用于在核心之间分配工作。此外，该架构支持虚拟化硬件工作列表，使其可扩展到生成比硬件结构中可用容量更多动态工作的工作负载。

基于嵌套并行模式的编程。Lee 等人 [2014a] 提出了 GPU 上嵌套并行模式的局部性感知映射，利用了以下观察：嵌套并行计算到 GPU 线程没有普遍最优的映射。具有嵌套并行性的算法（例如 map/reduce 操作）可以根据 GPU 程序的编写方式在不同层次上向 GPU 暴露其并行性。作者利用了 GPU 上嵌套并行映射的三种推广：

• 一维映射，并行化顺序程序的外层循环；• 线程块/线程映射，将顺序程序外层循环的每次迭代分配给一个线程块，并在线程块内并行化内部模式；以及- 基于 warp 的映射，将外层循环的每次迭代分配给一个 warp，并在 warp 内并行化内部模式。这项工作提出了一个自动编译框架，它基于局部性和嵌套模式中暴露的并行度生成预测性能分数，以选择哪种映射最适合一组常见的嵌套并行模式。这些模式包括集合操作，如 map、reduce、foreach、filter 等。该框架尝试将线程映射到集合中每个元素的操作。该框架通过首先将应用程序中的每个嵌套层次分配给一个维度（x、y、z 等）来处理模式的嵌套。双重嵌套模式（即包含 reduce 的 map）具有两个维度。然后映射确定 CUDA 线程块中给定维度的线程数。在设置线程块的维度和大小之后，该框架通过使用线程跨度和分裂的概念为每个线程分配多个元素，进一步控制内核中的并行度。在二维内核（即两个级别的模式嵌套）中，如果每个维度分配 span（1），则内核中启动的每个线程仅负责操作集合中的一个元素。这种映射暴露了最大的并行度。相反，span（all）表示每个线程操作集合中的所有元素。跨度可以是（1）和（all）之间的任何数字。Span（all）用于两种特殊情况：当维度的大小在内核启动之前未知时（例如，当内部模式中操作的元素数量动态确定时）以及当模式需要同步时（例如，reduce 操作）。

由于 span（all）会严重限制可暴露的并行性，导致 GPU 利用率不足，该框架还提供了 split 的概念。split（2）表示每个线程处理给定维度中一半的元素（可以将其视为 span（all）/2）。使用 split 时，框架会启动第二个内核（称为合并内核）来聚合各个 split 的结果，产生与使用 span（all）划分内核时相同的结果。

为了选择每个维度的块大小以及每个维度的 split/span，该框架使用基于硬约束和软约束的评分算法。该算法遍历整个搜索空间——所有可能的维度、块大小和 span。搜索空间随循环嵌套层数呈指数增长。然而，指数的底数小于 100，且典型内核包含的层数少于 3 层。因此，整个空间可以在几秒钟内完成搜索。搜索会剪枝违反硬约束的配

置——即那些导致执行不正确的配置，例如块中最大线程数过高。它为软约束分配加权分数，例如确保顺序内存访问模式被分配到 x 维度以改善内存合并。

该框架还执行两项常见的 GPU 优化：预分配内存，而不是在嵌套内核中动态分配全局内存；当它确定将数据预取到共享内存对嵌套模式有益时，利用共享内存。结果表明，自动生成的代码与专家调优的代码具有竞争力。

动态并行。Wang 和 Yalamanchili [2014] 描述了在 Kepler GPU 硬件上使用 CUDA 动态并行的开销，并发现这些开销可能相当大。具体而言，他们指出了在所研究的工作负载中限制效率的几个关键问题。首先，应用程序使用了大量设备启动的内核。其次，每个内核通常只有 40 个线程（仅比一个 warp 多一点）。第三，虽然每个动态内核中执行的代码相似，但启动配置不同，导致内核配置信息的大量存储开销。最后，第四，为了实现并发，设备启动的内核被放置在单独的流中，以利用 Kepler 支持的 32 个并行硬件队列（Hyper-Q）。他们发现这些因素结合起来导致利用率非常低。

Wang 等人 [2016a] 随后提出了动态线程块启动（DTBL），它修改了 CUDA 编程模型，使设备启动的内核能够共享硬件队列资源，从而实现更大的并行性和更好的 GPU 硬件利用率。他们提案的关键是使动态启动的内核能够与运行相同代码的现有内核聚合在一起。这是通过维护聚合线程块链表来支持的，修改后的硬件在启动内核时使用该链表。他们

通过修改 GPGPU-Sim 来评估 DTBL，发现 DTBL 相比 CDP 性能提升了 1.4 倍，相比未使用 CDP 的高度调优 CUDA 版本提升了 1.2 倍。

Wang 等人 [2016b] 随后探讨了动态启动的线程块被调度到哪个 SM 的影响。他们发现，通过鼓励子线程块被调度到与父 SM 相同的 SM 上，同时考虑跨 SM 的工作负载分布，相比朴素的轮询分布机制，性能提升了 27%。

5.3　对事务内存的支持

本节总结了在 GPU 架构上支持事务内存（TM）[Harris 等人, 2010, Herlihy 和 Moss, 1993] 编程模型的各种提案。

这些提案的动机是 TM 编程模型有潜力缓解 GPU 应用中具有充足不规则并行性的线程之间管理不规则细粒度通信的挑战。在现代 GPU 上，应用程序开发者可以通过屏障粗化线程间的同步，或者尝试使用许多现代 GPU 上可用的单字原子操作来实现这些通信的细粒度锁。前一种方法可能涉及对底层算法的重大修改，而后一种方法涉及细粒度锁定的开发工作不确定性，对于实际的、市场驱动的软件开发来说风险太大（有几个例外）。在 GPU 上启用 TM 简化了同步，并提供了一个强大的编程模型，促进细粒度通信和并行工作负载的强扩展。TM 的这一承诺希望鼓励软件开发人员探索这些不规则应用的 GPU 加速。

在 GPU 上支持 TM 的独特挑战。GPU 的高度多线程特性给 TM 系统设计带来了

一系列新的挑战。GPU 上的 TM 系统不是运行数十个具有相对较大占用空间的并发事务——这是最近许多针对多核处理器的 TM 研究的重点——而是旨在扩展到数万个小型并发事务。这反映了 GPU 的高度多线程特性，数万个线程协同工作，每个线程执行一个小任务以实现共同目标。这些小型事务以字级粒度进行跟踪，从而实现比缓存块更细的冲突检测分辨率。此外，GPU 中每个核心的私有缓存由数百个 GPU 线程共享。这大大降低了利用缓存一致性协议检测冲突的好处，而这一技术是大多数为具有大型 CPU 核心的传统 CMP 设计的硬件事务内存所采用的。

5.3.1 KILO TM

Kilo TM [Fung 等人，2011] 是第一个发表的针对 GPU 架构的硬件 TM 提案。

Kilo TM 采用基于值的冲突检测 [Dalessandro et al., 2010，Olszewski et al., 2007] 来消除冲突检测对全局元数据的需求。每个事务只需读取全局内存中的现有数据进行验证——以确定其是否与另一个已提交事务存在冲突。这种验证形式利用了 GPU 内存子系统高度并行的特性，避免了冲突事务之间的任何直接交互，并以最细粒度检测冲突。

然而，基于值的冲突检测的原生实现要求事务串行提交。为了提高提交并行度，Kilo TM 融合了现有 TM 系统 [Chafi et al., 2007，Spear et al., 2008] 的思想，并通过创新解决方案对其进行了扩展。特别地，Fung 等人 [2011] 引入了最近性布隆过滤器（recency bloom filter），这是一种新颖的数据结构，利用时间和顺序的概念来压缩大量小项目集。Kilo TM 使用该结构压缩所有提交事务的写集。每个提交事务向最近性布隆过滤器查询一个近似的冲突事务集——该冲突集中的一些事务是误报。Kilo TM 利用这种近似信息来调度数百个非冲突事务并行进行验证和提交。最近性布隆过滤器的近似特性使其能够保持较小规模（约几 KB 量级），因此可以驻留在片上以实现快速访问。使用最近性布隆过滤器提升事务提交并行度是 Kilo TM 不可或缺的组成部分。

分支发散与事务内存。事务内存编程模型引入了一种新型分支发散。当一个线程束完成一个事务时，其每个活跃线程都将尝试提交。部分线程可能中止并需要重新执行其事务，而其他线程可能通过验证并提交其事务。由于这一结果在整个线程束中可能并不一致，线程束在验证后可能发生发散。Fung 等人 [2011] 提出了一种对 SIMT 硬件的简单扩展，以处理由事务中止引入的这种特定类型的分支发散。该扩展独立于 Kilo TM 的其他设计方面，但它是支持 GPU 上 TM 的必要组成部分。

图 5.2 展示了如何扩展 SIMT 栈以处理由事务中止引起的控制流发散。当线程束进入事务时（在行 B，tx_begin 处），它将两个特殊条目压入 SIMT 栈①。第一个类型为 R 的条目存储用于重启事务的信息。其活跃掩码初始为空，其 PC 字段指向 tx_begin 之后的指令。第二个类型为 T 的条目跟踪当前事务尝试。在 tx_commit 处（行 F），任何验证失败的线程在 R 条目中设置其掩码位。当线程束完成提交过程（即其活跃线程要么已提交要么已中止）时，弹出 T 条目②。然后，将使用 R 条目中的活跃掩码和 PC 将新的 T 条目压入栈，以重启已中止的线程。接着，清除 R 条目中的活跃掩码③。如

果 R 条目中的活跃掩码为空，则弹出 T 和 R 条目，露出原始的 N 条目⑤。然后修改其 PC 以指向 tx_commit 之后的指令，线程束恢复正常执行。事务内线程束的分支发散以与非事务性发散相同的方式处理④。

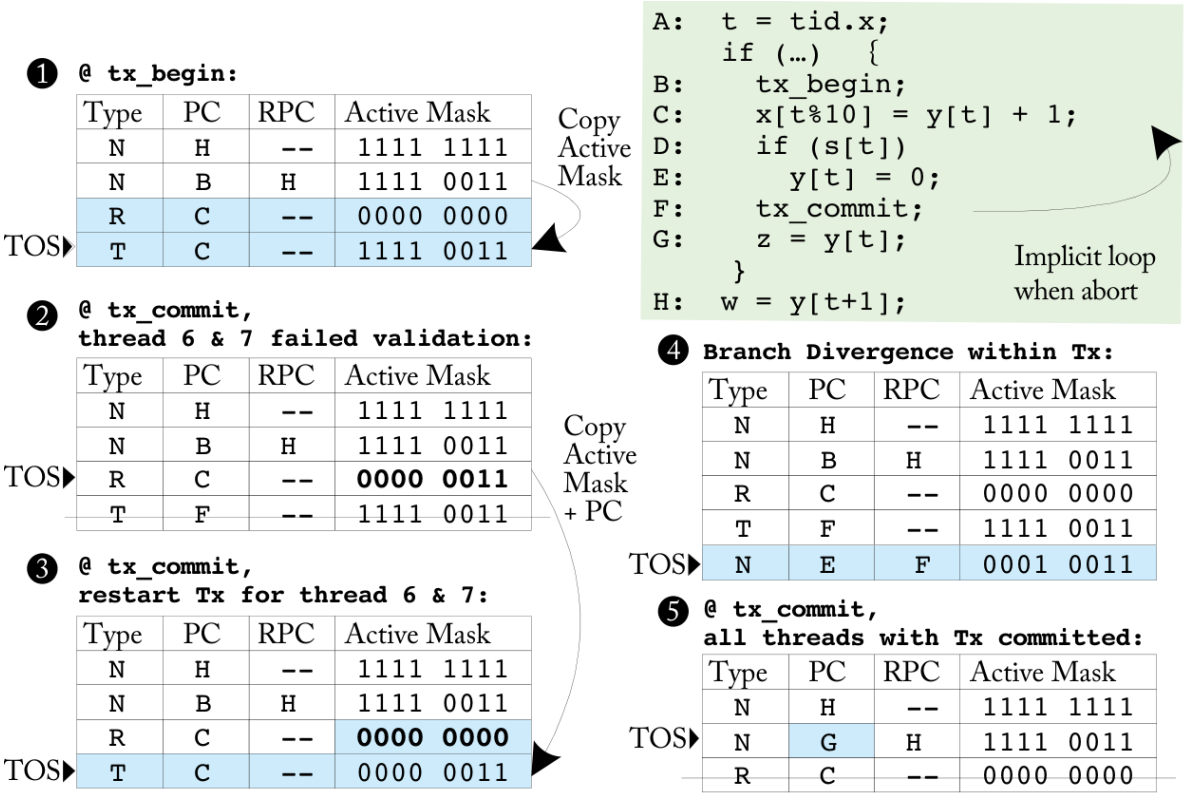


图 5.2　由事务中止引起的控制流发散的 SIMT 栈扩展示例：线程 6 和 7 验证失败后重新执行。

5.3.2　线程束 TM 与时间冲突检测

在其后续论文中，Fung 和 Aamodt[2013] 提出了两种不同的增强措施，以提高 Kilo TM 的性能和能效：线程束级事务管理（WarpTM）和时间冲突检测（TCD）。

线程束级事务管理利用 GPU 编程模型中的线程层次结构——线程束内线程之间的空间局部性——来提高 Kilo TM 的效率。特别地，WarpTM 分摊了 Kilo TM 的控制开销并提升了 GPU 内存子系统的效用。这些优化只有在能够高效解决线程束内冲突时才可能实现，因此低开销的线程束内冲突解决机制对于保持 WarpTM 的收益至关重要。为此，Fung 和 Aamodt[2013] 提出了一种两阶段并行线程束内冲突解决机制，能够高效并行地解决线程束内的冲突。在所有线程束内冲突解决后，Kilo TM 可以将同一线程束中多个事务的验证和提交的标量内存访问合并为更宽的访问。这种称为验证和提交合并的优化是使 Kilo TM 能够充分利用为向量宽访问优化的宽 GPU 内存系统的关键。

时间冲突检测是一种低开销机制，它使用一组全局同步的片上定时器来检测只读事务的冲突。一旦初始化，这些片上定时器中的每一个都在其微架构模块中本地运行，不与其他定时器通信。这种无需通信的隐式同步将 TCD 与各种软件 TM 系统中使用的现有基于时间戳的冲突检测 [Dalessandro et al., 2010，Spear et al., 2006，Xu et al., 2014]

区分开来。TCD 使用从这些定时器捕获的时间戳来推断事务的内存读取相对于其他事务更新的顺序。Kilo TM 结合 TCD 来检测无冲突的只读事务，这些事务可以直接提交而无需基于值的冲突检测。这样做，它显著降低了这些事务的内存带宽开销，而对于使用事务进行数据结构遍历的 GPU-TM 应用来说，这些事务可能频繁出现。

5.4 异构系统

异构系统中的并发管理。Kayiran 等人 [2014] 提出了一种并发限制方案来限制 GPU 多线程，从而减少多道程序 CPU/GPU 系统中的内存和网络争用。在异构系统上，来自 GPU 的干扰可能导致同时执行的 CPU 应用程序性能显著下降。他们提出的线程级并行（TLP）限制方案观察共享 CPU/GPU 内存控制器和互连网络中的拥塞指标，以估计每个 GPU 核心上应主动调度的 GPU 线程束数量。他们提出了两种方案：一种仅专注于提高 CPU 性能，另一种则通过平衡因受限多线程而导致的 GPU 性能下降与 CPU 干扰，寻求优化整体系统吞吐量（包括 CPU 和 GPU）。作者在 GPU 核心与 CPU 核心比例为 2：1 的瓦片式异构架构上评估了线程束调度对性能的影响，其依据是 NVIDIA GPU SM 的面积大约为采用相同工艺技术的现代乱序 Intel 芯片的一半。基线配置完全共享 CPU 和 GPU 之间的网络带宽和内存控制器，以最大化资源利用率。使用该方案，作者观察到限制 GPU TLP 可能对 GPU 性能产生正面或负面影响，但绝不会损害 CPU 性能。

为了提高 CPU 性能，作者引入了一种以 CPU 为中心的并发管理技术，该技术监控全局内存控制器中的停顿。该技术分别统计因内存控制器输入队列满而停顿的内存请求数量，以及因从 MC 到核心的回复网络满而停顿的内存请求数量。这些指标在每个内存控制器本地监控，并聚

合到一个集中式单元中，该单元将信息发送给 GPU 核心。启发式驱动方案为这些值设置高阈值和低阈值。如果两个请求停顿计数之和较低（基于阈值），则增加 GPU 上主动调度的线程束数量。如果两个计数之和较高，则减少活跃线程束数量，希望由此减少 GPU 内存流量从而提高 CPU 性能。

一种试图最大化整体系统吞吐量的更平衡的技术扩展了以 CPU 为中心的方法，以考虑线程束限制对 GPU 性能的影响。这种平衡技术监控在并发重平衡间隔（其工作中为 1，024 个周期）内 GPU 无法发射的周期数。当前多线程限制级别下 GPU 停顿的移动平均值存储在每个 GPU 核心上，并用于确定是否应增加或减少多线程级别。平衡技术分两个阶段调节 GPU 的 TLP。在第一阶段，其操作与以 CPU 为中心的解决方案相同，不考虑 GPU 停顿，仅根据内存争用来限制 GPU TLP。在第二阶段（当 GPU TLP 限制开始导致 GPU 性能下降，因为 GPU 的延迟容忍度已降低时开始），如果系统预测限制 GPU 并发会损害 GPU 性能，则停止限制。该预测通过查找目标多线程级别下 GPU 停顿的移动平均值来进行，该值是从该级别下的早期执行中记录的。如果目标多线程级别下观察到的 GPU 停顿与当前多线程级别之间的差异超过阈值 k 值，则不降低

TLP 级别。该 k 值可由用户设置，是指定 GPU 性能优先级的代理。

异构系统一致性。Power 等人 [2013a] 提出了一种硬件机制，以在集成系统上高效支持 CPU 和 GPU 之间的缓存一致性。他们发现，随着 GPU 产生的内存流量增加，目录带宽成为显著瓶颈。他们采用粗粒度区域一致性 [Cantin 等人，2005] 来减少传统基于缓存块的一致性目录中过多的目录流量。一旦获得粗粒度区域的权限，大多数请求将无需访问目录，一致性流量可以卸载到非一致性直接访问总线，而不是低带宽的一致性互连网络。

面向 CPU-GPU 架构的异构 TLP 感知缓存管理。Lee 和 Kim[2012] 评估了在异构环境中管理 CPU 核心和 GPU 核心之间共享末级缓存（LLC）的效果。他们证明，虽然缓存命中率是 CPU 工作负载的关键性能指标，但许多 GPU 工作负载对缓存命中率不敏感，因为内存延迟可以通过线程级并行来隐藏。为了确定 GPU 应用是否对缓存敏感，他们开发了一种每核心性能采样技术，其中一些核心绕过共享 LLC，一些核心插入到最近使用位置。基于这些核心的相对性能，他们可以为其余 GPU 核心设置绕过策略：如果性能提高则插入 LLC，如果不敏感则绕过。

其次，他们观察到以前以 CPU 为中心的缓存管理偏向于访问更频繁的核心。GPU 核心在 LLC 上产生的流量是 CPU 核心的五到十倍。这增加了缓存容量对 GPU 的偏向，降低了 CPU 应用的性能。他们提出扩展先前提出的基于效用的缓存分区工作 [Qureshi 和 Patt，2006]，以考虑 LLC 访问的相对比率。当 GPU 核心对缓存敏感时，CPU 核心的访问缓存路分配会增加，超出基于效用的缓存分区所提供的范围，以考虑 CPU 和 GPU 之间访问幅度和延迟敏感性的差异。

参考文献

Die photo analysis. http://vlsiarch.eecs.harvard.edu/accelerators/die-photo-analysis 1

https://en.wikipedia.org/wiki/GDDR5_SDRAM 76

Top500.org 2

Tor M. Aamodt, Wilson W. L. Fung, Inderpreet Singh, Ahmed El-Shafiey, Jimmy Kwa, Tayler Hetherington, Ayub Gubran, Andrew Boktor, Tim Rogers, Ali Bakhoda, and Hadi Jooybar. GPGPU-Sim 3.x Manual. 16, 38

M. Abdel-Majeed and M. Annavaram. Warped register file: A power efficient register file for GPGPUs. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), February 2013. DOI: 10.1109/hpca.2013.6522337. 64

M. Abdel-Majeed, A. Shafaei, H. Jeon, M. Pedram, and M. Annavaram. Pilot register file: Energy efficient partitioned register file for GPUs. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), February 2017. DOI: 10.1109/hpca.2017.47. 65

Dominic Acocella and Mark R. Goudy. United States Patent #7,750,915: Concurrent Access of Data Elements Stored Across Multiple Banks in a Shared Memory Resource (Assignee: NVIDIA Corp.), July 2010. 68

Neha Agarwal, David Nellans, Mark Stephenson, Mike O' Connor, and Stephen W. Keckler. Page placement strategies for GPUs within heterogeneous memory systems. In Proc. of the ACM Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2015. DOI: 10.1145/2694344.2694381. 82

Jade Alglave, Mark Batty, Alastair F. Donaldson, Ganesh Gopalakrishnan, Jeroen Ketema, Daniel Poetzl, Tyler Sorensen, and John Wickerson. GPU concurrency: Weak behaviours and programming assumptions. In Proc. of the ACM Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 577–591, 2015. DOI: 10.1145/2694344.2694391. 72

John R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. Conversion of control dependence to data dependence. In Proc. of the ACM Symposium on Principles and Practices of Parallel Programming (PPoPP), pages 177–189, 1983. DOI: 10.1145/567067.567085. 16

Robert Alverson, David Callahan, Daniel Cummings, Brian Koblenz, Allan Porterfield, and Burton Smith. The tera computer system. In Proc. of the ACM International Conference on Supercomputing (ICS), pages 1–6, 1990. DOI: 10.1145/77726.255132. 16

R700-Family Instruction Set Architecture. AMD, March 2009. 49

AMD Southern Islands Series Instruction Set Architecture. AMD, 1.1 ed., December 2012. 13, 17, 19, 20, 26, 54, 58, 74

A. Arunkumar, S. Y. Lee, and C. J. Wu. ID-cache: Instruction and memory divergence based cache management for GPUs. In Proc. of the IEEE International Symposium on Workload Characterization (IISWC), 2016. DOI: 10.1109/iiswc.2016.7581276. 79

Akhil Arunkumar, Evgeny Bolotin, Benjamin Cho, Ugljesa Milic, Eiman Ebrahimi, Oreste Villa, Aamer Jaleel, Carole-Jean Wu, and David Nellans. MCM-GPU: Multi-chipmodule GPUs for continued performance scalability. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 320–332, 2017. DOI: 10.1145/3079856.3080231. 84

Krste Asanovic, Stephen W. Keckler, Yunsup Lee, Ronny Krashinsky, and Vinod Grover. Convergence and scalarization for data-parallel architectures. In Proc. of the ACM/IEEE International Symposium on Code Generation and Optimization (CGO), 2013. DOI: 10.1109/cgo.2013.6494995. 54, 56, 58

Rachata Ausavarungnirun, Saugata Ghose, Onur Kayran, Gabriel H. Loh, Chita R. Das, Mahmut T. Kandemir, and Onur Mutlu. Exploiting inter-warp heterogeneity to improve GPGPU performance. In Proc. of the ACM/IEEE International Conference on Parallel Architecture and Compilation Techniques (PACT), 2015. DOI: 10.1109/pact.2015.38. 80

Ali Bakhoda, George L. Yuan, Wilson W. L. Fung, Henry Wong, and Tor M. Aamodt. Analyzing CUDA workloads using a detailed GPU simulator. In Proc. of the IEEE Symposium of Performance and Analysis of Systems and Software, (ISPASS'09), pages 163–174, 2009. DOI: 10.1109/ispass.2009.4919648. 6, 14, 78

Ali Bakhoda, John Kim, and Tor M. Aamodt. Throughput-effective on-chip networks for manycore accelerators. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 421–432, 2010. DOI: 10.1109/micro.2010.50. 77

Ali Bakhoda, John Kim, and Tor M. Aamodt. Designing on-chip networks for throughput accelerators. ACM Transactions on Architecture and Code Optimization (TACO), 10(3):21, 2013. DOI: 10.1145/2509420.2512429. 77

Markus Billeter, Ola Olsson, and Ulf Assarsson. Efficient stream compaction on wide SIMD many-core architectures. In Proc. of the ACM Conference on High Performance Graphics, pages 159–166, 2009. DOI: 10.1145/1572769.1572795. 45

Nicolas Brunie, Sylvain Collange, and Gregory Diamos. Simultaneous branch and warp interweaving for sustained GPU performance. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 49–60, 2012. DOI: 10.1109/isca.2012.6237005. 44, 53

Ian Buck, Tim Foley, Daniel Horn, Jeremy Sugerman, Kayvon Fatahalian, Mike Houston, and Pat Hanrahan. Brook for GPUs: Stream computing on graphics hardware. In Proc. of the ACM International Conference on Computer Graphics and Interactive Techniques (SIGGRAPH), pages 777–786, 2004. DOI: 10.1145/1186562.1015800. 6

Brian Cabral. what is "SASS" short for? https://stackoverflow.com/questions/9798258/what-is-sass-short-for, November 2016. 14

J. F. Cantin, M. H. Lipasti, and J. E. Smith. Improving multiprocessor performance with coarsegrain coherence tracking. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), June 2005. DOI: 10.1109/isca.2005.31. 100

Edwin Catmull. A subdivision algorithm for computer display of curved surfaces. Technical Report, DTIC Document, 1974. 72

Hassan Chafi, Jared Casper, Brian D. Carlstrom, Austen Mcdonald, Chi Cao, Minh Woongki Baek, Christos Kozyrakis, and Kunle Olukotun. A scalable, non-blocking approach to transactional memory. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 97–108,

2007. DOI: 10.1109/hpca.2007.346189. 97

Guoyang Chen, Bo Wu, Dong Li, and Xipeng Shen. PORPLE: An extensible optimizer for portable data placement on GPU. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2014a. DOI: 10.1109/micro.2014.20. 83

Xi E. Chen and Tor M. Aamodt. A first-order fine-grained multithreaded throughput model. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 329–340, 2009. DOI: 10.1109/hpca.2009.4798270. 78

Xuhao Chen, Li-Wen Chang, Christopher I. Rodrigues, Jie Lv, Zhiying Wang, and Wen- Mei Hwu. Adaptive cache management for energy-efficient GPU computing. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2014b. DOI: 10.1109/micro.2014.11. 81, 82

Sylvain Collange, David Defour, and Yao Zhang. Dynamic detection of uniform and affine vectors in GPGPU computations. In Proc. of the European Conference on Parallel Processing (Euro-Par), 2010. DOI: 10.1007/978-3-642-14122-5_8. 59

Brett W. Coon and John Erik Lindholm. United States Patent #7,353,369: System and Method for Managing Divergent Threads in a SIMD Architecture (Assignee NVIDIA Corp.), April 2008. 26, 49, 50

Brett W. Coon, Peter C. Mills, Stuart F. Oberman, and Ming Y. Siu. United States Patent #7,434,032: Tracking Register Usage During Multithreaded Processing Using a Scorebard having Separate Memory Regions and Storing Sequential Register Size Indicators (Assignee NVIDIA Corp.), October 2008. 34

Brett W. Coon, John Erik Lindholm, Gary Tarolli, Svetoslav D. Tzvetkov, John R. Nickolls, and Ming Y. Siu. United States Patent #7,634,621: Register File Allocation (Assignee NVIDIA Corp.), December 2009. 35

Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. ACM Transactions on Programming Languages and Systems (TOPLAS), 13(4):451–490, 1991. DOI: 10.1145/115372.115320. 16

Luke Dalessandro, Michael F. Spear, and Michael L. Scott. NOrec: Streamlining STM by abolishing ownership records. In Proc. of the ACM Symposium on Principles and Practices of Parallel Programming (PPoPP), pages 67–78, 2010. DOI: 10.1145/1693453.1693464. 96, 99

R. H. Dennard, F. H. Gaensslen, and K. Mai. Design of ion-implanted MOSFET's with very small physical dimensions. IEEE Journal of Solid-State Circuits, October 1974. DOI: 10.1109/jssc.1974.1050511. 1

Gregory Diamos, Benjamin Ashbaugh, Subramaniam Maiyuran, Andrew Kerr, Haicheng Wu, and Sudhakar Yalamanchili. SIMD re-convergence at thread frontiers. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 477–488, 2011. DOI: 10.1145/2155620.2155676. 26, 51, 54

Gregory Frederick Diamos, Richard Craig Johnson, Vinod Grover, Olivier Giroux, Jack H. Choquette, Michael Alan Fetterman, Ajay S. Tirumala, Peter Nelson, and Ronny Meir Krashinsky. Execution of divergent threads using a convergence barrier, July 13, 2015. 27, 28, 29, 30

Roger Eckert. United States Patent #7,376,803: Page stream sorter for DRAM systems (Assignee: NVIDIA Corp.), May 2008. 73

Roger Eckert. United States Patent #9,195,618: Method and System for Scheduling Memory Requests (Assignee: NVIDIA Corp.), November 2015. 73

John H. Edmondson and James M. Van Dyke. United States Patent #7872657: Memory Addressing Scheme using Partition Strides, January 2011. 75

John H. Edmondson et al. United States Patent #8,464,001: Cache and Associated Method with Frame Buffer Managed Dirty Data Pull and High-Priority Clean Mechanism, June 2013. 75, 76

Ahmed ElTantaway, Jessica Wenjie Ma, Mike O' Connor, and Tor M. Aamodt. A scalable multi-path microarchitecture for efficient GPU control flow. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2014. DOI: 10.1109/hpca.2014.6835936. 26, 28, 30, 31, 49, 52

Ahmed ElTantawy and Tor M. Aamodt. MIMD synchronization on SIMT architectures. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 1–14, 2016. DOI: 10.1109/micro.2016.7783714. 26, 27, 30, 32, 93

Ahmed ElTantawy and Tor M. Aamodt. Warp scheduling for fine-grained synchronization. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 375–388, 2018. DOI: 10.1109/hpca.2018.00040. 93

Alexander L. Minken et al., United States Patent #7,649,538: Reconfigurable High Performance Texture Pipeline with Advanced Filtering (Assignee: NVIDIA Corp.), January 2010. 72

Wilson W. L. Fung. GPU Computing Architecture for Irregular Parallelism. Ph.D. thesis, University of British Columbia, January 2015. DOI: 10.14288/1.0167110. 41

Wilson W. L. Fung and Tor M. Aamodt. Thread block compaction for efficient SIMT control flow. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 25–36, 2011. DOI: 10.1109/hpca.2011.5749714. 26, 28, 42, 43, 50, 91

Wilson W. L. Fung and Tor M. Aamodt. Energy efficient GPU transactional memory via spacetime optimizations. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 408–420, 2013. DOI: 10.1145/2540708.2540743. 98

Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. Dynamic warp formation and scheduling for efficient GPU control flow. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 407–420, 2007. DOI: 10.1109/micro.2007.4408272. 14, 23, 25, 42, 44, 49, 91

Wilson W. L. Fung, Inderpreet Singh, Andrew Brownsword, and Tor M. Aamodt. Hardware transactional memory for GPU architectures. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 296–307, 2011. DOI: 10.1145/2155620.2155655. 96, 97

Wilson Fung et al. Dynamic warp formation: Efficient MIMD control flow on SIMD graphics hardware. ACM Transactions on Architecture and Code Optimization (TACO), 6(2):7:1–7:37, 2009. DOI: 10.1145/1543753.1543756. 42, 49

M. Gebhart, S. W. Keckler, and W. J. Dally. A compile-time managed multi-level register file hierarchy. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), December 2011a. DOI: 10.1145/2155620.2155675. 63

Mark Gebhart, Daniel R. Johnson, David Tarjan, Stephen W. Keckler, William J. Dally, Erik Lindholm, and Kevin Skadron. Energy-efficient mechanisms for managing thread context in throughput processors. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2011b. DOI: 10.1145/2000064.2000093. 63

Mark Gebhart, Daniel R. Johnson, David Tarjan, Stephen W. Keckler, William J. Dally, Erik Lindholm, and Kevin Skadron. Energy-efficient mechanisms for managing thread context in throughput processors. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 235–246, 2011c. DOI: 10.1145/2000064.2000093. 88

Isaac Gelado, John E. Stone, Javier Cabezas, Sanjay Patel, Nacho Navarro, and Wen-mei W. Hwu. An asymmetric distributed shared memory model for heterogeneous parallel systems. In Proc. of the ACM Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 347–358, 2010. DOI: 10.1145/1736020.1736059. 3

Syed Zohaib Gilani, Nam Sung Kim, and Michael J. Schulte. Power-efficient computing for compute-intensive GPGPU applications. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2013. DOI: 10.1109/hpca.2013.6522330. 59, 61

David B. Glasco et al. United States Patent #8,135,926: Cache-Based Control of Atomic Operations in Conjunction with an External Alu Block, March 2012. 76

David B. Glasco et al. United States Patent #8,539,130: Virtual Channels for Effective Packet Transfer, September 2013. 75

Scott Gray. Assembler for NVIDIA Maxwell architecture. https://github.com/NervanaSyst ems/-maxas 16, 40

Zvika Guz, Evgeny Bolotin, Idit Keidar, Avinoam Kolodny, Avi Mendelson, and Uri C. Weiser. Many-core vs. many-thread machines: Stay away from the valley. IEEE Computer Architecture Letters, 8(1):25–28, 2009. DOI: 10.1109/l-ca.2009.4. 5

Ziyad S. Hakura and Anoop Gupta. The design and analysis of a cache architecture for texture mapping. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 108–120, 1997. DOI: 10.1145/264107.264152. 73

Rehan Hameed, Wajahat Qadeer, Megan Wachs, Omid Azizi, Alex Solomatnikov, Benjamin C. Lee, Stephen Richardson, Christos Kozyrakis, and Mark Horowitz. Understanding sources of inefficiency in general-purpose chips. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 37–47, 2010. DOI: 10.1145/1815961.1815968. 1

Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. EIE: efficient inference engine on compressed deep neural network. In

Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 243–254, 2016. DOI: 10.1109/isca.2016.30. 6

Mark Harris. An Easy Introduction to CUDA C and CCC. https://devblogs.nvidia.com/para llelforall/easy-introduction-cuda-c-and-c/, 2012. 11

Tim Harris, James Larus, and Ravi Rajwar. Transactional Memory, 2nd ed. Morgan & Claypool, 2010. DOI: 10.1201/b11417-16. 96

Steven J. Heinrich et al. United States Patent #9,595,075: Load/Store Operations in Texture Hardware (Assignee: NVIDIA Corp.), March 2017. 68, 73

John Hennessy and David Patterson. Computer Architecture—A Quantitative Approach, 5th ed. Morgan Kaufmann, 2011. 1, 10, 71, 72, 78

Maurice Herlihy and J. Eliot B. Moss. Transactional memory: Architectural support for lockfree data

structures. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 289–300, 1993. DOI: 10.1109/isca.1993.698569. 96

Jared Hoberock, Victor Lu, Yuntao Jia, and John C. Hart. Stream compaction for deferred shading. In Proc. of the ACM Conference on High Performance Graphics, pages 173–180, 2009. DOI: 10.1145/1572769.1572797. 45

H. Peter Hofstee. Power efficient processor architecture and the cell processor. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 258–262, 2005. DOI: 10.1109/hpca.2005.26. 68

Mark Horowitz, Elad Alon, Dinesh Patil, Samuel Naffziger, Rajesh Kumar, and Kerry Bernstein. Scaling, power, and the future of CMOS. In IEEE International Electron Devices Meeting, 2005. DOI: 10.1109/iedm.2005.1609253. 5

Homan Igehy, Matthew Eldridge, and Kekoa Proudfoot. Prefetching in a texture cache architecture. In Proc. of the ACM SIGGRAPH/EUROGRAPHICS Workshop on Graphics hardware, pages 133–ff, 1998. DOI: 10.1145/285305.285321. 72, 73, 74

Hyeran Jeon, Gokul Subramanian Ravi, Nam Sung Kim, and Murali Annavaram. GPU register file virtualization. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 420–432, 2015. DOI: 10.1145/2830772.2830784. 64

W. Jia, K. A. Shaw, and M. Martonosi. MRPB: Memory request prioritization for massively parallel processors. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2014. DOI: 10.1109/hpca.2014.6835938. 78, 79

Wenhao Jia, Kelly A Shaw, and Margaret Martonosi. Characterizing and improving the use of demand-fetched caches in GPUs. In Proc. of the ACM International Conference on Supercomputing (ICS), pages 15–24, 2012. DOI: 10.1145/2304576.2304582. 78

Adwait Jog, Onur Kayiran, Nachiappan Chidambaram Nachiappan, Asit K. Mishra, Mahmut T. Kandemir, Onur Mutlu, Ravishankar Iyer, and Chita R. Das. OWL: Cooperative thread array aware scheduling techniques for improving GPGPU performance. In Proc. of the ACM Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2013a. DOI: 10.1145/2451116.2451158. 90

Adwait Jog, Onur Kayiran, Asit K. Mishra, Mahmut T. Kandemir, Onur Mutlu, Ravishankar Iyer, and Chita R. Das. Orchestrated scheduling and prefetching for GPGPUs. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2013b. DOI: 10.1145/2508148.2485951. 90

Norman P. Jouppi, CliffYoung, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2017. DOI: 10.1145/3079856.3080246. 2

David R. Kaeli, Perhaad Mistry, Dana Schaa, and Dong Ping Zhang. Heterogeneous Computing with OpenCL 2.0. Morgan Kaufmann, 2015. 10

Ujval J. Kapasi et al. Efficient conditional operations for data-parallel architectures. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 159–170, 2000. DOI: 10.1109/micro.2000.898067. 45

O. Kayiran, N. C. Nachiappan, A. Jog, R. Ausavarungnirun, M. T. Kandemir, G. H. Loh, O. Mutlu, and C. R. Das. Managing GPU concurrency in heterogeneous architectures. In Proc. of the ACM/IEEE International

Symposium on Microarchitecture (MICRO), 2014. DOI: 10.1109/micro.2014.62. 99

Onur Kayiran, Adwait Jog, Mahmut T. Kandemir, and Chita R. Das. Neither more nor less: Optimizing thread-level parallelism for GPGPUs. In Proc. of the ACM/IEEE International Conference on Parallel Architecture and Compilation Techniques (PACT), 2013. 86

S. W. Keckler, W. J. Dally, B. Khailany, M. Garland, and D. Glasco. GPUs and the future of parallel computing. Micro, IEEE, 31(5):7–17, September 2011. DOI: 10.1109/mm.2011.89. 53, 58

Shane Keil and John H. Edmondson. United States Patent #8,195,858: Managing Conflicts on Shared L2 Bus, June 2012. 77

Shane Keil et al. United States Patent #8,307,165: Sorting Requests to the Dram for High Page Locality, November 2012. 77

Farzad Khorasani, Rajiv Gupta, and Laxmi N. Bhuyan. Efficient warp execution in presence of divergence with collaborative context collection. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2015. DOI: 10.1145/2830772.2830796. 45

J. Y. Kim and C. Batten. Accelerating irregular algorithms on GPGPUs using fine-grain hardware worklists. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MI- CRO), 2014. DOI: 10.1109/micro.2014.24. 93

Ji Kim, Christopher Torng, Shreesha Srinath, Derek Lockhart, and Christopher Batten. Microarchitectural mechanisms to exploit value structure in SIMT architectures. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2013. DOI: 10.1145/2508148.2485934. 57, 58, 59, 60, 61, 62

Sangman Kim, Seonggu Huh, Xinya Zhang, Yige Hu, Amir Wated, Emmett Witchel, and Mark Silberstein. GPUnet: Networking abstractions for GPU programs. In Proc. of the USENIX Symposium on Operating Systems Design and Implementation, pages 6–8, 2014. DOI: 10.1145/2963098. 2

David B. Kirk and W. Hwu Wen-Mei. Programming Massively Parallel Processors: A Hands-on Approach. Morgan Kaufmann, 2016. DOI: 10.1016/c2011-0-04129-7. 9

John Kloosterman, Jonathan Beaumont, D. Anoushe Jamshidi, Jonathan Bailey, Trevor Mudge, and Scott Mahlke. Regless: Just-in-time operand staging for GPUs. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 151–164, 2017. DOI: 10.1145/3123939.3123974. 65

R. Krashinsky, C. Batten, M. Hampton, S. Gerding, B. Pharris, J. Casper, and K. Asanovic. The vector-thread architecture. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 52–63, June 2004. DOI: 10.1109/isca.2004.1310763. 52

Ronny M. Krashinsky. United States Patent Application #20130042090 A1: Temporal SIMT Execution Optimization, August 2011. 53, 58

David Kroft. Lockup-free instruction fetch/prefetch cache organization. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 81–87, 1981. DOI: 10.1145/285930.285979. 33, 71

Jens Krüger and Rüdiger Westermann. Linear algebra operators for GPU implementation of numerical algorithms. In ACM Transactions on Graphics (TOG), V. 22, pages 908–916, 2003. DOI: 10.1145/882262.882363. 6

Junjie Lai and André Seznec. Performance upper bound analysis and optimization of SGEMM on Fermi and

Kepler GPUs. In Proc. of the ACM/IEEE International Symposium on Code Generation and Optimization (CGO), pages 1–10, 2013. DOI: 10.1109/cgo.2013.6494986. 16

Nagesh B. Lakshminarayana and Hyesoon Kim. Effect of instruction fetch and memory scheduling on GPU performance. In Workshop on Language, Compiler, and Architecture Support for GPGPU, 2010. 88

Ahmad Lashgar, Ebad Salehi, and Amirali Baniasadi. A case study in reverse engineering GPG- PUs: Outstanding memory handling resources. ACM SIGARCH Computer Architecture News, 43(4):15–21, 2016. DOI: 10.1145/2927964.2927968. 34

C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh. Basic linear algebra subprograms for fortran usage. ACM Transactions on Mathematical Software, 5(3):308–323, September 1979. DOI: 10.1145/355841.355848. 10

HyoukJoong Lee, Kevin J. Brown, Arvind K. Sujeeth, Tiark Rompf, and Kunle Olukotun. Locality-aware mapping of nested parallel patterns on GPUs. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2014a. DOI: 10.1109/micro.2014.23. 94

J. Lee and H. Kim. TAP: A TLP-aware cache management policy for a CPU-GPU heterogeneous architecture. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2012. DOI: 10.1109/hpca.2012.6168947. 100

S. Y. Lee and C. J. Wu. Ctrl-C: Instruction-aware control loop based adaptive cache bypassing for GPUs. In Proc. of the IEEE International Conference on Computer Design (ICCD), pages 133–140, 2016. DOI: 10.1109/iccd.2016.7753271. 80

Sangpil Lee, Keunsoo Kim, Gunjae Koo, Hyeran Jeon, Won Woo Ro, and Murali Annavaram. Warped-compression: Enabling power efficient GPUs through register compression. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2015. DOI: 10.1145/2749469.2750417. 59, 60, 61

Shin-Ying Lee and Carole-Jean Wu. CAWS: Criticality-aware warp scheduling for GPGPU workloads. In Proc. of the ACM/IEEE International Conference on Parallel Architecture and Compilation Techniques (PACT), 2014. DOI: 10.1145/2628071.2628107. 93

Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, et al. Debunking the 100X GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 451–460, 2010. DOI: 10.1145/1815961.1816021. 2

Yunsup Lee, Rimas Avizienis, Alex Bishara, Richard Xia, Derek Lockhart, Christopher Batten, and Krste Asanović. Exploring the tradeoffs between programmability and efficiency in data-parallel accelerators. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 129–140, 2011. DOI: 10.1145/2000064.2000080. 53

Yunsup Lee, Vinod Grover, Ronny Krashinsky, Mark Stephenson, Stephen W. Keckler, and Krste Asanović. Exploring the design space of SPMD divergence management on data-

parallel architectures. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2014b. DOI: 10.1109/micro.2014.48. 55, 56

Jingwen Leng, Tayler Hetherington, Ahmed ElTantawy, Syed Gilani, Nam Sung Kim, Tor M. Aamodt, and Vijay Janapa Reddi. GPUWattch: Enabling energy optimizations in GPG- PUs. In Proc. of the

ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 487–498, 2013. DOI: 10.1145/2508148.2485964. 6

Adam Levinthal and Thomas Porter. Chap—A SIMD graphics processor. In Proc. of the ACM International Conference on Computer Graphics and Interactive Techniques (SIGGRAPH), pages 77–82, 1984. DOI: 10.1145/800031.808581. 50

Dong Li, Minsoo Rhu, Daniel R. Johnson, Mike O'Connor, Mattan Erez, Doug Burger, Donald S. Fussell, and Stephen W. Redder. Priority-based cache allocation in throughput processors. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2015. DOI: 10.1109/hpca.2015.7056024. 82

E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym. NVIDIA Tesla: A unified graphics and computing architecture. Micro, IEEE, 28(2):39–55, March–April 2008a. DOI: 10.1109/mm.2008.31. 9

Erik Lindholm, Mark J. Kilgard, and Henry Moreton. A user-programmable vertex engine. In Proc. of the ACM International Conference on Computer Graphics and Interactive Techniques (SIGGRAPH), pages 149–158, 2001. DOI: 10.1145/383259.383274. 6, 21

John Erik Lindholm, Ming Y. Siu, Simon S. Moy, Samuel Liu, and John R. Nickolls. United States Patent #7,339,592: Simulating Multiported Memories Using Lower Port Count Memories (Assignee NVIDIA Corp.), March 2008b. 35, 38

Erik Lindholm et al. United States Patent #9,189,242: Credit-Based Streaming Multiprocessor Warp Scheduling (Assignee NVIDIA Corp.), November 2015. 33, 41

John S. Liptay. Structural aspects of the system/360 model 85, II: The cache. IBM Systems Journal, 7(1):15–21, 1968. DOI: 10.1147/sj.71.0015. 70

Z. Liu, S. Gilani, M. Annavaram, and N. S. Kim. G-Scalar: Cost-effective generalized scalar execution architecture for power-efficient GPUs. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2017. DOI: 10.1109/hpca.2017.51. 60, 61, 62

Samuel Lui, John Erik Lindholm, Ming Y. Siu, Brett W. Coon, and Stuart F. Oberman. United States Patent Application 11/555,649: Operand Collector Architecture (Assignee NVIDIA Corp.), May 2008. 35

Michael D. McCool, Arch D. Robison, and James Reinders. Structured Parallel Programming: Patterns for Efficient Computation. Elsevier, 2012. 10

Jiayuan Meng, David Tarjan, and Kevin Skadron. Dynamic warp subdivision for integrated branch and memory divergence tolerance. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 235–246, 2010. DOI: 10.1145/1815961.1815992. 30, 48, 90

Alexander L. Minken and Oren Rubinstein. United States Patent #6,629,188: Circuit and Method for Prefetching Data for a Texture Cache (Assignee: NVIDIA Corp.), September 2003. 72

Alexander L. Minkin et al. United States Patent #8,266,383: Cache Miss Processing Usinga Defer/Replay Mechanism (Assignee: NVIDIA Corp.), September 2012. 68, 69, 71

Alexander L. Minkin et al. United States Patent #8,595,425: Configurable Cache for Multiple Clients (Assignee: NVIDIA Corp.), November 2013. 68

Michael Mishkin, Nam Sung Kim, and Mikko Lipasti. Write-after-read hazard prevention in GPGPUSIM. In Workshop on Deplicating, Deconstructing, and Debunking (WDDD), June 2016. 39, 40

John Montrym and Henry Moreton. The geforce 6800. IEEE Micro, 25(2):41–51, 2005. DOI: 10.1109/mm.2005.37. 1

Veynu Narasiman, Michael Shebanow, Chang Joo Lee, Rustam Miftakhutdinov, Onur Mutlu, and Yale N. Patt. Improving GPU performance via large warps and two-level warp scheduling. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 308–317, 2011. DOI: 10.1145/2155620.2155656. 33, 38, 43, 88, 91

John R. Nickolls and Jochen Reusch. Autonomous SIMD flexibility in the MP-1 and MP-2. In Proc. of the ACM Symposium on Parallel Algorithms and Architectures (SPAA), pages 98–99, 1993. DOI: 10.1145/165231.165244. 9

Cedric Nugteren, Gert-Jan Van den Braak, Henk Corporaal, and Henri Bal. A detailed GPU cache model based on reuse distance theory. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 37–48, 2014. DOI: 10.1109/hpca.2014.6835955. 79

NVIDIA's Next Generation CUDA Compute Architecture: Fermi. NVIDIA, 2009. 16, 46

Nvidia. NVIDIA tesla V100 GPU architecture. 2017. 27, 31

NVIDIA Corp. Pascal l1 cache. https://devtalk.nvidia.com/default/topic/1006066/pascal-l1-cache/?offset=6 70

NVIDIA Corp. Inside volta: The world's most advanced data center GPU. https://devblogs.nvidia.com/parallelforall/inside-volta/, May 2017. 1, 17, 26

NVIDIA Corporation. NVIDIA's Next Generation CUDA Compute Architecture: Kepler TM GK110, a. 13

NVIDIA Corporation. NVIDIA GeForce GTX 680, b. 16

NVIDIA Corporation. CUDA Binary Utilities, c. 16

Parallel Thread Execution ISA (Version 6.1). NVIDIA Corporation, CUDA Toolkit 9.1 ed., November 2017. 14

Lars Nyland et al. United States Patent #8,086,806: Systems and Methods for Coalescing Memory Accesses of Parallel Threads (Assignee: NVIDIA Corp.), December 2011. 71

Marek Olszewski, Jeremy Cutler, and J. Gregory Steffan. JudoSTM: A dynamic binaryrewriting approach to software transactional memory. In Proc. of the ACM/IEEE International Conference on Parallel Architecture and Compilation Techniques (PACT), pages 365–375, 2007. DOI: 10.1109/pact.2007.4336226. 96

Jason Jong Kyu Park, Yongjun Park, and Scott Mahlke. Chimera: Collaborative preemption for multitasking on a shared GPU. In Proc. of the ACM Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2015. DOI: 10.1145/2694344.2694346. 92

David A. Patterson and John L. Hennessy. Computer Organization and Design: The Hardware/- Software Interface. 2013. 78

Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Phillip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry. Base-delta-immediate compression: Practical data compression for onchip caches. In Proc. of the ACM/IEEE International Conference on Parallel Architecture and Compilation Techniques (PACT), 2012. DOI: 10.1145/2370816.2370870. 60

J. Power, A. Basu, J. Gu, S. Puthoor, B. M. Beckmann, M. D. Hill, S. K. Reinhardt, and D. A. Wood. Heterogeneous system coherence for integrated CPU-GPU systems. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), December 2013a. DOI: 10.1145/2540708.2540747. 100

Jason Power, Arkaprava Basu, Junli Gu, Sooraj Puthoor, Bradford M. Beckmann, Mark D. Hill, Steven K. Reinhardt, and David A. Wood. Heterogeneous system coherence for integrated CPU-GPU systems. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 457–467, 2013b. DOI: 10.1145/2540708.2540747. 4

M. K. Qureshi and Y. N. Patt. Utility-based cache partitioning: A low-overhead, highperformance, runtime mechanism to partition shared caches. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 423–432, 2006. DOI: 10.1109/micro.2006.49. 101

Xiaowei Ren and Mieszko Lis. Efficient sequential consistency in GPUs via relativistic cache coherence. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 625–636, 2017. DOI: 10.1109/hpca.2017.40. 72

Minsoo Rhu and Mattan Erez. CAPRI: Prediction of compaction-adequacy for handling control-divergence in GPGPU architectures. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 61–71, 2012. DOI: 10.1109/isca.2012.6237006. 44

Minsoo Rhu and Mattan Erez. The dual-path execution model for efficient GPU control flow. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 591–602, 2013a. DOI: 10.1109/hpca.2013.6522352. 49

Minsoo Rhu and Mattan Erez. Maximizing SIMD resource utilization in GPGPUs with SIMD lane permutation. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2013b. DOI: 10.1145/2485922.2485953. 46

Scott Rixner, William J. Dally, Ujval J. Kapasi, Peter Mattson, and John D. Owens. Memory access scheduling. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 128–138, 2000. DOI: 10.1109/isca.2000.854384. 77

James Roberts et al. United States Patent #8,234,478: Using Data Cache Array as a Dram Load/Store Buffer, July 2012. 75

Timothy G. Rogers, Mike O'Connor, and Tor M. Aamodt. Cache-conscious wavefront scheduling. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2012. DOI: 10.1109/micro.2012.16. 33, 78, 79, 88

Timothy G. Rogers, Mike O'Connor, and Tor M. Aamodt. Divergence-aware warp scheduling. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2013. DOI: 10.1145/2540708.2540718. 79, 90

Timothy G. Rogers, Daniel R. Johnson, Mike O'Connor, and Stephen W. Keckler. A variable warp size architecture. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2015. DOI: 10.1145/2749469.2750410. 53

Sangmin Seo, Gangwon Jo, and Jaejin Lee. Performance characterization of the NAS parallel benchmarks in OpenCL. In Proc. of the IEEE International Symposium on Workload Characterization (IISWC), pages 137–148, 2011. DOI: 10.1109/iiswc.2011.6114174. 10

A. Sethia, D. A. Jamshidi, and S. Mahlke. Mascar: Speeding up GPU warps by reducing memory pitstops. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 174–185, 2015. DOI: 10.1109/hpca.2015.7056031. 91, 92

Ankit Sethia and Scott Mahlke. Equalizer: Dynamic tuning of GPU resources for efficient execution. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), 2014. DOI: 10.1109/mi-

cro.2014.16. 86

Ryan Shrout. AMD ATI radeon HD 2900 XT review: R600 arrives. PC Perspective, May 2007. 75

Mark Silberstein, Bryan Ford, Idit Keidar, and Emmett Witchel. GPUfs: Integrating a file system with GPUs. In Proc. of the ACM Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 485–498, 2013. DOI: 10.1145/2451116.2451169. 2

Inderpreet Singh, Arrvindh Shriraman, Wilson W. L. Fung, Mike O' Connor, and Tor M. Aamodt. Cache coherence for GPU architectures. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), pages 578–590, 2013. DOI: 10.1109/hpca.2013.6522351. 72

Michael F. Spear, Virendra J. Marathe, William N. Scherer, and Michael L. Scott. Conflict detection and validation strategies for software transactional memory. In Proc. of the EATCS International Symposium on Distributed Computing, pages 179–193, Springer-Verlag, 2006. DOI: 10.1007/11864219_13. 99

Michael F. Spear, Maged M. Michael, and Christoph Von Praun. RingSTM: Scalable transactions with a single atomic instruction. In Proc. of the ACM Symposium on Parallel Algorithms and Architectures (SPAA), pages 275–284, 2008. DOI: 10.1145/1378533.1378583. 97

Michael Steffen and Joseph Zambreno. Improving SIMT efficiency of global rendering algorithms with architectural support for dynamic micro-kernels. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 237–248, 2010. DOI: 10.1109/micro.2010.45. 45

Ivan E. Sutherland. Sketchpad a Man-machine Graphical Communication System. Ph.D. thesis, 1963. DOI: 10.1145/62882.62943. 6

David Tarjan and Kevin Skadron. On demand register allocation and deallocation for a multithreaded processor, June 30, 2011. U.S. Patent App. 12/649,238. 64

Sean J. Treichler et al. United States Patent #9,098,383: Consolidated Crossbar that Supports a Multitude of Traffic Types, August 2015. 75

Dean M. Tullsen, Susan J. Eggers, Joel S. Emer, Henry M. Levy, Jack L. Lo, and Rebecca L. Stamm. Exploiting choice: Instruction fetch and issue on an implementable simultaneous multithreading processor. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 1996. DOI: 10.1145/232973.232993. 88

Rafael Ubal, Byunghyun Jang, Perhaad Mistry, Dana Schaa, and David Kaeli. Multi2Sim: A simulation framework for CPU-GPU computing. In Proc. of the ACM/IEEE International Conference on Parallel Architecture and Compilation Techniques (PACT), pages 335–344, 2012. DOI: 10.1145/2370816.2370865. 17

Aniruddha S. Vaidya, Anahita Shayesteh, Dong Hyuk Woo, Roy Saharoy, and Mani Azimi. SIMD divergence optimization through intra-warp compaction. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 368–379, 2013. DOI: 10.1145/2485922.2485954. 44, 46

Wladimir J. van der Lann. Decuda. http://wiki.github.com/laanwj/decuda/ 14

Jin Wang and Sudhakar Yalamanchili. Characterization and analysis of dynamic parallelism in unstructured GPU applications. In Proc. of the IEEE International Symposium on Workload Characterization (IISWC), pages 51–60, 2014. DOI: 10.1109/iiswc.2014.6983039. 95

Jin Wang, Norm Rubin, Albert Sidelnik, and Sudhakar Yalamanchili. Dynamic thread block launch: A lightweight execution mechanism to support irregular applications on GPUs. In Proc. of the

ACM/IEEE International Symposium on Computer Architecture (ISCA), pages 528–540, 2016a. DOI: 10.1145/2749469.2750393. 95

Jin Wang, Norm Rubin, Albert Sidelnik, and Sudhakar Yalamanchili. Laperm: Locality aware scheduler for dynamic parallelism on GPUs. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2016b. DOI: 10.1109/isca.2016.57. 96

Kai Wang and Calvin Lin. Decoupled affine computation for SIMT GPUs. In Proc. of the ACM/IEEE International Symposium on Computer Architecture (ISCA), 2017. DOI: 10.1145/3079856.3080205. 58, 61, 62

D. Wong, N. S. Kim, and M. Annavaram. Approximating warps with intra-warp operand value similarity. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2016. DOI: 10.1109/hpca.2016.7446063. 60, 61

X. Xie, Y. Liang, Y. Wang, G. Sun, and T. Wang. Coordinated static and dynamic cache bypassing for GPUs. In Proc. of the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2015. DOI: 10.1109/hpca.2015.7056023. 81

Yunlong Xu, Rui Wang, Nilanjan Goswami, Tao Li, Lan Gao, and Depei Qian. Software transactional memory for GPU architectures. In Proc. of the ACM/IEEE International Symposium on Code Generation and Optimization (CGO), pages 1:1–1:10, 2014. DOI: 10.1145/2581122.2544139. 99

Y. Yang, P. Xiang, M. Mantor, N. Rubin, L. Hsu, Q. Dong, and H. Zhou. A case for a flexible scalar unit in SIMT architecture. In Proc. of the IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2014. DOI: 10.1109/ipdps.2014.21. 47

George L. Yuan, Ali Bakhoda, and Tor M. Aamodt. Complexity effective memory access scheduling for many-core accelerator architectures. In Proc. of the ACM/IEEE International Symposium on Microarchitecture (MICRO), pages 34–44, 2009. DOI: 10.1145/1669112.1669119. 77

Hou Yunqing. Assembler for NVIDIA FERMI. https://github.com/hyqneuron/asfermi 16

Eddy Z. Zhang, Yunlian Jiang, Ziyu Guo, and Xipeng Shen. Streamlining GPU applications on the fly: Thread divergence elimination through runtime thread-data remapping. In Proc. of the ACM International Conference on Supercomputing (ICS), pages 115–126, 2010. DOI: 10.1145/1810085.1810104. 45

William K. Zuravleffand Timothy Robinson. U.S. patent #5,630,096: Controller for a synchronous dram that maximizes throughput by allowing memory requests and commands to be issued out of order, May 13, 1997. 77

作者传记

Tor M. Aamodt

Tor M. Aamodt 是英属哥伦比亚大学电气与计算机工程系的教授，自 2006 年起担任教职。他目前的研究聚焦于通用 GPU 架构和节能计算，最近包括机器学习加速器。他与研究组的学生共同开发了广泛使用的 GPGPU-Sim 模拟器。他的三篇论文被 IEEE Micro 杂志选为“Top Picks”，第四篇被选为“Top Picks”荣誉提名。他的一篇论文还被选为《ACM 通讯》的“研究亮点”。他是 MICRO 名人堂成员。他于 2012–2015 年担任 IEEE 计算机体系结构快报的副编辑，2012–2016 年担任国际高性能计算应用期刊的副编辑，是 ISPASS 2013 的程序主席，ISPASS 2014 的大会主席，并曾在众多程序委员会任职。他于 2012–2013 年在斯坦福大学计算机科学系担任访问副教授。他于 2010 年获得 NVIDIA 学术合作奖，2016–2019 年获得 NSERC 发现加速器奖，以及 2016 年谷歌教师研究奖。

Tor 在多伦多大学获得了工程科学学士、应用科学硕士和博士学位。他的博士研究工作大部分是在英特尔微架构研究实验室实习期间完成的。之后，他在 NVIDIA 从事 GeForce 8 系列 GPU 内存系统架构（“帧缓冲”）的工作——这是 NVIDIA 首款支持 CUDA 的 GPU。

Tor 是不列颠哥伦比亚省注册的专业工程师。

Wilson Wai Lun Fung

Wilson Wai Lun Fung 是三星电子奥斯汀研发中心（SARC）高级计算实验室（ACL）的架构师，致力于下一代 GPU IP 的开发。他对计算机体系结构的理论和实践方面都感兴趣。Wilson 是 NVIDIA 研究生奖学金、NSERC 研究生奖学金和 NSERC 加拿大研究生奖学金的获得者。Wilson 是广泛使用的 GPGPU-Sim 模拟器的主要贡献者之一。他的两篇论文被 IEEE Micro 杂志评选为计算机体系结构领域的“最佳选择”。Wilson 在不列颠哥伦比亚大学获得了计算机工程学士、应用科学硕士和博士学位。在博士期间，Wilson 在 NVIDIA 实习。

Timothy G. Rogers

Timothy G. Rogers 是普渡大学电气与计算机工程系的助理教授，他的研究聚焦于大规模多线程处理器设计。他热衷于探索能够同时提高程序员生产力和能源效率的计算机系统与体系结构。Timothy 是 NVIDIA 研究生奖学金和 NSERC 亚历山大 · 格雷厄姆 · 贝尔加拿大研究生奖学金的获得者。他的工作被 IEEE Micro 杂志评选为计算机体系结构领域的“最佳选择”，并被《ACM 通讯》选为“研究亮点”。在博士期间，Timothy 在 NVIDIA 研究院和 AMD 研究院实习。在攻读研究生之前，Timothy 在艺电（Electronic Arts）担任软件工程师，并在麦吉尔大学获得电气工程学士学位。